Parámetros: SPP, variantes, WFC, CSCV y WFM

AlgoProject — manual de uso. Actualizado el 2026-10-04 con `tools/manual.py`.

1. 33. Configurar los SPP en SQX — las dos tareas de permutación
2. 8. SPP — sacar a Python todo lo que deja el Sys. Param Permutation
3. 9. Diccionario — todo lo que se puede extraer, campo por campo
4. 15. SPP Ultra — ¿merece esta estrategia las 5.000 variantes?
5. 18. Variantes — fabricar las 5.000 estrategias del diseño
6. 37. El retest de las variantes para el WFC y el CSCV
7. 19. Walk forward correlation — ¿sirve de algo optimizar los parámetros?
8. 25. CSCV — ¿te fías de tu forma de elegir parámetros?
9. 39. La nube de parámetros — ¿el punto elegido es un pico de suerte?
10. 52. Superficies por mercado — ¿la región buena de parámetros es la misma en otros mercados?
11. 34. Configurar la Walk Forward Matrix en SQX
12. 9. Walk-Forward Matrix — sacar a Python todo lo que guarda el cross-check WFM
13. 14. Walk-Forward Matrix — ¿reoptimizar sirve de algo?

33. Configurar los SPP en SQX — las dos tareas de permutación

> **Ojo con el nombre.** Esta página es la mitad de SQX del paso 15: **dejar las tareas `SPP IS` y `SPP OOS` configuradas** dentro de un custom project. Sacar el perfil que producen es `09-optimizacion.pdf › 08-spp`, el diccionario de sus columnas es `09-optimizacion.pdf › 09-diccionario-spp`, y leerlo para decidir si una

> estrategia merece 5.000 variantes es `09-optimizacion.pdf › 15-sppultra` (paso 16), que incluye las superficies por pareja de parámetros (dos desplegables, eje X y eje Y, con θ_0 marcado).

Qué pregunta responde

El SPP —*System Parameter Permutation*— mueve **cada parámetro de la estrategia** arriba y abajo y vuelve a correr el backtest en cada combinación. Lo que queda es un mapa: si el beneficio sólo existe en el valor exacto que salió del generador, la estrategia está clavada en un pico y no hay nada debajo; si la vecindad entera gana dinero, hay una meseta.

Ese mapa es el `optimizationProfile.bin` que cada `.sqx` del databank de salida se lleva dentro, y es lo único que `studies/breakage/spp/` sabe leer.

⚠ El SPP es `OptProfileSysParamPermutation`, no `SequentialOptimization`

Los dos viven **en el mismo bloque** de crosschecks, los dos hablan de permutar parámetros, y sólo uno es el SPP. Encender el otro costó 47 núcleos durante 91 minutos y no produjo perfil ninguno (medido el 2026-09-22, está en `knowhow/sqx-drive/spp-task-type.md`). El comando enciende el correcto y no te deja elegir: ésa es media razón de que exista.

Cuándo lo usas, y cuándo no

Úsalo cuando ya tienes un custom project con una población cribada dentro — lo que sobrevive al MC Retest— y quieres el mapa de parámetros de cada superviviente.

No lo uses sobre una población sin cribar. Un SPP son miles de backtests **por estrategia**: es con diferencia el paso más caro de la cadena, y sobre diez mil estrategias no termina nunca.

No lo uses para filtrar. Este comando apaga los cuatro chequeos de aceptación del propio SPP (`ProfitOptPct` y compañía) a propósito: si SQX borra lo que no le gusta, `sppUltra` está leyendo la superficie de una población ya seleccionada, sin saberlo.

Las dos tareas, y por qué no se comparan entre sí

tarea	ventana	qué es
`SPP IS`	`build` (2008–2017 en XAUUSD)	la vecindad en la muestra con la que se construyó
`SPP OOS`	`oos1` (2018–2022)	la misma vecindad leída fuera de muestra

Parecen un antes y un después y **no lo son**: medido el 2026-09-19 sobre `Strategy 17.9.39`, el SPP de IS y el de OOS de la misma estrategia, con ajustes idénticos, comparten **6 tuplas de ~11.600**. SQX no recorre la misma rejilla dos veces. Son dos lecturas del mismo entorno, y ése es exactamente el motivo de que exista la fábrica de variantes del paso 16.5: para comparar hay que fabricar las tuplas uno mismo.

Antes de empezar

- El proyecto tiene que existir y ser **custom** (`sqx.projects.builder`), clonado del donante congelado, que es el que trae las tareas `SPP IS` y `SPP OOS` con esos títulos. Una tarea que no esté se reporta con `⊘` y no se inventa.
- El `.cfx` **no puede estar abierto** por una instancia de SQX: la reescribe al salir y se lleva el cambio por delante sin decir nada (regla dura 4). El comando lo comprueba y se niega.
- Costes pactados: `python3 -m core.assets <SIMBOLO>` tiene que salir en verde (regla dura 5).

No toca SQX ni ningún worker: sólo lee y reescribe un fichero. Tarda menos de un segundo.

Cómo se ejecuta

```
python3 -m core.assets XAUUSD                       # preflight, BLOQUEANTE
python3 -m sqx.projects.spp XAUUSD \
    --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_estudio/project.cfx \
    --input "MCR 8 Stress"
```

flag	obligatorio	qué hace
`<SIMBOLO>`	sí	el activo, para sus costes y sus ventanas
`--cfx`	sí	el `project.cfx` del custom project
`--input`	sí	el databank que lee la **primera** tarea. No hay valor por defecto: son los supervivientes del paso anterior, y adivinarlo es correr el paso más caro sobre la población equivocada
`--json`	no	la misma información como JSON, para encadenar

La segunda tarea lee lo que escribió la primera. Con la aceptación apagada no se cae nadie, así que la población entera llega también a la lectura de fuera de muestra.

Qué produce

Reescribe **en el sitio** el `project.cfx` que le des: las dos tareas quedan activas, con el crosscheck encendido, la rejilla escrita, las condiciones apagadas y sus databanks encadenados. No escribe nada más y no crea copia de seguridad — el `.cfx` es del proyecto, y para volver atrás se clona otra vez del donante.

Cómo se lee el resultado

```
XAUUSD  XAUUSD  <- MCR 8 Stress
  ✓ SPP IS   build 2008.01.01→2017.12.31  ±35 % en 18 pasos, tope 15,000 permutaciones, precision 1
      MCR 8 Stress → SPP IS   19 condicion(es) y 2 chequeo(s) del SPP apagados
  ✓ SPP OOS  oos1 2018.01.01→2022.12.31  ±35 % en 18 pasos, tope 15,000 permutaciones, precision 1
      SPP IS → SPP OOS   19 condicion(es) y 2 chequeo(s) del SPP apagados

El perfil lo guarda cada .sqx del databank de salida: sacalo con sqx/export/export_spp.py y leelo
con studies/breakage/spp/.
⚠ Una SPP a la vez por instalacion: el perfil ocupa decenas de GB mientras se construye.
```

Qué mirar, línea a línea:

- **`±35 % en 18 pasos`** — cuánto se mueve cada parámetro y con qué finura. Un paso son ~4 %, que es el default del dueño. Los 12 pasos sobre ±30 que trae el donante son 5 % por paso y son gruesos.
- **`tope 15,000 permutaciones`** — ⚠ la línea más importante. El donante trae `1000000001`, que es el centinela de SQX para **exhaustivo**: todas las combinaciones de todos los parámetros. Eso no es una tarea larga, es una tarea sin final. El comando lo escribe siempre, nunca lo hereda.
- **`19 condicion(es) ... apagadas`** — si sale `0` en las dos, el proyecto ya venía con ellas apagadas; merece comprobarlo, porque lo normal en un clon del donante es 19.
- **`2 chequeo(s) del SPP apagados`** — los `Eval*Check` del propio crosscheck, que son su aceptación interna. Con ellos vivos, SQX borra las estrategias cuyo mapa no le gusta.
- **`precision 1`** — el backtest corre al timeframe de la tarea, no a 1 minuto. ⚠ Esto **diverge** de `precision.default: 2` de la doctrina, y es a propósito: son miles de backtests por estrategia y a 1 minuto no acaba. Es lo que llevan las tareas del maestro. Está anotado en `OPEN.md` como decisión pendiente del dueño.

- Un `⊘` en vez del `✓` significa que el proyecto no lleva esa tarea: se clonó sin ella.
- Un `⚠ esta tarea corre ademas: …` significa que ese `.cfx` no pasó por la doctrina y va a correr otros crosschecks a la vez. Se arregla clonando con `sqx.projects.builder`.

Un ejemplo completo

```
# 1. el proyecto ya existe, con su población cribada en "MCR 8 Stress"
python3 -m core.assets XAUUSD
python3 -m sqx.projects.spp XAUUSD --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_estudio/project.cfx \
    --input "MCR 8 Stress"

# 2. correrlo en el custodio, UNA sola cosa a la vez
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=start
name=XAUUSD_estudio','custodian')"
# ... termina ...
bin/sqx-worker.sh --role custodian stop

# 3. sacar los perfiles y leerlos (pasos 15→16)
python3 -m sqx.export.export_spp --project XAUUSD_estudio --databank "SPP IS" --role custodian
python3 -m studies.breakage.spp.report --project XAUUSD_estudio --databank SPP_IS
```

Qué NO te dice

- **No dice si la estrategia es buena.** Dice cómo se comporta su vecindad de parámetros. Una estrategia mala puede tener una meseta preciosa de resultados malos.
- **No compara IS con OOS.** Las dos rejillas no coinciden (6 tuplas de 11.600); quien compare columna con columna entre los dos databanks está comparando estrategias distintas.
- **No filtra nada**, por diseño. El veredicto lo da `studies/breakage/spp/` y se aplica con `/curate`.
- **El conteo `permutations` del perfil no es el número de filas.** Medido: un perfil que dice 2.533 exporta 3.940 filas. Para saber cuántas hay de verdad, `len(results)`.

Si algo falla

lo que sale	qué significa
`el custodian tiene este proyecto abierto…`	hay una instancia levantada sobre ese `.cfx`. Párala y repite; si no, SQX se come el cambio al salir
`⊘ SPP IS NO configurada — el proyecto no lleva la tarea SPP IS`	el clon se hizo sin esa tarea. Vuelve a clonar con `--tasks Retest` y el `--only` que la incluya
`XAUUSD: … sin valor pactado`	falta un coste en `assets/symbols/`. Pregúntale al dueño (regla dura 5)
el run termina en segundos y no hay perfil	señal clásica de que se corrió sobre una variante fabricada sin parámetros reales. Un SPP de verdad tarda media hora larga por estrategia y **no dice nada** mientras tanto: la señal honesta de que trabaja es la memoria de la JVM subiendo

8. SPP — sacar a Python todo lo que deja el Sys. Param Permutation

Qué pregunta responde

Cuando pasas una estrategia por el cross-check **Sys. Param Permutation** de StrategyQuant X, SQX mueve un poco cada parámetro de la estrategia, la vuelve a backtestear miles de veces y te enseña un panel con histogramas. Ese panel es lo único que ves dentro del programa, y no se puede exportar desde el menú.

Este comando saca **todo ese panel a CSV**: cuántas permutaciones se probaron, cuántas fueron rentables, la mediana de cada una de las 135 métricas de SQX, el valor original de la estrategia y el histograma completo de cada métrica, bin a bin. Y si SQX guardó el detalle, además **una fila por permutación** con sus parámetros y sus 152 estadísticos.

La pregunta de fondo es la del sobreajuste. Si tu estrategia gana 24.269 € y la permutación mediana gana 1.858 €, tu resultado **no es lo normal de esa familia de parámetros: es el pico**. Ese cociente —la columna `orig_over_median` — es el número que buscas.

Cuándo lo usas, y cuándo no

Úsalo cuando ya has corrido la tarea SPP dentro de SQX y las estrategias están volcadas en un databank (`SPP IS`, `SPP OOS`...).

Cuándo NO sirve:

- Si el databank no ha pasado por SPP, los `.sqx` no llevan perfil dentro y el comando no encuentra nada. No es un error: sencillamente esas estrategias no tienen ese dato.
- **No te da los trades de las permutaciones, y nunca podrá dártelos.** No es una limitación del exportador: es que SQX no los escribe. Cada permutación se guarda como su cadena de parámetros más un bloque `SQStats`, y ese bloque solo admite seis tipos de registro, todos numéricos —enteros, flotantes y longs—; no hay ninguna vía por la que se escriba una lista de órdenes. Las operaciones que sí tienes son las del backtest principal, y ésas se sacan con `python3 -m sqx.export.export_trades --project XAUUSD --databank "SPP IS" --symbol XAUUSD_DukasM1_Infinox`.
- **El detalle permutación a permutación solo existe si la casilla estaba quitada al correr el SPP.** Es *Settings → Performance → "Don't store data for 3D charts in Optimization profile"*. Con la casilla puesta SQX tira esos resultados al guardar y deja únicamente medianas e histogramas — y las estrategias guardadas así no lo recuperan aunque la quites después: hay que **volver a correr el SPP**. Con la casilla quitada el `.sqx` engorda mucho: los cinco de `SPP IS` pasaron de 125 KB a 2,2 MB cada uno.

Antes de empezar

- El databank tiene que estar **volcado a disco**. Si la carpeta está vacía, abre SQX, entra en el databank y deja que sincronice.
- **No hace falta arrancar nada.** El comando lee los `.sqx` directamente, no usa SQX ni el worker, y puedes lanzarlo con la GUI abierta sin ningún riesgo.

Cómo se ejecuta

```
python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	sí	nombre del databank, con sus espacios y entre comillas si los tiene

Tarda un par de segundos por estrategia. No toca SQX.

```
$ python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"
5 profiles -> /home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/SPP_IS/2026-09-10/spp
  runs.csv         5 rows
  metrics.csv      675 rows
  histograms.csv   13500 rows
  permutations.csv 21205 rows
  permutation_params.csv 213642 rows
```

Qué produce

En `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/spp/`:

archivo	qué es
`runs.parquet`	una fila por estrategia: permutaciones, cuántas ganaron y perdieron, beneficio medio y máximo, desviación típica y los parámetros que se movieron
`metrics.parquet`	una fila por estrategia y métrica: la mediana de las permutaciones, el valor original y su cociente
`histograms.parquet`	una fila por estrategia, métrica y bin: la frecuencia, y qué bin contiene la mediana y cuál el valor original
`spp.parquet`	**solo si SQX guardó el detalle** — **una fila por permutación**: `strategy`, `permutation`, una columna por parámetro que se movió (vacía en las estrategias que no mueven ese parámetro) y después los 152 estadísticos. La fila `permutation = -1` es la estrategia original, contra la que se comparan las demás. Desde el 23-09-2026 sustituye a `permutations.csv` + `permutation_params.csv`: 49 MB de CSV son 6 MB de parquet, y quien necesita cuatro columnas lee cuatro columnas
`manifest.json`	qué se leyó, cuántos perfiles había y con qué versión del código

Las carpetas van por fecha: una exportación nueva no borra la anterior.

Cómo se lee el resultado

```
metrics.csv  ·  Strategy 1.19.29  ·  4309 permutaciones

metric                 median          orig  orig/median
NetProfit             1544.68      24269.22        15.71
ProfitFactor             1.02          1.32         1.29
SharpeRatio              0.04          0.64        16.00
ReturnDDRatio            0.17          3.86        22.71
DrawdownPct             12.91          5.52         0.43
NumberOfTrades         754.00        763.00         1.01
WinningPct              49.09         60.16         1.23
RExpectancy              0.01          0.13        11.02
Stability                0.48          0.81         1.69
RSquared                 0.68          0.92         1.35
SQN                      0.07          0.77        11.00
CAGR                     0.15          2.20        14.67

histograms.csv  ·  NetProfit  ·  20 bins

 bin        edge   freq
   0   -91726.57      3
   1   -83065.79      9
   2   -75242.62     11   #
   3   -67184.05     12   #
   4   -59379.72     28   #
   5   -52520.72     21   #
   6   -43520.67     33   ##
   7   -35642.23     57   ###
   8   -27890.06     73   ####
   9   -20005.67    221   ###########
  10   -12177.81    534   ##########################
  11    -4371.00    626   ###############################
  12     3455.02    706   ################################### <- mediana
  13    11275.22    488   ########################
  14    19064.38    546   ###########################
  15    26881.34    551   ############################ <- original
  16    34665.00    290   ##############
  17    42304.19     94   #####
  18    47129.08      5
  19    50364.92      1
```

Arriba, `metrics.csv` para una estrategia. **La columna que decide es `orig_over_median`:**

- **Cerca de 1** — tu estrategia rinde como la permutación típica. El resultado no depende de haber acertado los parámetros exactos. Es lo que quieres ver.
- **Muy por encima de 1** — el resultado vive en un pico. En el ejemplo, `NetProfit` sale 13 veces la mediana y `ReturnDDRatio` 15 veces: mover los parámetros un punto destruye la estrategia.
- **Por debajo de 1 en una métrica de riesgo** (`DrawdownPct` sale 0,45) — el original sufre menos drawdown que la permutación mediana, que es la misma señal vista del revés.

Abajo, `histograms.csv` para `NetProfit`. Las 4.301 permutaciones repartidas en 20 bins, con la mediana en el bin 13 y el valor original en el 15. Lo que importa es **cuánta masa queda a la izquierda del cero**: aquí unas 1.300 de 4.301 permutaciones pierden dinero.

`runs.csv` resume lo mismo en una línea. `profitable_pct` es el porcentaje de permutaciones con beneficio; en las cinco estrategias de `SPP IS` va del 54 % al 92 %, y la de 92 % es la que menos depende de sus parámetros.

Y con el detalle guardado, lo que de verdad quieres

```
permutations.csv  ·  Strategy 1.19.29  ·  4309 permutaciones x 152 estadisticos

metric              original     mediana          p5         p95  percentil
NetProfit           24269.22     1544.68   -30311.12    30635.28      87.1%
ProfitFactor            1.32        1.02        0.72        1.52      86.9%
SharpeRatio             0.64        0.04       -0.69        0.83      87.3%
ReturnDDRatio           3.86        0.17       -0.88        6.28      88.2%
DrawdownPct             5.52       12.91        1.80       38.83      19.1%
NumberOfTrades        763.00      754.00       35.00     1768.00      51.1%
Stability               0.81        0.48       -0.81        0.85      90.0%
RSquared                0.92        0.68        0.01        0.95      86.7%

permutation_params.csv  ·  NetProfit mediano por valor de un parametro

   LWMAPeriod1      n  NetProfit med.
             8    325          291.06
             9    273        -3075.79
            10    616         2690.02
            11    325         4434.71
            12    319         2448.69
            13    270        -3600.57
            14    383         3292.84
            15    258        -2834.89
            16    465         5517.88
            17    461         6548.57
            18    328         -243.86
            19    287         -417.04
```

Arriba, lo que las medianas no te podían dar: **el percentil**. `NetProfit` original está en el percentil 87 de sus propias permutaciones, y `Stability` en el 90. Y los cuantiles: el 5 % peor de las permutaciones pierde 30.311 €. Con `metrics.csv` solo sabías la mediana; aquí tienes toda la distribución y puedes pedir el percentil que quieras.

Abajo, lo que **ninguna** parte del panel de SQX enseña: **qué parámetro es el que hace daño**. Agrupando las 4.309 permutaciones por el valor de `LWMAPeriod1`, el `NetProfit` mediano salta de −3.601 € a +6.549 € según el valor. Eso es una superficie de parámetros, y es lo que decide si la estrategia tiene una meseta donde vivir o un pico. Se hace en tres líneas:

```
w = q[q.strategy == "Strategy 1.19.29"].pivot(index="permutation", columns="parameter",
values="value")
j = w.join(p[p.permutation >= 0].set_index("permutation")["NetProfit"])
print(j.groupby("LWMAPeriod1").NetProfit.median())
```

Un ejemplo completo

```
$ python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"
5 profiles -> /home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/SPP_IS/2026-09-10/spp
  runs.parquet     5 rows
  metrics.parquet  675 rows
  histograms.parquet 13500 rows
  spp.parquet      21205 rows
```

Y en Python:

```
import pandas as pd
from pathlib import Path

d = Path.home() / "Desktop/AlgoData/raw/XAUUSD/SPP_IS/2026-09-10/spp"
m = pd.read_parquet(d / "metrics.parquet")

frag = m[m.metric == "NetProfit"].set_index("strategy")["orig_over_median"]
print(frag.sort_values(ascending=False))
```

Para leer un perfil suelto, sin exportar nada:

```
from pathlib import Path
from core import optprofile

p = optprofile.read(Path.home() / "Desktop/SQX/user/projects/XAUUSD/databanks/SPP IS/Strategy
1.19.29.sqx")
print(p["permutations"], p["profitable_pct"], p["params"])
print(p["medians"]["NetProfit"], p["orig"]["NetProfit"])

if p["permutation_results"]:                        # la casilla estaba quitada
    print(p["results"][0]["params"])
    print(p["results"][0]["stats"]["NetProfit"])
```

Qué NO te dice

- **No es una prueba fuera de muestra.** Todas las permutaciones se corren sobre el mismo tramo de datos que la estrategia original. Un SPP impecable sigue siendo compatible con una estrategia que no funcione en el futuro; lo que descarta es una cosa distinta —que el resultado dependa de haber clavado los parámetros—, no que el edge sea real.
- **Los histogramas vienen ya agregados por SQX**, en 20 bins que él eligió. Si el databank no guardó el detalle, ésa es toda la distribución que hay: no se puede pedir otro percentil ni cruzar dos métricas permutación a permutación. Con el detalle guardado, `spp.parquet` deja hacer las dos cosas y los histogramas sobran.
- **En `spp.parquet`, 34 de los 152 estadísticos salen como `stat:<tipo>:<índice>`.** SQX los guarda por su posición en un array, sin nombre. Los 118 nombrados se calibraron cruzando el resultado original de 34 perfiles contra la tabla de medianas del mismo fichero, que sí lleva nombre; los 34 restantes valen 0 en todas las estrategias, que es justo por lo que no hubo con qué distinguirlos. El inventario completo, campo a campo, está en el capítulo 9.

- **28 de las 135 métricas de `metrics.parquet` salen como `id:<número>`.** SQX guarda las métricas bajo un hash de su nombre que ya no se puede deshacer, así que los nombres se reconstruyeron comparando valores contra una exportación de databank de 50 estrategias. Esas 28 valen exactamente 0 en todas las estrategias de esta instalación, así que no había con qué distinguirlas — y por eso mismo tampoco aportan nada. Otras seis llevan interrogación (`Exposure?` / `ExposurePosition?`, `Outlier?` / `Outlier2?`, `CalmarRatio?` / `AnnualPctReturnDDRatio?`): son tres parejas cuyos dos miembros valen lo mismo en las 225 estrategias con perfil de esta máquina, así que se sabe **de qué pareja es cada columna, pero no cuál de las dos**. Las 101 restantes están verificadas una a una.

Si algo falla

- **`0 profiles`** — el databank no ha pasado por SPP, o los `.sqx` no están en disco. Comprueba la carpeta `user/projects/<proyecto>/databanks/<databank>/`.
- **`KeyError: 'optimizationProfile.bin'`** no puede salir: el comando salta los `.sqx` que no lo llevan. Si sale otra cosa al leer un `.sqx`, es que ese archivo está corrupto; el traceback dice cuál.

9. Diccionario — todo lo que se puede extraer, campo por campo

Esta página no explica cómo se corre nada: eso está en el capítulo 8. Aquí está el **inventario completo** de lo que sale de una estrategia que ha pasado por el Sys. Param Permutation, con el nombre exacto de cada campo y qué es. Es la página que se tiene abierta al lado mientras se escribe el `pandas`.

Todo lo que aparece aquí se lee del `.sqx` directamente. No hace falta SQX abierto ni el worker.

De dónde sale cada cosa

fichero	una fila por	lo llena
`runs.parquet`	estrategia	el resumen de la tirada de permutaciones
`metrics.parquet`	estrategia × métrica	la tabla mediana / valor original del panel
`histograms.parquet`	estrategia × métrica × bin	los histogramas que dibuja el panel
`spp.parquet`	estrategia × permutación	**solo si la casilla 3D estaba quitada**: los parámetros en columnas y después los estadísticos. Hasta el 23-09-2026 eran dos CSV, `permutations.csv` (estadísticos) y `permutation_params.csv` (parámetros en forma larga); las secciones 4 y 5 describen las columnas, que son las mismas
`trades.parquet`	operación	el backtest principal, vía `export_trades.py`
la librería de barras (`03-datos-costes-y-registro.pdf › 13-barras`)	barra	las velas sobre las que operó

---

1. `runs.parquet` — la tirada entera en una línea

campo	qué es
`strategy`	nombre de la estrategia, tal y como aparece en el databank
`permutations`	cuántas permutaciones se probaron
`profitable`	cuántas acabaron con beneficio positivo
`losing`	cuántas acabaron en pérdida
`zero`	cuántas acabaron exactamente en cero, normalmente porque no llegaron a operar
`profitable_pct`	`profitable` en porcentaje. **La cifra de cabecera del SPP**: cuánto del entorno de parámetros gana
`avg_profit`	beneficio medio de todas las permutaciones, en divisa de la cuenta
`top_profit`	el beneficio de la mejor permutación
`stdev`	desviación típica del beneficio entre permutaciones. Mide lo mucho que se mueve el resultado al mover los parámetros
`uniform_changes`	contador interno de SQX sobre lo uniforme que es la distribución. No está documentado qué escala usa
`parameters`	los nombres de los parámetros que se permutaron, separados por espacio

Hay dos campos más que el lector de Python devuelve y que no se escriben al CSV porque no son tabulares: `last_count` (repite `permutations`) y `stdev_computed` (siempre 0 en esta instalación).

2. `metrics.parquet` — la tabla del panel

campo	qué es
`strategy`	nombre de la estrategia
`metric`	nombre de la métrica; los 135 posibles están en la tabla grande de más abajo
`median`	el valor **mediano** de esa métrica entre todas las permutaciones
`orig`	el valor de la estrategia original, la que tú tienes guardada
`orig_over_median`	`orig / median`. **La columna que decide.** Cerca de 1 el resultado no depende de los parámetros; muy por encima, vive en un pico

3. `histograms.parquet` — la distribución tal y como la agrupó SQX

campo	qué es
`strategy`	nombre de la estrategia
`metric`	métrica a la que pertenece el histograma
`bin`	número de bin, de 0 a 19. Siempre son 20
`edge`	etiqueta del borde inferior del bin. **Puede venir vacía**: SQX se salta etiquetas para que quepan en el eje
`frequency`	cuántas permutaciones cayeron en ese bin
`is_median`	`True` en el bin donde cae la mediana
`is_orig`	`True` en el bin donde cae el valor original

4. `spp.parquet`, los estadísticos — una fila por permutación

campo	qué es
`strategy`	nombre de la estrategia
`permutation`	número de permutación desde 0. **La fila `-1` es la estrategia original**, para comparar contra ella sin buscarla aparte
los otros 152	los estadísticos, uno por columna. Los nombrados son los de la tabla grande; los demás salen como `stat:<tipo>:<índice>`

5. `spp.parquet`, los parámetros — qué valores le tocaron a cada permutación (una columna por parámetro; antes, forma larga con `parameter` y `value`)

campo	qué es
`strategy`	nombre de la estrategia
`permutation`	mismo número que en `permutations.csv`; se cruzan por `strategy` + `permutation`
`parameter`	nombre del parámetro, tal y como lo llama la estrategia (`LWMAPeriod1`, `ExitAfterBars1`...)
`value`	el valor que SQX le puso en esa permutación

Los nombres de parámetro **cambian de una estrategia a otra**: son los de sus propios bloques. Por eso van en formato largo y no en columnas.

---

6. Las 135 métricas

Esto es el catálogo entero de columnas de databank de tu instalación, que es exactamente lo que el SPP guarda por métrica. Los nombres cortos son los que usan `metrics.csv` y `permutations.csv`; el nombre en SQX es el que ves en la GUI.

- **unidad** — `€` divisa de la cuenta · `%` porcentaje · `pips` · `ratio` · `entero` · `texto` · `fecha` · `–` no declarada en el código de la columna.
- **mejor** — hacia dónde considera SQX que es mejor: `↑` más alto, `↓` más bajo, `≈` ni una cosa ni otra, `–` no declarado.
- **por permutación** — si además aparece en `permutations.csv`, es decir si se puede mirar permutación a permutación y no solo como mediana.

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`AHPR`	AHPR	Rendimiento medio por periodo de tenencia (Arithmetic Holding Period Return), calculado sobre los retornos anuales.	ratio	↓	sí
`ActualDD`	Actual Drawdown	Drawdown que había abierto durante la ÚLTIMA operación del backtest. No es el máximo, es el del final.	—	—	sí
`ActualDrawdownPct`	Actual Drawdown / Max DD	El drawdown de la última operación dividido por el drawdown máximo. Cerca de 1 significa que el backtest termina en su peor momento.	—	—	sí
`AddMarketsNetProfitMedian`	AddMarketsNetProfitMedian	Mediana del beneficio neto en los mercados adicionales del retest cruzado. Vale 0 si la estrategia no se retesteó en otros mercados.	—	—	sí
`AddMarketsProfitFactorMedian`	AddMarketsProfitFactorMedian	Mediana del profit factor en los mercados adicionales del retest cruzado.	—	—	sí
`AddMarketsRetDDMedian`	AddMarketsRetDDMedian	Mediana del Ret/DD en los mercados adicionales del retest cruzado.	—	—	sí
`AddMarketsSharpeRatioMedian`	AddMarketsSharpeRatioMedian	Mediana del Sharpe en los mercados adicionales del retest cruzado.	—	—	sí
`AmbiguousTrades`	Ambiguous Trades	Operaciones que abren y cierran en la misma barra. Son ambiguas porque el backtest no puede saber qué pasó primero dentro de la barra.	entero	↓	sí
`AmbiguousTradesPct`	Ambiguous Trades %	Las anteriores como porcentaje del total de operaciones. Es la	%	↓	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
		métrica de fiabilidad del backtest, no de rentabilidad.			
`AnnualPctReturn`	Annual % Return	Rentabilidad porcentual anualizada sobre los años de datos.	%	↑	sí
`AnnualPctReturnDDRatio`	CAGR/Max DD %	CAGR dividido por el drawdown máximo en porcentaje.	ratio	↑	sí
`AvgAbsTrade`	Avg. Abs Trade	Beneficio absoluto medio por operación, contando las perdedoras en valor absoluto.	€	↑	sí
`AvgBarsInTrade`	Avg. Bars in Trade	Barras que dura una operación de media.	ratio	↓	sí
`AvgBarsLoss`	Avg. Bars Loss	Barras que dura de media una operación perdedora.	ratio	↓	sí
`AvgBarsWin`	Avg. Bars Win	Barras que dura de media una operación ganadora.	ratio	↑	sí
`AvgConsecLosses`	Avg Consec. Losses	Longitud media de las rachas de pérdidas.	ratio	↓	sí
`AvgConsecWins`	Avg Consec. Wins	Longitud media de las rachas de ganancias.	ratio	↑	sí
`AvgDrawdown`	Avg. Drawdown	Drawdown medio de todos los que hubo, no solo el máximo.	€	↓	sí
`AvgLoss`	Avg. Loss	Pérdida media de las operaciones perdedoras.	€	↓	sí
`AvgParametersStability`	Avg. Parameters Stability	Estabilidad media de los parámetros en el Walk Forward Matrix. Solo tiene valor si se corrió WF.	ratio	↑	no
`AvgPctDrawdown`	Avg. % Drawdown	Drawdown medio expresado en porcentaje.	%	↓	sí
`AvgPctProfitPerYear`	Avg. % Profit Per Year	Beneficio porcentual medio por año.	%	↑	sí
`AvgProfitPerDay`	Avg. Profit Per Day	Beneficio medio por día de trading.	€	↑	sí
`AvgProfitPerMonth`	Avg. Profit Per Month	Beneficio medio por mes.	€	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`AvgProfitPerYear`	Avg. Profit Per Year	Beneficio medio por año.	€	↑	sí
`AvgTrStddevRatio`	Avg. trade / StdDev ratio	Operación media dividida por la desviación típica de las operaciones. Es el Sharpe por operación, sin anualizar.	—	—	sí
`AvgTrade`	Avg. Trade	Beneficio medio por operación (expectativa en dinero).	€	↑	sí
`AvgTradesPerDay`	Avg. Trades Per Day	Operaciones por día.	ratio	↓	sí
`AvgTradesPerMonth`	Avg. Trades Per Month	Operaciones por mes.	ratio	↓	sí
`AvgTradesPerYear`	Avg. Trades Per Year	Operaciones por año.	ratio	↓	sí
`AvgWin`	Avg. Win	Ganancia media de las operaciones ganadoras.	€	↑	sí
`BacktestDuration`	Backtest Duration (s)	Segundos que tardó el backtest en correr. Es un dato de la máquina, no de la estrategia.	texto	↓	no
`BestWF`	Best WF	Mejor resultado del Walk Forward. Vacío si no se corrió WF.	texto	↑	sí
`BiggestMAE`	Biggest MAE	La peor excursión adversa de todas las operaciones: cuánto llegó a estar en contra la operación que más sufrió.	€	↑	sí
`CAGR`	CAGR	Tasa de crecimiento anual compuesta.	%	↑	sí
`CalmarRatio`	CalmarRatio	Rentabilidad anualizada dividida por el drawdown máximo.	ratio	↑	sí
`Commission`	Commission/Swap ($)	Comisiones y swap pagados en total, en divisa de la cuenta.	€	↓	sí
`Complexity`	Complexity	Medida de complejidad de la estrategia que calcula SQX a partir de su estructura.	entero	↓	sí
`DateGenerated`	Date generated	Fecha en que SQX generó la estrategia.	fecha	↓	no

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`DateLastModified`	Date last modified	Fecha del último cambio o retest de la estrategia.	fecha	↓	no
`DegreesOfFreedom`	Degrees of freedom	Grados de libertad: operaciones menos parámetros. Cuanto más bajo, más fácil es que el resultado sea casualidad.	entero	↓	sí
`DoFRatio`	DoF Ratio	Operaciones divididas por número de parámetros optimizados. Cuántos datos hay por cada grado de libertad.	ratio	↑	sí
`Drawdown`	Drawdown	Máxima caída desde un máximo de la curva de balance, en divisa de la cuenta.	€	↓	sí
`DrawdownPct`	Max DD %	El drawdown máximo en porcentaje.	%	↓	sí
`DrawdownPctOnInitial`	DD% of Initial	El drawdown máximo como porcentaje del depósito inicial, no del pico.	—	—	sí
`DrawdownPips`	Max DD pips	El drawdown máximo medido en pips.	pips	↓	sí
`EdgeDecayRatio`	Edge Ratio Decay	Puntuación 0-100 de calidad fuera de muestra construida sobre cuatro pilares. Columna propia, no estándar de SQX.	ratio	↑	sí
`EdgeRatioInPips`	Edge Ratio	Edge ratio en pips: relación entre excursión favorable y adversa. La definición exacta no viene documentada en esta instalación.	—	—	sí
`Efficiency`	Efficiency	Beneficio neto dividido por beneficio bruto: qué parte de lo que ganó se quedó después de las pérdidas.	—	—	sí
`EntryIndicators`	Entry indicators	Lista de indicadores que usan las condiciones de entrada. Texto.	texto	↑	no

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`EquityAngle`	EquityAngle	Ángulo de la recta de regresión de la curva de equity.	ratio	↑	sí
`EquitySlope`	EquitySlope	Lo recta que es la curva de equity multiplicado por lo empinada que es.	ratio	↑	sí
`ExitIndicators`	Exit indicators	Lista de indicadores que usan las condiciones de salida. Texto.	texto	↑	no
`ExitQuality`	Exit quality	Beneficio total dividido por el MFE total: cuánto del movimiento favorable disponible llegó a capturar.	ratio	↑	sí
`Expectancy`	Expectancy	Expectativa por operación.	ratio	↑	sí
`Exposure`	Exposure	Barras con posición abierta dividido por barras totales de la muestra.	%	↓	sí
`ExposurePosition`	Exposure Position	Lo mismo que Exposure, en la versión que usa el motor Stockpicker considerando el número de posiciones abiertas.	%	↓	sí
`FiltersResult`	Filters result	PASSED o FAILED según los filtros de aceptación del databank. Texto.	texto	↑	no
`Fitness`	Fitness	Valor de fitness con el que el algoritmo genético ordenó la estrategia.	ratio	↑	sí
`GiniOOS`	Gini Coeff	Coeficiente de Gini del P&L por operación: 0 es que todas aportan lo mismo, 1 es que una sola se lo lleva todo. Columna propia.	ratio	↓	sí
`GrossLoss`	Gross loss	Suma de todas las operaciones perdedoras.	€	↓	sí
`GrossProfit`	Gross profit	Suma de todas las operaciones ganadoras.	€	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`InitialDeposit`	Initial deposit	Depósito inicial del backtest.	ratio	↑	no
`KellyFormula`	Kelly formula	Fracción de Kelly: qué porcentaje del capital sugeriría arriesgar la fórmula con este win rate y payout.	—	—	sí
`LongestTrade`	Longest trade (days)	Duración en días de la operación más larga.	entero	↓	sí
`MagicNumber`	Magic number	Identificador que SQX pone a la estrategia para el broker. No es una métrica.	texto	↓	no
`MaxConsecLosses`	Max Consec. Losses	Racha más larga de pérdidas seguidas.	entero	↓	sí
`MaxConsecWins`	Max Consec. Wins	Racha más larga de ganancias seguidas.	entero	↑	sí
`MaxIntradayDrawdown`	Max Intraday Drawdown	Drawdown máximo mirando también dentro del día, no solo balances de cierre.	€	↓	sí
`MaxLoss`	Max Loss	La peor operación individual.	€	↓	sí
`MaxNewHighDuration`	Max Drawdown Duration	Días que tardó como mucho en hacer un nuevo máximo de equity. Es la duración del drawdown más largo.	entero	↓	sí
`MaxProfit`	Max Profit	La mejor operación individual.	€	↑	sí
`MaxTSIntradayDrawdown`	Max TS Intraday Drawdown	Drawdown intradía máximo en la versión TradeStation.	€	↑	sí
`MiniEquityChart`	Mini equity chart	La miniatura de la curva de equity que pinta la GUI. No es un número.	texto	↓	no
`NSymmetry`	NSymmetry	Simetría de beneficio entre largos y cortos, con la variante de que si un lado gana y el otro pierde devuelve −1 en vez de 0.	%	↑	no
`NetProfit`	Net profit	Beneficio neto total en divisa de la cuenta. Es la métrica de referencia del SPP.	€	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`NetProfitInPct`	Net profit in %	El beneficio neto en porcentaje.	%	↑	sí
`NetProfitInPips`	Net profit in pips	El beneficio neto en pips.	pips	↑	sí
`Note`	Note	Nota escrita a mano en la estrategia. Texto.	texto	↓	no
`NumberOfCanceled`	# of canceled	Órdenes pendientes que se cancelaron sin llegar a ejecutarse.	entero	≈	no
`NumberOfLosses`	# of losses	Número de operaciones perdedoras.	entero	↓	sí
`NumberOfProfits`	# of profits	Número de operaciones ganadoras.	entero	↑	sí
`NumberOfTrades`	# of trades	Número total de operaciones. En el SPP es la métrica de control: si una permutación baja de golpe, es que la regla dejó de dispararse.	entero	≈	sí
`OpenDrawdown`	Open Drawdown	Drawdown calculado sobre el balance abierto más el MAE, es decir contando lo que se sufrió sin cerrar.	€	↓	sí
`OpenDrawdownPct`	Open Drawdown %	El anterior en porcentaje.	%	↓	sí
`Outlier`	Outlier1	Peso de la operación atípica, SIN filtrar operaciones con el mismo P&L.	—	—	sí
`Outlier2`	Outlier1	Peso de la operación atípica, filtrando las operaciones con el mismo P&L.	—	—	sí
`ParameterCount`	Param Count	Parámetros realmente buscados: periodos, constantes, niveles de entrada y salidas usadas. Excluye MagicNumber, variables de señal, Shift y salidas sin usar. Columna propia.	entero	↓	sí
`Parameters`	Parameters	Volcado en texto de los parámetros de la	texto	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
		estrategia.			
`PayoutRatio`	Payout ratio	Ganancia media dividida por pérdida media.	ratio	↑	sí
`PriceIndicators`	Price indicators	Indicadores usados en los niveles de precio de entrada (stop o limit). Texto.	texto	↑	no
`ProbSharpeRatio`	PSR	Sharpe probabilístico (0-1): probabilidad de que el Sharpe verdadero sea mayor que cero. Columna propia.	ratio	↑	sí
`ProfitFactor`	Profit factor	Beneficio bruto dividido por pérdida bruta.	ratio	↑	sí
`ProfitableMonths`	Profitable Months	Meses cerrados en positivo.	entero	↑	sí
`ProfitableMonthsPct`	% Profitable Months	Porcentaje de meses en positivo.	%	↑	sí
`RExpectancy`	R Expectancy	Expectativa medida en R, es decir en múltiplos del riesgo por operación.	ratio	↑	sí
`RExpectancyScore`	R Expectancy Score	R Expectancy ponderada por el número de operaciones, para que una expectativa alta con pocas operaciones no gane.	ratio	↑	sí
`RSquared`	RSquared	R^2 de la regresión sobre la curva de equity: lo recta que es.	ratio	↑	sí
`RecoveryFactor`	RecoveryFactor	Beneficio neto dividido por el drawdown máximo.	ratio	↑	sí
`ResultsName`	Results Name	Nombre del resultado. Texto.	texto	↑	no
`ReturnDDRatio`	Ret/DD Ratio	Retorno dividido por drawdown.	ratio	↑	sí
`ReturnOpenDDRatio`	Ret/OpenDD Ratio	Retorno dividido por el drawdown abierto.	ratio	↑	sí
`SQN`	SQN	System Quality Number de Van Tharp.	ratio	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`SQNScore`	SQN Score	SQN ponderado por el número de operaciones.	ratio	↑	sí
`SharpeRatio`	Sharpe Ratio	Ratio de Sharpe anualizado.	ratio	↑	sí
`SlippageInMoney`	Slippage ($)	Deslizamiento total pagado, en divisa de la cuenta.	€	↑	no
`SlopeRatio`	Slope Ratio	Pendiente fuera de muestra dividida por la pendiente en muestra (beneficio medio por operación OOS frente a IS). Columna propia.	ratio	↑	sí
`SortinoRatio`	Sortino Ratio	Ratio de Sortino: como el Sharpe pero castigando solo la volatilidad a la baja.	—	—	sí
`Stability`	Stability	Lo recta Y lo empinada que es la curva de equity, en una sola cifra.	ratio	↑	sí
`StabilitySQ3`	Stability SQ3	La misma estabilidad calculada como la calculaba StrategyQuant 3.	ratio	↑	sí
`Stagnation`	Stagnation	Días que la equity pasó estancada sin hacer nuevo máximo.	entero	↓	sí
`StagnationPct`	% Stagnation	El estancamiento como porcentaje de los días totales.	%	↓	sí
`StandardDev`	StandardDev	Desviación típica del P&L de las operaciones.	ratio	↓	sí
`Symbol`	Symbol	Símbolo sobre el que se hizo el test. Texto.	texto	↓	no
`Symmetry`	Symmetry	Simetría del beneficio entre el lado largo y el corto.	%	↑	no
`TRLRatio`	TRL Ratio	Operaciones reales divididas por el mínimo de operaciones necesarias para afirmar con 95 % de confianza que el Sharpe es mayor que cero. Columna propia.	ratio	↑	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`TSIndex`	TS Index	Índice TradeStation.	ratio	↑	sí
`TSWinLossRatio`	TS Win/Loss ratio	Ratio ganadoras/perdedoras en la definición de TradeStation.	ratio	↑	sí
`TimeFrame`	TimeFrame	Timeframe del test. Texto.	texto	↑	no
`TotalDataDays`	Total Data Days	Días que abarcan los datos.	entero	↓	sí
`TotalDataMonths`	Total Data Months	Meses que abarcan los datos.	entero	↓	sí
`TotalDataYears`	Total Data Years	Años que abarcan los datos.	entero	↓	sí
`TotalMFE`	Total MFE	Suma del MFE de todas las operaciones: todo el movimiento favorable que hubo disponible.	€	↓	sí
`TotalTradingDays`	Total Trading Days	Días en los que hubo actividad.	entero	↓	sí
`TotalTradingMonths`	Total Trading Months	Meses en los que hubo actividad.	entero	↓	sí
`TotalTradingYears`	Total Trading Years	Años en los que hubo actividad.	entero	↓	sí
`TradesSymmetry`	Trades Symmetry	Simetría del NÚMERO de operaciones entre largos y cortos.	%	↑	no
`UlcerIndex`	Ulcer Index %	Ulcer Index: mide la profundidad y la duración de los drawdowns a la vez.	—	—	sí
`UlcerPerformanceIndex`	Ulcer Performance Index	Ulcer Performance Index anualizado: retorno por unidad de Ulcer Index.	ratio	↑	sí
`WinLossRatio`	Win/Loss ratio	Ganadoras dividido por perdedoras.	ratio	↑	sí
`WinningPct`	Winning Percent	Porcentaje de operaciones ganadoras.	%	↑	sí
`WorstParametersStability`	Worst Parameters Stability	La peor estabilidad de parámetros de todos los Walk Forward. Solo tiene valor si se corrió WF.	ratio	↑	no
`WorstYearProfit`	Worst Year Profit	Beneficio del peor año.	€	↓	sí

campo	nombre en SQX	qué es	unidad	mejor	por permutación
`ZProbability`	ZProbability	Probabilidad asociada al Z-Score: si las rachas son o no compatibles con el azar.	ratio	↑	sí
`ZScore`	ZScore	Z-Score de las rachas: mide si ganancias y pérdidas se agrupan más de lo que cabría esperar por azar.	ratio	↑	sí

> **Aviso sobre `Outlier` y `Outlier2`:** SQX exporta las dos con la misma etiqueta, `Outlier1`. No es un fallo del exportador, es lo que trae la columna. Distínguelas por el nombre corto, nunca por la etiqueta.

Las 21 que no aparecen por permutación

`AvgParametersStability`, `BacktestDuration`, `DateGenerated`, `DateLastModified`, `EntryIndicators`, `ExitIndicators`, `FiltersResult`, `InitialDeposit`, `MagicNumber`, `MiniEquityChart`, `Note`, `PriceIndicators`, `ResultsName`, `Symbol`, `TimeFrame`, `WorstParametersStability` son metadatos o texto: describen la estrategia, no su resultado, así que no cambian de una permutación a otra y SQX no los repite.

`NSymmetry`, `Symmetry`, `TradesSymmetry`, `NumberOfCanceled` y `SlippageInMoney` sí son numéricas y sí tienen su hueco guardado — pero valen 0 en todas las estrategias de esta instalación, y por eso quedaron entre las que no se pudieron nombrar (siguiente apartado). Están, no se sabe en qué hueco.

---

7. Lo que solo existe por permutación

Cuatro estadísticos aparecen en `permutations.csv` y no son columna de databank, así que no salen en `metrics.csv`:

campo	qué es
`DataLength`	número de barras que consumió el backtest de esa permutación
`MaxNewHighDurationFrom`	inicio del drawdown más largo, en **epoch de milisegundos** (`pd.to_datetime(v, unit="ms")`)
`MaxNewHighDurationTo`	fin de ese mismo drawdown, mismo formato
`MaxNewHighDurationPct`	ese drawdown como porcentaje del periodo total

Y dos más llevan interrogación — `AnnualPctReturnDDRatio?` y `CalmarRatio?` — por el motivo del apartado siguiente.

---

8. Lo que no tiene nombre, y por qué

SQX no guarda los nombres: guarda un hash del nombre de la clase (en la tabla de métricas) o la posición en un array (en los estadísticos por permutación). El hash es `SQUtils.betterHashCode`, que vive en un jar embebido en el ejecutable y no está en disco, así que no se puede deshacer.

Los nombres de esta página se reconstruyeron **cruzando valores**: exportando las 135 columnas de un databank para 50 estrategias y viendo qué valor coincidía con cuál. Lo que no se pudo distinguir es lo que vale lo mismo en todas partes.

grupo	cuántos	qué son
verificados uno a uno	101 de 135 métricas	nombre seguro
identificados hasta la pareja	6 métricas	`Exposure?` / `ExposurePosition?`, `Outlier?` / `Outlier2?`, `CalmarRatio?` / `AnnualPctReturnDDRatio?`. Sabes de qué pareja es la columna, no cuál de las dos. Sus dos miembros valen lo mismo en los 225 perfiles de esta máquina
sin nombre, en `metrics.csv`	28 métricas	salen como `id:<número>`. **Valen 0 en todas las estrategias de esta instalación** — que es justo por lo que no hubo con qué distinguirlas, y también por lo que no aportan nada
sin nombre, en `permutations.csv`	34 estadísticos	salen como `stat:<tipo>:<índice>`, mismo motivo. `f` es un flotante, `i` un entero, `l` un long

Los identificadores concretos, por si algún día aparecen con valor distinto de cero y se pueden nombrar:

Métricas sin nombre (`metrics.csv`)

`id:-1032057831` `id:-1401870665` `id:-1558884181` `id:-1679613218`
`id:-1783525730` `id:-1792934761` `id:-1832811737` `id:-184077012`
`id:-1925735574` `id:-2028835157` `id:-2090795189` `id:-2100959438`
`id:-222360201` `id:-301431951` `id:-652428356` `id:-833100900`
`id:1041921791` `id:1775062197` `id:1876603141` `id:1945540697`
`id:195695425` `id:2059656148` `id:2133209636` `id:40355801`
`id:437767648` `id:500275187` `id:568822547` `id:982587326`

Estadísticos sin nombre (`permutations.csv`)

`stat:f:36` `stat:f:47` `stat:f:50` `stat:f:53` `stat:f:54` `stat:f:55`
`stat:f:58` `stat:f:6` `stat:f:62` `stat:f:63` `stat:f:64` `stat:f:65`
`stat:f:66` `stat:f:67` `stat:f:68` `stat:f:69` `stat:f:70` `stat:f:71`
`stat:f:72` `stat:f:73` `stat:f:74` `stat:f:75` `stat:f:76` `stat:f:77`
`stat:f:78` `stat:f:83` `stat:f:84` `stat:f:88` `stat:f:89` `stat:i:7`
`stat:l:0` `stat:l:1` `stat:l:2` `stat:l:3`

---

9. Lo que se extrae de la misma estrategia sin ser del SPP

`trades/<estrategia>.csv` — las operaciones del backtest principal

Son las que sí existen. 16 columnas, tal y como las escribe `orderstocsv`:

campo	qué es
`Ticket`	número de operación, empieza en 1
`Symbol`	símbolo. Si el fichero trae varios mercados, **esta columna es el separador**
`Type`	`Buy` o `Sell`
`Open time`	fecha y hora de entrada
`Open price`	precio de entrada
`Size`	tamaño en lotes. **Varía operación a operación** porque el sizing es por riesgo
`Close time`	fecha y hora de salida
`Close price`	precio de salida
`Profit/Loss`	resultado en divisa de la cuenta, ya con costes dentro
`Balance`	balance acumulado después de esta operación
`Sample type`	`IST` en muestra, `OOS1` fuera. **No siempre está bien**: depende de cómo se retesteó la estrategia por última vez
`Close type`	por qué se cerró: `Exit Signal`, `Exit After X Bars`, `End Of Friday (Time)`, `EndTest`...
`MAE ($)`	máxima excursión adversa, **en divisa de la cuenta, no en puntos**
`MFE ($)`	máxima excursión favorable, ídem
`Time in trade`	duración en texto (`2h 0m`)
`Comment`	comentario de la orden

La última fila puede ser una orden pendiente que nunca se ejecutó (`Close type=EndTest`, con precio de cierre vacío). Se tira.

`bars/bars_<TF>.csv` — las velas

`Date`, `Time`, `Open`, `High`, `Low`, `Close`, `Volume`. Es el fichero contra el que se reconcilian los precios de las operaciones.

Y del `.sqx`, sin exportar nada

qué	cómo
las métricas guardadas por muestra (IS / OOS / periodo completo)	`core.sqxstats.stats()`
la curva de equity diaria, un punto por día natural	`core.sqxstats.equity()`
el perfil SPP entero	`core.optprofile.read()`
símbolo, timeframe, hash de identidad, parámetros	`core.sqxfile`

---

10. Lo que no existe, por mucho que se busque

Los trades de las permutaciones. No es que falte el comando. Cada permutación se guarda como su cadena de parámetros más un bloque `SQStats`, y ese bloque solo admite seis tipos de registro — entero, long y flotante, cada uno con índice o con nombre — y nada más. La clase tiene un campo para objetos, pero el serializador no lo escribe nunca. No hay ninguna ruta por la que una lista de órdenes llegue al fichero.

La equity de cada permutación. Mismo motivo: `dailyEquity.bin` solo existe para el resultado principal.

Un percentil distinto de la mediana, si la casilla 3D estaba puesta. Con el detalle tirado solo quedan medianas e histogramas de 20 bins. Con el detalle guardado se puede pedir cualquier cosa.

15. SPP Ultra — ¿merece esta estrategia las 5.000 variantes?

Qué pregunta responde

Cuando pasas una estrategia por el cross-check **Sys. Param Permutation**, SQX prueba miles de combinaciones de sus parámetros y te enseña unos histogramas. Este comando lee esa tirada entera y contesta tres cosas que la pantalla de SQX no dice:

1. **Qué parámetros mueven de verdad el resultado** y cuáles están muertos.
2. **Dónde está la meseta** de cada parámetro — la zona ancha donde funciona, que no es lo mismo que el punto donde funciona mejor.
3. **Si toda la familia es ruido.** Si pruebas 3.000 combinaciones de algo que no sirve para nada, la mejor de las 3.000 va a parecer buena igualmente. Esto calcula cuánto de buena parecería por pura suerte, y compara.

Y deja preparado el **diseño de las 5.000 variantes** para la siguiente etapa.

Cuándo lo usas, y cuándo no

Úsalo con un databank en el que hayas corrido el cross-check SPP **con la opción *"Don't store data for 3D charts in Optimization profile"* desactivada**, y después de haber sacado los datos con `python3 -m sqx.export.export_spp`.

No sirve para comparar dentro y fuera de muestra. Y no es una limitación del programa: es que los datos no dan para eso. Dos tiradas SPP, una en IS y otra en OOS, **no se pueden emparejar**. Medido el 2026-09-19 sobre `Strategy 17.9.39`, con la misma estrategia y ajustes idénticos: de unas 11.600 combinaciones en cada tirada, **solo 6 coinciden**. El muestreador sortea, y sortea distinto cada vez. Ésa es exactamente la razón por la que existen las 5.000 variantes fabricadas a mano.

Antes de empezar

- **No toca SQX.** Lee archivos ya exportados, así que puedes lanzarlo con la GUI abierta y mientras haya un build corriendo.
- Tiene que existir el export: `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/spp/` con sus cinco CSV. Si no está, córrelo antes:

```
python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"
```

Cómo se ejecuta

```
python3 -m studies.breakage.spp.report --project XAUUSD --databank SPP_IS
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	no	carpeta del export, **con guiones bajos en vez de espacios**. Por defecto `SPP_IS`
`--day`	no	fecha del export a leer. Por defecto, el más reciente que tenga tabla SPP
`--strategy`	no	se puede repetir. Por defecto, todas las estrategias del export

Tarda unos segundos por estrategia.

Qué produce

Todo va a `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/spp/` (hasta el 25-09, suelto en la carpeta del día):

- **`estrategias/<nombre>.html`** / **`.json`** — el informe de cada estrategia (antes, una sección de `sppultra.md`), en el contrato de estudios que pinta la ventana.
- **`design_brief_<Estrategia>.json`** — uno por estrategia. Es el archivo que lee la etapa de fabricación de variantes. No lo edites a mano: se regenera cada vez que corres el comando.

Los informes **se acumulan** (una carpeta por fecha) y no se borra nada.

Cómo se lee el resultado

En pantalla te sale una línea por estrategia:

```
Strategy 1.19.29         proceed  n_eff= 4,283  max 17.01 vs nulo 10.04  vivos=10 congelados=0
Strategy 17.9.39         proceed  n_eff= 3,381  max 14.64 vs nulo  5.11  vivos=7  congelados=1
Strategy 23.16.37        proceed  n_eff= 4,300  max  8.44 vs nulo  5.35  vivos=10 congelados=1
Strategy 4.33.46         proceed  n_eff= 4,002  max 13.93 vs nulo  8.01  vivos=9  congelados=0
Strategy 41.5.25         proceed  n_eff= 4,320  max  7.58 vs nulo  4.85  vivos=12 congelados=0
```

- **`max`** **contra** **`nulo`** es el veredicto. `nulo` es lo que habría sacado la mejor de esas mismas combinaciones si ningún parámetro hiciera nada. `proceed` es que lo supera; `noise` es que no, y entonces **no hace falta seguir con esa estrategia**: te ahorras las 5.000 variantes.
- **`n_eff`** son las combinaciones con un resultado *distinto*. Siempre es menor que el número de filas, porque los parámetros muertos repiten el mismo backtest muchas veces. Es este número el que se usa para el umbral, no el de filas.
- **`congelados`** son los parámetros que el test de duplicados demostró que no hacen nada. Salen del diseño de variantes.

Dentro del informe, la tabla que más se mira:

parámetro	ReturnDDRatio	NetProfit	NumberOfTrades	grupos	idénticos	inerte
DICrossPeriod1	0.2929	0.4702	0.2698	224	0	
DICrossShift1	0.2359	0.0763	0.7847	189	0	
ExitAfterBars1	0.0264	0.0238	0.0691	184	0	
CBlock_SqzMmnInt21	0.0173	0.0193	0.0197	217	217	sí
IsBars1	0.0016	0.0007	0.0103	188	1	

Léela así, y en este orden:

- **`grupos`** **e** **`idénticos`** es la prueba. `grupos` son los pares de combinaciones que solo se diferencian en ese parámetro; `idénticos` son los que además dieron **exactamente el mismo resultado**. `CBlock_SqzMmnInt21`: 217 de 217. Ese parámetro no hace nada, está demostrado, y se congela.
- **Las columnas de números (η²)** son "qué porcentaje de la variación explica este parámetro". Y hay que leerlas con cuidado: fíjate en que `DICrossShift1` explica el **7,6 %** del beneficio neto pero el **78,5 %** del número de operaciones. **El mismo parámetro, dos respuestas muy distintas según qué métrica mires.** Por eso la tabla enseña varias y no una.

- **Por qué no se congela por η²**: mira `CBlock_SqzMmnInt21` otra vez. Está demostrado que no hace nada, y sin embargo su η² es 0,0173, más alto que el de `IsBars1` (0,0016), que sí hace cosas. Eso pasa porque el SPP sortea desequilibrado: a cada valor de un parámetro muerto le tocaron mezclas distintas de los demás, y esa diferencia se cuela en el η². **Fiarse del η² para congelar habría dejado dentro un parámetro muerto y fuera uno vivo.**

Y la tabla de mesetas:

```
| parámetro      | from | to   | width | center | argmax | original |
|----------------|------|------|-------|--------|--------|----------|
| DICrossPeriod1 | 40.0 | 46.0 | 3     | 43.0   | 43.0   | 67.0     |
| ExitAfterBars1 | 9.0  | 13.0 | 5     | 11.0   | 12.0   | 13.0     |
```

`center` es el **punto medio de la meseta**, no el mejor punto. Es lo que hace el `BestValue` del propio SQX. Cuando `width` es 1 no hay meseta: hay un pico, y el brief lo marca con `"spike": true`. Un parámetro que hay que acertar clavado no es un parámetro que puedas poner en real.

⚠ **Un pico puede ser falso.** Si la rejilla de pasos del SPP no pasa por el valor original de un parámetro (en las tres estrategias de USDJPY M30, `BBerDeviation1` va 1,89 · 2,01 · 2,13… y el original es 2,9), en ese nivel sólo está la propia estrategia original, y la tabla la da como argmax con `width` 1. Eso no es un pico: es la original sola. Está anotado en `OPEN.md` §79.

Las superficies por pareja (pestaña «Superficies por pareja»)

La tabla de mesetas mira los parámetros **de uno en uno**. Esta pestaña los mira **de dos en dos**: eliges en dos desplegables el parámetro del **eje X** y el del **eje Y**, entre todos los de la estrategia, y ves la rejilla de esa pareja. Están calculadas todas las parejas; si eliges el mismo parámetro en los dos ejes no hay rejilla.

Superficies por pareja

Elige un parámetro para cada eje. Meseta = el decil superior de las celdas de esa rejilla (10 %), la misma definición que la nube de parámetros. Una celda con pocas tuplas pesa lo mismo que una llena: una SPP muestrea desequilibrado. Si eliges el mismo parámetro en los dos ejes no hay rejilla.

Aquí se ve la combinación por defecto de Eje X (BBerPeriod1), Eje Y (ExitAfterBars1). Las demás están en el resultado JSON y en la ventana.

`x: BBerPeriod1 · y: ExitAfterBars1`

ExitAfterBars1 contra BBerPeriod1 — mediana de ReturnDDRatio

Cada celda es la mediana de ReturnDDRatio de todas las tuplas que usaron esa pareja de niveles, sean cuales sean los demás parámetros, θ_0 fuera. Cortes en los cuartiles y en el percentil 90: la banda de arriba es la meseta (≥ 3.82). Sin contar a θ_0, su celda (77 tuplas) vale 4.26, por encima del 94 % de las celdas, dentro de la meseta.

	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60	62	64	66	68
11	3.46	1.08	1.31	0.98	1.58	1.55	1.43	1.58	2.10	3.50	2.58	2.48	1.10	0.55	0.995	1.38	1.06	1.31	0.5
12	1.83	1.58	0.155	0.52	1.22	2.79	1.69	1.51	2.10	3.58	2.33	1.95	2.34	0.14	0.34	0.45	1.40	0.88	0.93
13	1.35	1.26	0.72	0.7	1.14	1.75	1.29	1.55	0.77	3.42	2.35	1.56	2.08	0.965	0.02	0.43	1.08	0.685	0.92
14	2.16	1.27	0.84	0.84	1.65	1.37	3.04	2.48	3.41	3.51	2.76	3.59	2.37	0.93	0.93	1.13	1.78	1.36	1.81
15	2.12	2.19	1.48	1.22	2.21	2.31	3.06	2.38	4.15	4.37	4.03	5.09	3.79	1.70	1.20	1.25	1.61	1.66	0.91
16	1.88	1.60	1.46	1.64	2.19	1.75	2.92	2.57	4.21	5.12	5.13	6.33	4.58	1.75	2.00	2.14	2.62	1.64	1.86
17	1.70	1.38	1.25	1.33	1.72	2.31	3.30	2.20	3.84	3.85	4.08	3.81	2.62	2.45	2.05	2.68	2.33	2.34	1.83
18	1.68	1.02	1.08	1.01	1.75	2.90	2.98	2.61	4.51	4.26	5.03	6.21	4.33	2.19	2.58	2.80	2.21	2.55	2.56
19	1.27	1.11	0.705	0.95	1.81	1.94	2.40	2.57	3.98	5.13	4.31	3.03	4.16	1.64	1.30	1.89	1.61	1.54	1.61
20	1.02	0.93	0.87	0.77	1.26	1.64	1.50	2.99	2.35	5.13	4.11	7.60	2.67	1.20	1.40	1.05	1.27	1.00	1.15
21	0.7	0.45	0.07	0.53	1.15	1.07	1.02	1.09	1.77	3.53	3.69	4.42	2.04	0.79	1.17	0.8	1.74	1.27	1.21
22	1.05	0.63	0.225	0.535	1.22	0.97	1.39	1.46	2.88	3.32	3.73	3.99	3.67	0.59	1.34	1.46	1.90	1.42	0.95
23	0.775	0.69	0.17	0.52	1.00	0.7	0.9	0.825	2.35	3.58	3.05	4.81	3.15	1.73	1.18	1.01	1.03	1.77	1.47
24	0.905	0.56	0.695	0.45	0.925	0.38	1.07	1.41	2.29	3.02	2.92	3.67	2.56	1.35	1.14	1.32	0.85	1.30	1.19

< 1.07 1.61 2.57 3.82

`θ₀: fila 18, columna 50`

Estrategia 9.27.83 de `Test_USDJPY_donchianUpperCrossUp_M30`, SPP de IS, eje X `BBerPeriod1`, eje Y `ExitAfterBars1`.

- **Cada celda** es la mediana de `ReturnDDRatio` de todas las combinaciones que usaron esa pareja de valores, sean cuales sean los demás parámetros. **La estrategia original no cuenta** en ninguna celda: si contara, en su propia celda se votaría a sí misma.
- **θ_0** es la estrategia original, marcada en su celda (debajo de la rejilla: «θ_0: fila 18, columna 50»).
- **Los colores** cortan en los cuartiles y en el percentil 90 de las celdas de esa rejilla. **La banda más oscura es la meseta: el 10 % de arriba de las celdas** (tu definición, 2026-09-27; la misma que la nube de parámetros).
- **La nota de cada rejilla** te dice dónde cae θ_0: «Sin contar a θ_0, su celda (77 tuplas) vale 4,26, por encima del 94 % de las celdas, dentro de la meseta». Si pone «θ_0 está solo en su celda», el SPP no probó ninguna otra combinación con esos dos valores y la rejilla no dice nada de su vecindad.

Debajo, la tabla **«θ_0 en cada pareja»** lo resume: una fila por pareja, cuántas combinaciones hay en la celda de θ_0, el suelo de la meseta, el rango de θ_0 entre las celdas y si está en la meseta (`sí`, `no` o `sin vecinos`). En las tres estrategias de USDJPY, **todas las parejas con `BBerDeviation1` salen «sin vecinos»**: el SPP no volvió a probar nunca 2,9.

Cómo decidir con ella: si θ_0 cae fuera de la meseta en las parejas de los parámetros vivos, la estrategia está en un pico de dos dimensiones aunque cada parámetro suelto parezca estable. Es una razón para mirar con lupa el diseño de variantes, no para descartarla aquí: el veredicto de esta etapa sigue siendo el del ruido.

La pestaña va en cada `estrategias/<nombre>.json` / `.html` que escribe `report.py`, entre las mesetas y el diseño. La captura es de `python3 -m studies.breakage.spp.report --project Test_USDJPY_donchianUpperCrossUp_M30 --databank SPP_IS`, corrido el 2026-09-27.

Métricas para criterios (pestaña «Métricas para criterios») — sin umbral todavía

Desde el 2026-10-04 el estudio calcula además las métricas que pediste como criterios. **Ninguna tiene umbral y ninguna cambia el veredicto**: se ven en esta pestaña, son columnas del SPP en la tabla de Databanks y el autopiloto las lee como `spp.verdict_csv.<columna>` de `verdict.csv`.

- **Distancia del original a la mediana** (el criterio heredado de SQX), por separado en IS y en OOS1 y para las seis métricas del panel 1. Se publica de dos maneras: en **% de la mediana** (`distancia_pct_…`, el «± X %» de SQX; falta cuando la mediana está a menos de 0,25 IQR de cero, porque dividir por ella explota) y en **rangos intercuartílicos** de la rejilla (`distancia_IQR_…`). La segunda existe porque «± X %» premia la poca dispersión: una rejilla estrecha pasa aunque el original sea su extremo. Al lado se ve entre qué valores cae el 90 % central de las propias permutaciones: un original fuera de ahí es un extremo de su vecindad.
- **Decaimiento de la mediana** (`decaimiento_…`): mediana(OOS1) / mediana(IS) en Sharpe, Sortino, Profit Factor y R/Edge ratio. Es un cociente de medianas — IS y OOS1 no comparten permutaciones, no hay ratio por variante —; 1 es «no decae». Solo se publica si la mediana IS supera su suelo (0,1 en Sharpe y Sortino, 1 en Profit Factor y R/Edge ratio). Sharpe y Sortino son los anualizados de SQX (√252), comparables entre una IS de 8 años y una OOS1 de 4.
- **A de Vargha–Delaney** (`A_…`): probabilidad de que una permutación OOS1 al azar supere a una IS al azar. 0,5 = iguales; por debajo, decae. Es un tamaño de efecto, no un p-valor.
- **Wasserstein / IQR** (`Wasserstein_…`): cuántos IQR de la IS hay que mover una distribución para convertirla en la otra. Bilateral: también sube si la OOS1 es mejor.

Junto a cada valor la tabla enseña lo que leería **una familia sin ventaja**: la misma rejilla OOS1, con su forma, centrada donde cae una estrategia de Sharpe 0 con el ruido de un solo camino de precios (± 1/√años: ± 0,5 de Sharpe con 4 años). **Con 4 años de OOS1 ese rango cubre los valores reales de las diez estrategias del Nikkei** (A de Sharpe real 0,25 a 0,76; sin ventaja 0,00 a 0,75), así que hoy estas métricas no distinguen una estrategia buena de una sin ventaja: se quedan como diagnóstico y **no se ofrecen como criterio** en la zona Criterios hasta medirlas sobre un SPP de estrategias sin ventaja o con una OOS más larga. No hay p-valores: las permutaciones comparten un camino de precios y su número no es un tamaño de muestra. El p-valor del KS no se usa.

Un ejemplo completo

```
$ python3 -m studies.breakage.spp.report --project XAUUSD --databank SPP_IS --day 2026-09-10
Strategy 1.19.29         proceed  n_eff= 4,283  max 17.01 vs nulo 10.04  vivos=10 congelados=0
Strategy 17.9.39         proceed  n_eff= 3,381  max 14.64 vs nulo 5.11  vivos=7 congelados=1
Strategy 23.16.37        proceed  n_eff= 4,300  max 8.44 vs nulo 5.35  vivos=10 congelados=1
Strategy 4.33.46         proceed  n_eff= 4,002  max 13.93 vs nulo 8.01  vivos=9 congelados=0
Strategy 41.5.25         proceed  n_eff= 4,320  max 7.58 vs nulo 4.85  vivos=12 congelados=0

-> /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21
```

Las cinco pasan el filtro de ruido. `Strategy 17.9.39` es la que lo pasa más holgada (14,64 contra 5,11, casi tres veces) y además es la única con un parámetro congelado además de `23.16.37`.

El brief que deja para esa estrategia dice, en resumen: siete parámetros vivos, uno congelado (`CBlock_SqzMmnInt21` fijo en 20), y los niveles a probar en cada eje — por ejemplo `DICrossPeriod1` en `[40, 43, 46, 49, 52, 58, 61, 64, 67]`, que es un abanico centrado en la meseta (43) pero **estirado hasta 67 para que quepa el valor original de la estrategia**. Eso es a propósito: si la meseta se ha movido, tiene que poder verse.

Qué NO te dice

- **No dice que la estrategia funcione.** Dice que su superficie de parámetros contiene más de lo que contendría buscando en ruido. Es un filtro para no gastar cómputo, no una aprobación.
- **No compara dentro con fuera de muestra**, por lo de las 6 combinaciones en común. Cualquier cosa que leas aquí es dentro de muestra.
- **El η² no es una medida limpia** en esta tirada, por el muestreo desequilibrado. Sirve para repartir niveles entre los parámetros vivos. No lo uses para decidir que algo no importa: para eso están las columnas `grupos` e `idénticos`.
- **La meseta es una estimación hecha con un solo tramo de historia.** Que un parámetro tenga una meseta ancha en 2008-2017 no significa que la tenga en 2018-2022. Precisamente eso es lo que va a medir la etapa siguiente.
- **`Ret/DD` crece con la longitud de la ventana.** Aquí da igual porque nunca se comparan dos ventanas, pero **no reutilices esta configuración para leer dos periodos distintos** sin cambiar la métrica.

Si algo falla

- **`IndexError` al buscar el export** — no hay ninguna carpeta con `spp/` dentro de `raw/<proyecto>/<databank>/`. Corre `sqx.export.export_spp` primero, o revisa que el databank lleve guiones bajos (`SPP_IS`, no `SPP IS`).
- **`KeyError` con el nombre de una estrategia** — el nombre tiene que ser exactamente el que usa SQX, espacios y puntos incluidos: `Strategy 17.9.39`.
- **Una estrategia con `congelados=0` y todos los `grupos` a 0** — el muestreador no llegó a generar pares que se diferencien en un solo parámetro. No es que no haya parámetros muertos: es que no hay pruebas. Hace falta más presupuesto de permutaciones.

18. Variantes — fabricar las 5.000 estrategias del diseño

Qué pregunta responde

SQX no tiene ningún comando que cambie los parámetros de una estrategia ya hecha. Ni uno. Así que si quieres saber **qué pasaría si este indicador usara 61 periodos en vez de 67**, la única forma es escribir un `.sqx` nuevo con ese 61 dentro y volver a testearlo.

Este comando es la fábrica. Le das el **diseño** que salió de SPP Ultra (etapa 15) y te escribe en disco las cinco mil estrategias que ese diseño pide, más una tabla que dice qué combinación lleva cada archivo.

No decide nada sobre trading. Decide **qué cinco mil combinaciones de entre cientos de miles** vale la pena probar, y luego las escribe.

Cuándo lo usas, y cuándo no

Úsalo cuando ya tienes el `design_brief_<Estrategia>.json` de SPP Ultra y el veredicto es `proceed`. Si el veredicto es `noise`, la familia entera no se distingue del azar y fabricar cinco mil variantes de ella es gastar medio giga y una tarde en medir ruido; el comando no te lo impide, pero te enseña el veredicto en la primera línea.

Fabricar el diseño entero es puro cálculo y escritura de archivos: no toca SQX. La única excepción, desde el 2026-09-29, es el **piloto** — un retest corto y automático de unas pocas docenas de segundos antes de fabricar las 5.000, que sí despierta el custodio (ver más abajo). `--no-pilot` o `--design-only` lo evitan si de verdad quieres el comando sin tocar nada.

No te da ningún resultado. Un archivo recién fabricado no se ha testeado: lleva dentro los números del padre hasta que SQX lo vuelva a correr. Si lees el `.sqx` fabricado esperando ver el resultado de la combinación nueva, vas a leer el del padre. Ver *Qué NO te dice*.

Antes de empezar

1. **Que exista el diseño.** Es el archivo que escribe la etapa 15:

```
python3 -m studies.breakage.spp.report --project XAUUSD --databank SPP_IS
```

Deja `~/Desktop/AlgoData/reports/XAUUSD/SPP_IS/<fecha>/design_brief_Strategy_17-9-39.json`.

2. **Que exista el export SPP del que salió ese diseño**, con sus CSV. La fábrica lo vuelve a leer, pero sólo cuatro columnas: de ahí saca los **centinelas**, que son combinaciones cuyo resultado SQX ya calculó.

3. **Sitio en disco.** Con el ajuste por defecto, 5.000 variantes ocupan **unos 505 MB**. Con `shape: minimal` bajan a 70 MB y con `full` suben a 631 MB.

4. No hace falta cerrar SQX ni parar nada a mano: el piloto arranca y para el custodio solo, igual que la etapa siguiente. Con `--no-pilot` o `--design-only` el comando no habla con SQX en absoluto.

Cómo se ejecuta

```
python3 -m sqx.variants.make \
  --brief ~/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21/design_brief_Strategy_17-9-39.json \
  --project XAUUSD
```

flag	obligatorio	qué hace
`--brief`	sí	ruta del `design_brief_<Estrategia>.json`. De él salen los parámetros, sus niveles, los congelados y el objetivo de 5.000
`--min-variants`	no	el mínimo de variantes distintas por madre; **1.000** si no se pone (`minimum.variants` en `sqx/variants/config.yaml`). Si el ±30 % no da tantas, los parámetros enteros se ensanchan en pasos del 5 % hasta llegar, como mucho a ±60 %, y si aun así no llega lo avisa con ⚠
`--project`	sí	nombre del proyecto, sólo para decidir la carpeta de salida
`--limit N`	no	fabrica sólo las N primeras filas. **El diseño no cambia**: se calculan las 5.000 y se escriben N. Es lo que se usa para probar la cadena con tres archivos antes de comprometerse a un lote entero
`--design-only`	no	calcula el diseño, escribe `plan.csv` y para. No escribe ningún `.sqx`
`--no-pilot`	no	salta el piloto (ver abajo) aunque `config.yaml` lo tenga encendido

Tarda **unos 50 segundos** en escribir las 5.000 con el ajuste por defecto (6 segundos con `minimal`). El diseño en sí es instantáneo.

Los mandos que no son flags están en `sqx/variants/config.yaml`: la semilla, el radio de la vecindad, cómo se reconstruye el rango de un parámetro congelado, cuántos centinelas, y **la forma del archivo** (`minimal` / `no_profile` / `full`).

El piloto: unas pocas docenas de segundos antes de fabricar el lote entero

Medido el 2026-09-24 en `Strategy 17.9.39`: de 2.000 variantes fabricadas, **1.002 operaron menos de 30 veces dentro de muestra**. No es una cola, es la mitad del diseño — presupuesto gastado en combinaciones que ni siquiera son un punto de una superficie de rendimiento, y que además hay que descartar después, dejando parámetros con un solo valor útil.

Por eso, **desde el 2026-09-29 y por defecto**, antes de fabricar las 5.000 el comando fabrica y retestea unos cientos de tuplas de muestra (las que se mueven, nunca las congeladas) en el mismo proyecto — el mismo retest corto que usa la etapa siguiente, no uno nuevo — y cuenta cuántas veces operó cada una. Si un valor de un parámetro deja a sus tuplas por debajo de 30 operaciones de mediana (con suficientes puntos para fiarse), ese valor se descarta **antes** de fabricar el lote grande — nunca el parámetro entero, y nunca el valor con el que se construyó la madre.

`--no-pilot` lo salta (por ejemplo para una prueba de humo con `--limit`); `pilot.enabled: false` en `sqx/variants/config.yaml` lo apaga para siempre. **Nunca es silencioso**: `pilot.json` queda junto al lote con la mediana de operaciones y el soporte de cada nivel, y `design.json` lleva el mismo recorte bajo `pilot_dropped` — dos lotes de la misma madre, uno con el piloto y otro sin él, no son el mismo diseño y `pilot_dropped` es lo primero que hay que mirar para saber en qué se diferencian.

```
piloto: 287 reteseadas de 300, 2 niveles descartados en 1 parametros (< 30 operaciones en build)
```

El piloto también se puede correr suelto, para mirarlo sin fabricar nada más (`python3 -m sqx.variants.pilot --brief <design_brief.json> --project <PROYECTO> --out <carpeta>`) — es lo que `make` llama por dentro.

Qué produce

Todo en `~/Desktop/AlgoData/strategyPermutations/<proyecto>/<Estrategia>/`:

archivo	qué es
`pilot.json`	el piloto: cuántas tuplas de muestra, cuántas volvieron, qué niveles se descartaron y por qué. Ausente si se saltó
`plan.csv`	el diseño completo: una fila por combinación, con su estrato y su identificador. Siempre las 5.000, aunque fabriques 3
`sqx/P00000.sqx` …	las estrategias. Una por fila fabricada
`manifest.parquet`	**la tabla que une todo el estudio.** Una fila por archivo que hay de verdad en `sqx/`, leída del archivo, no del plan

Se sobrescribe. Hay un diseño vigente por estrategia y volver a correr el comando lo reemplaza, para que no pueda existir la pregunta "¿cuál de estas cinco mil carpetas es la buena?". Si borras `sqx/` a mano y vuelves a lanzar, el manifiesto se reconstruye con lo que haya.

Cómo se lee el resultado

Lo primero que sale son los costes vigentes del símbolo — es obligatorio y sale siempre:

```
# XAUUSD — overrides to apply

SQX symbol: XAUUSD_DukasM1_Infinox   verified: 2026-09-03

- **spread**: use `10.0` points (SQX default `10.0`) — PROVISIONAL 2026-09-21 — SQX default, not an
agreed Infinox figure. Owner will replace it.
- **commission**: use `8.0` $ per lot per side (SQX default `SizeBased 8`) — PROVISIONAL 2026-09-21 —
...
```

Luego el diseño:

```
Strategy 17.9.39  verdict=proceed  live space 3,888 tuples, with the frozen 19,440
  neighbourhood  budget   999  kept    57
  factorial      budget 3,939  kept 3,830
  coverage       budget 1,108  kept 1,108
  controls                     kept     5
  planned 5,000 of a target 5,000 (shortfall 0)
```

Se lee así:

- **`live space 3,888`** — con los niveles que eligió el diseño, **todas** las combinaciones posibles de los parámetros vivos son 3.888. Son menos que las 5.000 que se pedían: en esta estrategia el diseño no es una muestra, es la rejilla entera. `with the frozen 19,440` es lo mismo contando también los parámetros congelados, que sólo mueve el estrato de cobertura.
- **`budget`** / **`kept`** — lo que se le pidió a cada estrato y lo que aportó de nuevo. Aquí la vecindad sólo pudo dar 57 (varios parámetros tienen dos o tres niveles y la bola se acaba), y los 942 que le sobraron pasaron al siguiente. El factorial pidió 3.939 y dio 3.830 porque 109 de sus combinaciones ya las había puesto otro estrato: **nunca se repite una combinación**.
- **`shortfall 0`** — se llegó a las 5.000. Si saliera un número positivo querría decir que el espacio de diseño se agotó antes; no es un error, es información.

Y por último la fabricación y la verificación:

```
wrote 3 .sqx as 'no_profile', 0.3 MB
{
  "planned": 3,
  "on_disk": 3,
  "missing": [],
  "unexpected": [],
  "tuple_mismatch": [],
  "duplicate_tuples": 0
}
```

Esos cinco números son lo importante de todo el comando. No los calcula mirando lo que quería escribir: abre los archivos de vuelta, saca de dentro los parámetros que llevan y los compara.

campo	qué significa si no es cero / vacío
`missing`	el plan tenía un identificador que no llegó a ningún archivo
`unexpected`	hay un archivo en la carpeta que el plan no pidió — típicamente restos de una tirada anterior
`tuple_mismatch`	**el peor de todos.** El archivo `P01234` no lleva dentro la combinación que el plan dice que lleva. El estudio entero se une por ese identificador
`duplicate_tuples`	dos archivos con la misma combinación. SQX los puede fundir en uno y quedarte con filas del manifiesto sin archivo detrás

Si alguno falla, el comando **sale con error** y te dice que no cargues el lote. Todo vacío y a cero es lo que tiene que salir siempre.

Un ejemplo completo

Probar la cadena con tres archivos antes de comprometerse a cinco mil:

```
$ python3 -m sqx.variants.make \
    --brief ~/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21/design_brief_Strategy_17-9-39.json \
    --project XAUUSD --limit 3

[... los costes del símbolo ...]

Strategy 17.9.39  verdict=proceed  live space 3,888 tuples, with the frozen 19,440
  neighbourhood  budget   999  kept    57
  factorial      budget 3,939  kept 3,830
  coverage       budget 1,108  kept 1,108
  controls                     kept     5
  planned 5,000 of a target 5,000 (shortfall 0)

wrote 3 .sqx as 'no_profile', 0.3 MB
{
  "planned": 3,
  "on_disk": 3,
  "missing": [],
  "unexpected": [],
  "tuple_mismatch": [],
  "duplicate_tuples": 0
}

-> /home/sergioguslw/Desktop/AlgoData/strategyPermutations/XAUUSD/Strategy_17.9.39
```

Y mirando el manifiesto, que es lo que leerán todos los análisis de después:

```
variant_id  sqx_name                 stratum  origin  param_DICrossPeriod1  ...
canary_expect_netprofit  canary_expect_trades
P00000      Strategy 17.9.39 P00000   origin   True                    67.0
NaN                  <NA>
P00001      Strategy 17.9.39 P00001   canary   False                   43.0
38896.359375                 1077
P00002      Strategy 17.9.39 P00002   canary   False                   94.0
-37128.390625                 843
```

Las tres primeras filas de cualquier lote son **controles**, no material de análisis:

- **`P00000` es la madre**, la estrategia original reescrita por la fábrica sin cambiarle nada. Si la reescritura rompiese algo, es la fila que deja de parecerse a la estrategia de la que salió. Y es la única que la etapa de borrado **se niega a tocar**.
- **`P00001` y `P00002` son centinelas**: combinaciones que SQX ya probó en el SPP, con el resultado que dio guardado al lado. Están elegidas en los extremos de la rejilla — la de más beneficio y la de más pérdida — a propósito: una cadena mal montada devuelve un número intermedio creíble, no devuelve el máximo de una rejilla de cuatro mil.
- Y después hay **una pareja por cada parámetro congelado**: la madre con ese parámetro movido a tope. Como el diseño lo congeló por haber demostrado que no mueve nada, esas filas tienen que volver **exactamente iguales a `P00000`**. Es la única comprobación de que congelarlo estaba bien.

Cuando la etapa siguiente recoja los resultados, si los centinelas no vuelven con lo esperado, la cadena está rota en algún punto — y te enteras antes de mirar cinco mil resultados.

Qué NO te dice

- **No dice si una variante es buena.** No hay ni un número de rendimiento aquí. Esto escribe archivos; medirlos es la etapa siguiente.
- **Un `.sqx` fabricado lleva dentro los resultados del padre** hasta que SQX lo vuelva a testear: las operaciones, la curva de equity y las estadísticas guardadas son las de la madre. Con la forma `no_profile` (la de por defecto) y con `full` eso es así. Consecuencia incómoda: **una variante que por lo que sea no llegue a correr no se ve vacía, se ve como la madre.** El manifiesto no puede detectarlo; los centinelas y el conteo del databank sí.
- **Las expectativas de los centinelas son sobre la muestra IS**, y sólo valen mientras la ventana dentro de muestra del retest sea la misma sobre la que corrió el SPP. Un retest configurado con otra historia falla todos los centinelas por un motivo que no tiene nada que ver con la fábrica.
- **Los niveles de un parámetro congelado son una reconstrucción, no un dato.** El diseño congela un parámetro y da su valor, no un rango; el estrato de cobertura necesita variarlo y el rango se rehace igual que lo hace SQX en sus propias permutaciones (±30 %; un *shift* no se mueve nunca). Es la suposición más discutible del módulo y está anotada como tal.
- **La rejilla cubre SIEMPRE al menos ±30 % del valor original** (dueño, 2026-09-26). Un parámetro entero toma **todos** los valores enteros de su rango (un período 14–26 son 13 valores); uno decimal, hasta 9 niveles que el SPP exploró. Y cada madre tiene **al menos 1.000 variantes distintas**: si el ±30 % no las contiene, los enteros se ensanchan solos (USDJPY `Strategy 15.12.75`: de 216 a 1.105, EMA 12–28, salidas 7–19, fractal 3–7). Sobol reparte el estrato de cobertura por todo ese espacio. Los *shift* se quedan en su valor. Antes se quedaba en la meseta: EMA 20–27 y salidas 12–17, una franja.
- **Que el diseño contenga la combinación buena no está garantizado.** Una meseta que se haya movido más allá del ±30 % fuera de muestra se queda fuera.

Si algo falla

`not variables of this strategy: ['X']` — el diseño nombra un parámetro que la estrategia madre no tiene. Casi siempre es un `--brief` de una estrategia y un `.sqx` de otra: el `.sqx` se coge del campo `source` del propio diseño, así que si has movido o editado carpetas del export a mano, se descolocan. Vuelve a correr la etapa 15.

`settings.xml does not carry both name fields` — el `.sqx` madre no tiene los dos campos de nombre donde se esperan. Es señal de que la versión de SQX que lo escribió no es la de esta caja; no fuerces nada, dilo.

`the batch on disk is not the batch that was planned` — la verificación encontró algo. No cargues ese lote en SQX. Mira cuál de los cuatro campos salió con contenido; lo normal es `unexpected`, que son restos de una tirada anterior en `sqx/`.

Un `FileNotFoundError` sobre `spp.parquet` — falta el export SPP del que salió el diseño. Los centinelas salen de ahí. Reexpórtalo con `python3 -m sqx.export.export_spp`.

37. El retest de las variantes para el WFC y el CSCV

Escribe en un proyecto custom de SQX **las tres tareas** que producen el material de los pasos 17 (Walk Forward Correlation) y 18 (CSCV): una por tramo — `build`, `oos1`, `oos2` —, cada una con el spread y el slippage que le tocan, y cada una corriendo además los mercados adicionales.

Qué pregunta responde

Ninguna, todavía. Esto no analiza: **prepara los backtests**. Lo que produce son tres databanks con las mismas variantes corridas sobre tres tramos distintos, que es el material que luego leen `studies/optimisation/wfc/` (¿el in-sample predice el out-of-sample?) y su CSCV (¿mi forma de elegir parámetros sobreajusta?).

Los dos estudios comparten configuración a propósito: leen el mismo panel y la misma equity diaria del mismo lote. Si cada uno tuviera su bloque en `assets/_build.yaml` serían dos lotes distintos y sus respuestas ya no hablarían del mismo objeto. Por eso hay **un solo bloque** `wfc:`.

Cuándo lo usas, y cuándo no

Después de fabricar el lote de variantes (`/variants`, paso 16.5) y antes de correrlo. Nunca antes: las variantes son los ficheros que se van a retestear.

⚠ **Gasta `oos2`.** `assets/_policy.yaml` reserva ese tramo al WFC y a la WFM, y cada mirada lo gasta. Esto es el WFC, así que es uno de sus dos consumidores legítimos — pero no se corre "para ver qué sale", y el paso 19 sigue sin poder leerse hasta que 17, 18 y 19 estén los tres hechos.

Antes de empezar

1. El proyecto tiene que ser **custom**, clonado con `sqx/projects/builder.py`, nunca el `Retester` de serie (regla dura 10).
2. El proyecto tiene que estar **cerrado**: SQX reescribe el `.cfx` al salir y se pierde el cambio en silencio (regla dura 4). El comando lo comprueba y se niega.
3. `python3 -m core.assets XAUUSD` en verde (regla dura 5).

Cómo se ejecuta

```
python3 -m sqx.projects.wfc XAUUSD \
    --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_WFC/project.cfx \
    --timeframe M30 \
    --tasks Retest-Task1.xml,Retest-Task2.xml,Retest-Task5.xml
```

flag	obligatorio	qué hace
`--cfx`	sí	el proyecto custom donde se escriben las tres tareas
`--timeframe`	sí	el timeframe del proyecto; es el que llevan los mercados adicionales
`--tasks`	sí	tres ficheros de tarea **en el orden de** `wfc.tasks`: build, oos1, oos2. Se les cambia el título al de la doctrina y se activan
`--no-markets`	no	deja fuera los mercados adicionales en las tres patas de **este** proyecto, diga lo que diga `wfc.markets`. Para un estudio que sólo lee el mercado principal (el paso 24, `10-cierre.pdf` › `54-atr-calculator`)
`--json`	no	la misma salida en JSON

Salida real sobre una copia del donante congelado:

```
XAUUSD  XAUUSD  <- WFC Variants (las tres leen el MISMO lote)
  ✓ WFC 1 IS     build 2008.01.01→2017.12.31  -> WFC Build  1 condicion(es) apagada(s)
      mercados: XAGUSD_DukasM1_Infinox, BRENTCMDUSD_ftmo
  ✓ WFC 2 OOS1   oos1  2018.01.01→2022.12.31  -> WFC OOS1   1 condicion(es) apagada(s)
      mercados: XAGUSD_DukasM1_Infinox, BRENTCMDUSD_ftmo
  ✓ WFC 3 OOS2   oos2  2023.01.01→2026.08.30  -> WFC OOS2   1 condicion(es) apagada(s)
      mercados: XAGUSD_DukasM1_Infinox, BRENTCMDUSD_ftmo
```

Por qué tres tareas y no una

Una sola tarea de 2008 a 2026 llevaría **un** spread y **un** slippage para los dieciocho años, y habría que elegir cuál miente menos. Con tres, cada tramo cobra lo suyo: comprobado en el `.cfx` escrito, la tarea de `build` sale con spread 11.0 en la plata y slippage 2.5, y la de `oos2` con 22.0 y 5.

Y además sale gratis en información. Las tres leen el **mismo** databank de entrada (`WFC Variants`) y cada una escribe el suyo (`WFC Build`, `WFC OOS1`, `WFC OOS2`), así que el corte in-sample / out-of-sample deja de ser una fecha dentro de una curva y pasa a ser **de qué tarea vino** cada curva. El `<OutOfSample>` de cada tarea se vacía justamente por eso: una tarea que corrió un tramo entero *es* una muestra.

Los mercados adicionales van dentro

`markets: true` en la doctrina. Cada una de las tres tareas lleva también el crosscheck `RetestOnAdditionalMarkets` con los mercados de `assets/_markets.yaml`, cada uno con **sus** costes y recortado a su `data_from` si empieza más tarde que el tramo — en la tarea de `build`, el Brent corre 2013→2017 y la plata 2008→2017.

Son los **crossmarkets**, no los crossTF: un timeframe reescalado es la misma serie de precios remuestreada, así que no añade grados de libertad a un CSCV.

⚠ **El coste se multiplica por (1 + nº de mercados).** Con XAUUSD y sus dos mercados family, las tres tareas son **nueve** backtests por variante. Mira el tamaño del lote antes de arrancar.

Aquí tampoco se filtra nada

`wfc.conditions: []`. El comando apaga todas las condiciones de aceptación del crosscheck, fuerza `DeleteFailedStrategies` a false y pone `retestSelected="false"`, y dice cuántas apagó. Una variante descartada y una variante que perdió dinero son la misma cosa para el que cuenta, y entonces el PBO no significa nada.

Qué hay que hacer a mano todavía

Los tres databanks tienen que **existir** en el proyecto (`WFC Variants`, `WFC Build`, `WFC OOS1`, `WFC OOS2`): SQX empareja por el nombre exacto y **ignora en silencio** un databank que ningún `<Databank>` del `config.xml` declara. Créalos en la GUI del proyecto custom antes de correr nada, o clona un donante que ya los lleve.

Qué NO hace

No carga las variantes, no arranca el custodio y no cosecha nada. Eso es `/variants` y `sqx/variants/execute.py`. Esto sólo deja el proyecto escrito.

19. Walk forward correlation — ¿sirve de algo optimizar los parámetros?

Qué pregunta responde

Si eliges los parámetros de una estrategia por lo bien que van dentro de muestra, ¿siguen yendo bien fuera?

Esa es toda la pregunta, y es la que decide si la optimización es trabajo o es autoengaño. El estudio coge una estrategia madre, fabrica muchas versiones de ella con parámetros distintos, las retestea todas con la misma partición IS/OOS, y dibuja un punto por combinación: lo que ganó dentro contra lo que ganó fuera.

- Si la nube sube en diagonal, el ranking dentro de muestra **predice** el de fuera. Optimizar compra algo.
- Si la nube llena los cuatro cuadrantes, no predice nada. Lo que ganas optimizando dentro te lo quitan fuera, y el "mejor" juego de parámetros del backtest es ruido con buena pinta.

Cuándo lo usas, y cuándo no

Lo usas cuando ya tienes una estrategia que te interesa y quieres saber si sus parámetros significan algo, antes de gastar días en afinarlos. También cuando quieras comparar dos estrategias por su *robustez* en vez de por su beneficio.

No lo usas para elegir los parámetros. Este estudio no te dice cuáles poner: te dice si merece la pena elegirlos. Si sale que no, la respuesta es cambiar de estrategia o de enfoque, no buscar mejor dentro del mismo grid.

Tampoco lo usas con cuatro puntos. Con una docena de combinaciones el margen de error de una correlación es enorme, y el estudio te lo dirá a la cara (`indeciso`) en vez de fingir una conclusión. `indeciso` significa **fabrica más puntos**, no "baja el umbral".

Antes de empezar

Tres cosas, y las tres son bloqueantes:

1. **Los costes del símbolo tienen que estar acordados.**

`bash python3 -m core.assets XAUUSD`

Si sale con código distinto de cero, párate. Hoy XAUUSD sale en 0 pero con los tres valores marcados **PROVISIONAL**: son los defaults de SQX, no cifras pactadas con Infinox. Todo número con coste que produzcas arrastra esa salvedad, y el `state.json` del pipeline la marca con `costs_provisional: true`.

2. **Tiene que existir el export SPP de la estrategia**, en `AlgoData/raw/<proyecto>/<databank>/<fecha>/`. Es lo que lee `studies/breakage/spp` para decidir qué parámetros mueven el resultado y con qué niveles.

3. **El custodio tiene que poder arrancar.** Es el install `~/Desktop/SQX_w2`, puerto 5070, y es quien ejecuta los retests. El maestro no se toca en ningún momento.

Cómo se ejecuta

La forma corta, y la que vas a usar. Una línea hace las siete etapas:

```
python3 -m pipeline.run --project XAUUSD --databank SPP_IS --strategy "Strategy 17.9.39"
```

flag	obligatorio	qué hace
`--project`	sí	proyecto tal y como aparece en SQX
`--databank`	no	carpeta del export, con guiones bajos. Por defecto `SPP_IS`
`--strategy`	no	repetible. Por defecto, **todas** las del export
`--day`	no	fecha del export. Por defecto la más reciente

⚠ **Desde el 2026-09-27 la etapa `wfc` del pipeline se para** con `the following arguments are required: --family` : el estudio apunta cada lectura en el Ledger y necesita la familia de plantillas, que el pipeline todavía no le pasa. Hasta que se decida de dónde sale, córrelo suelto con las etapas por separado de abajo.

Cuántas combinaciones se fabrican lo decide `sample` en `pipeline/config.yaml`. El valor por defecto es **2.000**, y no es un número redondo elegido a ojo: es donde la respuesta deja de ser "no se puede saber". Medido sobre esta misma estrategia el mismo día:

puntos pedidos	utilizables	rho	intervalo 95 %	veredicto
11	9	0,18	[−0,55, 0,75]	`indeciso`
150	74	0,23	[−0,00, 0,43]	`indeciso`
600	297	0,22	[0,10, 0,32]	`indeciso`
2.000	**1.001**	**0,19**	**[0,13, 0,25]**	**`no_fiable`**

Fíjate en que **rho apenas se mueve** (0,18 → 0,19): lo que cambia es la anchura del intervalo. Con once puntos el número ya era el correcto y no servía para nada. Con `sample: 0` fabrica el diseño entero del brief. Están **repartidas** por todo el diseño, no cogidas por arriba — eso importa, y el apartado *Qué NO te dice* explica por qué.

La corrida entera de 2.000, arrancando con el custodio apagado: **2 min 16 s y 28 MB**.

Las etapas por separado, si quieres repetir solo una:

```
python3 -m sqx.variants.make    --brief <design_brief.json> --project XAUUSD --sample 11
python3 -m sqx.variants.execute --work <dir>     # carga en el custodio, retestea, exporta
python3 -m sqx.variants.equity  --work <dir>     # las curvas diarias de los 3 tramos, unidas
python3 -m sqx.variants.collect --work <dir>     # une el panel al manifiesto → metrics.parquet (lee
equity.parquet)
python3 -m sqx.variants.mothertrades --work <dir>   # las operaciones de la madre → trades.parquet
(las lee el paso 20)
python3 -m studies.optimisation.wfc.report --work <dir> --family <familia> [--inside build,oos1 --
outside oos2 | --split oos2_only]
```

Las operaciones de la madre. `collect` guarda en `<dir>/origin/` el fichero ya retesteado de la madre en cada tramo, y `mothertrades` las exporta con SQX (un solo arranque del conductor, unos segundos; admite varios `--work` de una vez, y el piloto automático lo llama una vez al final del paso 16.5 con todas las madres). Deja `trades.parquet` en el lote: sólo la madre, no las variantes (exportar un lote entero son ~90 minutos). El paso 20 reconstruye con ellas su curva de cierre a cierre en oos2. Hay que llamarlo **antes** de `execute --clear` sólo en el sentido de que `collect` ya haya corrido: `mothertrades` lee `origin/`, no los databanks.

El suelo de operaciones: 50, y es duro

Ningún backtest con menos de 50 operaciones en todo el periodo (`build` + `oos1` + `oos2`) entra en ninguna estadística. Decisión del dueño, 2026-09-24. Está en `wfc.min_trades_total` de `assets/_build.yaml` y lo aplica `collect.py`: lo que no llega **no se escribe** en `metrics.parquet`, así que ningún estudio de aguas abajo puede volver a meterlo por descuido.

Se cuenta por **variante × mercado**, porque un backtest es una estrategia corriendo en un mercado. Una variante que opera 400 veces en oro y 9 en plata tiene un resultado y un no-resultado:

```
P00003  Main      300 operaciones   usable=True
P00003  XAGUSD      9 operaciones   usable=False
```

No desaparece sin decirlo. `collect.py` avisa:

```
⚠  1 variantes y 12 celdas (variante x mercado) operan menos de 50 veces en
    build+oos1+oos2 y quedan FUERA de toda estadistica. Siguen en segments.parquet
    con usable=False, por si hace falta mirarlas.
```

y el recuento queda en `collected.json` (`thin`, `thin_cells`, `floor`) y en la nota de `estudios/wfc.html`.

⚠ **Hay un segundo suelo, y es otra cosa.** `min_trades: 30` en el `config.yaml` del WFC mira cada **lado** de la partición por separado: una variante puede sumar 200 operaciones y llevar sólo 4 en el tramo que hace de fuera de muestra. Actúa sobre lo que ya pasó el duro, y el informe dice cuántos puntos cayó cada uno:

```
PROGRESS 100 rho 0.19 — no_fiable (1001 puntos; 37 fuera por pocas operaciones en total,
                                   962 por pocas en un lado)
```

Con la composición `build+oos1` contra `oos2` (el atajo `oos2_only`) ese segundo suelo aprieta más, porque el lado fuera de muestra son 3,7 años en vez de 10. Si se come el lote, se baja **ése**, no el duro.

Los tres tramos, y las dos maneras de leerlos

El lote se retestea **una vez** en tres tareas de SQX — `build`, `oos1` y `oos2`, cada una a sus propios costes: `09-optimizacion.pdf › 37-wfc-retest` — y de ahí salen tres paneles. La cosecha los une:

- **suma lo que es sumable** (neto, número de operaciones, beneficio y pérdida brutos), así que el profit factor de una unión es `Σ bruto ganado / Σ bruto perdido` y no el promedio de tres profit factors, que pesaría un tramo de cinco años igual que uno de diez;
- **recalcula lo que no lo es** (drawdown y Sharpe) sobre la curva diaria ya unida, porque una unión puede cruzar un valle más profundo que cualquiera de sus partes.

Y con eso, el mismo lote responde a **varias preguntas distintas**, según qué tramos pongas dentro y cuáles fuera. A eso se le llama la **composición**, y la regla es tuya (2026-09-27):

- **`build` va siempre dentro de muestra.**
- Dentro es `build` o `build+oos1`.
- Fuera es **lo que queda**, y `oos1` puede quedarse fuera de las dos (apagado).

Salen cuatro composiciones, y ninguna más:

composición	dentro	fuera	cómo se pide	qué pregunta
`build__oos1`	`build`	`oos1`	`--inside build --outside oos1`	¿lo construido predice el tramo siguiente? **No toca `oos2`**
`build__oos2`	`build`	`oos2`	`--inside build --outside oos2`	la misma, saltándose `oos1`
`build__oos1+oos2`	`build`	`oos1` + `oos2`	`--split oos1_oos2` o `--inside build --outside oos1,oos2`	la normal: ¿predice todo lo que vino después?
`build+oos1__oos2`	`build` + `oos1`	`oos2`	`--split oos2_only` o `--inside build,oos1 --outside oos2`	**la estricta**: dando por vistos los dos primeros tramos, ¿sigue apareciendo en el que no ha mirado nadie?

Cualquier otra cosa (`--inside oos1` , un tramo a los dos lados, `--outside` vacío) se rechaza con `composición no admitida` . Sin `--inside` ni `--split` , manda `split_mode` de `engines/variants/config.yaml` , hoy `oos2_only` . **Cambiar de composición no re-corre ningún backtest**: las columnas de todas ya están en `metrics.parquet` .

Cada composición que lees queda apuntada en el Ledger, un renglón por tramo leído (paso 17), en el estudio `<símbolo>_<timeframe>_<familia>` . El símbolo y el timeframe los saca del propio lote; la familia se la das tú con `--family` (obligatoria: el lote no la sabe). **Antes de abrir nada** pregunta a la puerta (`ledger.gate.allow`) por cada tramo: si una composición toca `oos2` y la política no se lo permite al WFC, se para ahí y no lee ni una fila. Hoy el WFC sí está en la reserva de `oos2` (`assets/_policy.yaml`).

Probado el 2026-09-27 sobre el lote `Strategy_9.27.83` de `Test_USDJPY_donchianUpperCrossUp_M30` :

```
$ python3 -m studies.optimisation.wfc.report --work <lote> --family
Test_USDJPY_donchianUpperCrossUp_M30 --inside build --outside oos1
PROGRESS 20 USDJPY M30: IS=build, OOS=oos1, permitidos por el ledger
PROGRESS 55 150 puntos utiles de 150
PROGRESS 100 rho 0.43 — indeciso (150 puntos; 0 fuera por pocas operaciones en total, 0 por pocas en
un lado); 2 filas en el ledger del paso 17

$ python3 -m studies.optimisation.wfc.report --work <lote> --family
Test_USDJPY_donchianUpperCrossUp_M30 --inside build,oos1 --outside oos2
PROGRESS 20 USDJPY M30: IS=build+oos1, OOS=oos2, permitidos por el ledger
PROGRESS 55 150 puntos utiles de 150
PROGRESS 100 rho -0.25 — no_fiable (150 puntos; 0 fuera por pocas operaciones en total, 0 por pocas en
un lado); 3 filas en el ledger del paso 17
```

El mismo lote, dos respuestas: con `build` contra `oos1` el orden aguanta a medias (`indeciso`); con `build+oos1` contra `oos2` se invierte (`no_fiable`). Por eso la composición es parte del resultado y se apunta.

El PBO del CSCV no cambia con la composición: parte el historial en 16 bloques y lee sus 12.870 particiones (C(16,8)). Solo cambian sus cuatro números cronológicos. El WFC, en cambio, depende entero de la composición.

El reconocimiento SPP, dentro del proceso

El SPP es la etapa que dice **qué parámetros mueven el resultado y con qué recorrido**. Sin él no hay rejilla. Corre en el custodio, sobre una sola madre:

```
python3 -m sqx.variants.spp --work <dir> --mother "<ruta al .sqx>" --kind spp_is \
  --chart "XAUUSD_DukasM1_Infinox M30 5" --project <PROYECTO>
```

`--project` es **el proyecto de workflow de la propia madre** (regla dura 10, dueño 2026-09-29) — el mismo que `sqx.projects.builder --workflow` creó, nunca uno aparte hecho sólo para el reconocimiento. Se niega si le das `Builder` o `Retester`.

`--kind` es `spp_is` (2008–2017) o `spp_oos` (2018–2022); las ventanas salen de la tarea del donante. Los mandos están en `sqx/variants/config.yaml`, bloque `spp`: `steps`, `spread_pct` y `keep_pct`.

⚠ **`keep_pct` (el `PctToPass` de SQX) va a 0**, y no es un descuido. El donante lo tiene a 80 porque está **filtrando una población**; aquí se está **mapeando una superficie**, y hacen falta los perfiles de todas las madres, también de las que fallan. Con 80, una madre con el 70 % de permutaciones rentables es rechazada y **su perfil no se escribe en ningún sitio**.

Cuánto tarda, y por qué parece colgado. Un SPP sobre una madre real de 22 parámetros a precisión de 1 minuto escribe sus 22 líneas de log **en tres segundos** y luego no dice nada durante **más de media hora**. `action=status` marca `Total tested 0` todo ese rato: no existe progreso por permutación en SQX. La señal de que está vivo es **la memoria del JVM**, que sube sin parar mientras acumula resultados — se midieron 23,6 → 36,9 GB sobre un heap de 48 GB.

	señal	**qué significa**
RSS del JVM subiendo, sin log	está trabajando	espera
RSS plano, sin log	**colgado**	párralo

⚠ **La memoria es el límite, no el tiempo.** El perfil acaba pesando ~20 MB en disco pero decenas de gigas mientras se construye. **No lances dos SPP a la vez en el mismo install.**

Cambiar cómo se retestea: el arnés

El retest lo ejecuta un **arnés** en el custodio, dentro del proyecto de workflow de la propia madre (regla dura 10, dueño 2026-09-29) — nunca un proyecto aparte, y nunca `Builder` ni `Retester`. `sqx.projects.builder --workflow` ya deja las tareas "SPP IS" y "SPP OOS" en ese proyecto (`sqx/projects/stages.yaml`); `harness.write()` reescribe **sólo la que le toca**, encontrada por su **título**, nunca por el nombre del fichero — el `Retest-Task1.xml` de un proyecto de workflow es su tarea real `OOS`, no un hueco libre. Se reconstruye con:

```
# reconocimiento SPP dentro de muestra, sobre el proyecto de la madre
python3 -m sqx.variants.harness --kind spp_is --project <PROYECTO> --output SPPRecon_Out \
  --chart "XAUUSD_DukasM1_Infinox M30 5" --spp

# añadiendo un cross-check en otro mercado (la plata, mismo bróker y feed)
python3 -m sqx.variants.harness --kind spp_is --project <PROYECTO> --output SPPRecon_Out \
  --chart "XAUUSD_DukasM1_Infinox M30 5" --chart "XAGUSD_DukasM1_Infinox M30 5" --markets --spp
```

flag	qué hace
`--kind`	`spp_is`, `spp_oos` o `retest`. De qué tarea del donante se copia, y qué tarea del proyecto destino se reescribe por título («SPP IS», «SPP OOS» u «OOS»)
`--project`	el proyecto de workflow de la madre — nunca uno hecho sólo para el arnés
`--chart`	repetible, **el principal primero**: `'SÍMBOLO TIMEFRAME SPREAD'`
`--markets`	activa el cross-check en mercados adicionales
`--spp`, `--spp-spread`, `--spp-step-pct`	activan el SPP con esa resolución y ese ±%

Después de escribir la tarea, sólo ella queda activa para el próximo `action=start` (`sqx.projects.stage.just`) — el proyecto lleva las tareas de todos los demás pasos del workflow, y `action=start` corre cualquiera que esté activa.

⚠ **El custodio tiene que estar parado**: SQX reescribe el `project.cfx` al salir, así que un cambio hecho mientras está levantado se pierde en silencio. El comando se niega a escribir si detecta el puerto abierto.

⚠ **Todos los `<Chart>` se reapuntan, incluidos los de cross-checks desactivados.** SQX resuelve todos los símbolos de la tarea al cargar el proyecto, y uno que no existe mata la tarea sin hacerla fallar: `Total tested 0`, sin error, para siempre.

⚠ **El arnés se construye copiando una tarea del donante que sí ha corrido**, nunca a mano. Una tarea a la que le falta `<Databanks retestSelected="false">` retestea "la selección", la selección está vacía, y reporta cero sin quejarse. Eso costó dos días.

⚠ **`SPPRecon_In` / `SPPRecon_Out` son databanks propios del reconocimiento, no las del proyecto**: la tarea `Build` real ya escribe su población en un databank llamado literalmente `Results`, y cargar ahí la única madre que se está reconociendo la mezclaría con esa población.

Qué produce

Todo en `AlgoData/pipeline/<proyecto>/<estrategia>/`:

fichero	qué es
`plan.csv`	el diseño completo: qué combinaciones y de qué estrato
`sqx/P*.sqx`	las estrategias fabricadas. **Es lo único pesado**, y lo que se borra al final
`manifest.parquet`	contrato C2: qué es cada fichero, leído **del disco**, no de lo que se pretendía
`retest_build.csv`, `retest_oos1.csv`, `retest_oos2.csv`	el panel de cada tramo, tal y como lo escupe SQX
`equity.parquet`	la P&L diaria del activo principal, **los tres tramos unidos** en una sola serie continua
`equity_markets.parquet`	lo mismo para cada mercado adicional, en formato largo (fecha, variante, mercado, tramo)
`segments.parquet`	las 82 métricas que SQX guarda, por variante **× tramo × mercado**, más las uniones y el `usable` de cada celda. **Aquí sí están los descartados**
`metrics.parquet`	contrato C3: las métricas de cabecera por tramo y por unión, unidas al manifiesto, **sin los backtests que no llegan a 50 operaciones**. Es la tabla del estudio
`estudios/wfc_<dentro>__<fuera>.html`	**el gráfico** de esa composición, p. ej. `wfc_build+oos1__oos2.html`. Una por composición: la segunda no pisa la primera
`estudios/wfc_<dentro>__<fuera>.md`, `.json`	lo mismo en texto y en el formato que pinta la ventana
`estudios/wfc.html`, `.md`, `.json`	copia de **la última** composición leída: es la que abre la ventana y la que lee el paso 20
`wfc.json`	rho, su intervalo, el veredicto, cuántos puntos se descartaron y la `composition`. **Se queda en la raíz**: el pipeline lee de aquí sus escalares (de la última)
`AlgoData/ledger/<estudio>.jsonl`	un renglón por tramo leído, paso 17, criterio `wfc/<composición>`
`state.json`	el libro mayor. Sobrevive al borrado de las variantes

Cómo se lee el resultado

Por pantalla ves esto:

```
  ran        python3 -m sqx.variants.execute --work .../Strategy_17-9-39
   | PROGRESS 2 despertando el custodian
   | PROGRESS 95 2000 de 2000 reteseadas
   | custodian stopped
   | PROGRESS 100 2000 variantes x 3 tramos, build: 2000 en disco, oos1: 2000 en disco, oos2: 2000 en
disco
  collected  python3 -m sqx.variants.collect --work .../Strategy_17-9-39
   | PROGRESS 100 4 resultados distintos entre 5 controles
  wfc        python3 -m studies.optimisation.wfc.report --work .../Strategy_17-9-39
   | PROGRESS 55 1001 puntos utiles de 2000
   | PROGRESS 100 rho 0.19 — no_fiable
   |
   | rho 0.19, intervalo [0.13, 0.25] entero por debajo de 0.3: optimizar en IS no compra
   | nada fuera
```

El custodio lo arranca y lo para **el propio comando**. Si ya estaba levantado, lo deja levantado: alguien lo estará usando.

Línea por línea, porque cada número decide algo:

- **`4 resultados distintos entre 5 controles`.** Es la comprobación más importante de todo el estudio y conviene entenderla. Una variante recién fabricada lleva dentro los resultados **del padre** — SQX los guarda en `settings.xml` y la fábrica solo reescribe los parámetros. Si el retest no llegó a ejecutarse, las variantes no salen vacías: salen **con los números del padre**, todas iguales, y el estudio entero parecería correcto. Por eso se cuenta. Si salieran los cinco iguales, la etapa aborta y te dice que no uses el lote.

(Cuatro de cinco, no cinco de cinco, es lo correcto: uno de los controles es un "par inerte", una copia que solo cambia en un parámetro congelado. Que dé idéntico es la prueba de que congelarlo estaba justificado.)

- **`1001 puntos utiles de 2000`.** Se descartan las combinaciones que operan menos de 30 veces en alguna de las dos muestras. No es limpieza: una combinación que apenas opera da un beneficio que mide una o dos operaciones, y son la mitad del lote: dejarlas dentro convierte la correlación en un artefacto de los rincones degenerados del grid.

- **`rho 0.19`.** La correlación de **rangos** (Spearman) entre el beneficio dentro y fuera. Se usan rangos y no valores para que una combinación desbocada no decida ella sola el resultado.

- **`intervalo [0.13, 0.25]`.** Y esto es lo que de verdad hay que leer. Es el margen de error al 95 % sobre ese 0,19. Aquí está **entero por debajo** del umbral de 0,30, así que la conclusión sí se sostiene: hay una relación positiva y real —el intervalo no toca el cero— pero **demasiado débil para ordenar nada**. Elegir parámetros por su beneficio dentro de muestra, en esta estrategia, no compra prácticamente nada fuera.

Los tres veredictos posibles:

veredicto	qué significa	qué haces
`fiable`	el intervalo entero por encima del umbral	optimizar en IS sirve; sigue adelante
`no_fiable`	el intervalo entero por debajo	optimizar no compra nada; cambia de estrategia
`indeciso`	el intervalo cruza el umbral	**fabrica más puntos**. No bajes el umbral

El umbral (`rho_floor`, por defecto 0,30) es tuyo y está en `studies/optimisation/wfc/config.yaml`. Cambiarlo **no obliga a repetir nada**: el estudio guarda los números, no los juicios, y solo se recalcula el veredicto.

Un ejemplo completo

```
# 1. ¿Qué costes se aplican? Bloqueante.
python3 -m core.assets XAUUSD

# 2. Once combinaciones, de punta a punta: diseño, fabricación, retest, panel y gráfico
python3 -m pipeline.run --project XAUUSD --databank SPP_IS --strategy "Strategy 17.9.39"

# 3. Mira el gráfico
xdg-open ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39/estudios/wfc.html

# 4. Si sale `indeciso`, sube los puntos y repite
sed -i 's/^  sample: 2000/  sample: 5000/' pipeline/config.yaml
rm -rf ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39
python3 -m pipeline.run --project XAUUSD --databank SPP_IS --strategy "Strategy 17.9.39"

# 5. Cuando ya tengas la respuesta, tira las variantes y quédate el registro
python3 -m pipeline.cleanup --project XAUUSD --strategy "Strategy 17.9.39" --apply
```

Coste medido el 2026-09-25, tres madres de USDJPY H1 × 5.000 variantes × 3 tramos × 10 mercados (USDJPY y los nueve de su familia) en el custodio, en el proyecto propio `USDJPY_variantes`:

fase	por madre	memoria	disco
`make` (fabricar 5.000 `.sqx`)	3 s	—	67 MB en local
`execute` (cargar, retestear, exportar, volcar)	**63–71 min**	JVM **84–87 GB**: toca su techo de 80 GB y el recolector aguanta	**7,5–14,7 GB** en el custodio
`equity`	83–88 s	3,7–4,4 GB	—
`collect`	17 s	2,1 GB	—
`execute --clear` (vaciar los cuatro databanks)	**3–4 s**	—	el custodio a 0
WFC (paso 17)	0,7 s	0,3 GB	—
CSCV (paso 18)	**90–170 s**	1,6 GB	—
se guarda por madre			**~520–870 MB** de parquet (el 80 % son las curvas de los 9 mercados)

50 madres: ~55–60 h de custodio, ~3 h de Python (casi todo CSCV) y ~25–45 GB de parquet. El custodio **tiene** que vaciarse entre una madre y la siguiente: sin eso serían 50 × ~10 GB en su disco. `execute --clear` lo hace con SQX parado, **después** de `equity` y `collect`, que leen esos ficheros.

Métricas para criterios — sin umbral todavía

Desde el 2026-10-04 el WFC calcula dos métricas más, visibles en dos tablas de su panel y como columnas del WFC en la tabla de Databanks. **Ninguna tiene umbral ni cambia el veredicto.**

- **Persistencia del top** (`exceso_persistencia_oos1`, `…_oos2`, `…_minimo`): de las variantes en el mejor 20 % del build por Net Profit, qué porcentaje queda en la mitad superior de cada ventana fuera de muestra (`persistence.top` y `persistence.keep` en `config.yaml`: tus X e Y). Un top «persiste» un 50 % aunque no

haya nada — es lo que sale por azar —, así que el dato es el **exceso** sobre ese 50 %, en puntos. «Se cumple en todas las ventanas» es mirar el exceso de la peor (`exceso_persistencia_minimo`). Solo se leen las ventanas que el ledger deja leer.

- **Mediana del ratio Sharpe OOS/IS** (`mediana_ratio_Sharpe`): la mediana, variante a variante, del Sharpe fuera entre el Sharpe dentro — una mediana de ratios, no un cociente de medianas —, solo con las variantes cuyo Sharpe IS supera `sharpe_floor` (0,1). Los dos Sharpe son los anualizados de SQX, comparables entre tramos de distinta duración. Ojo: entrar por tener un Sharpe IS alto regresa a la media por sí solo, así que sin decaimiento alguno queda algo por debajo de 1.

Al lado de cada valor está lo que daría **la misma familia sin ventaja**: el beneficio de cada día fuera de muestra se multiplica por +1 o −1 al azar, el mismo signo para todas las variantes, mil veces. Cada variante espera cero y siguen pareciéndose entre sí como antes — por eso ese rango es unas tres veces más ancho que el que saldría contando las variantes como independientes (± 27 puntos frente a ± 9). En el Nikkei, build → OOS1: el exceso de persistencia supera el percentil 95 de «sin ventaja» en 4 de 10 madres (por azar sería media), así que **se ofrece como criterio** en la zona Criterios, sin umbral; la mediana del ratio lo supera en 1 de 10 — un solo camino de precios mueve a todas las variantes a la vez — y **se queda como diagnóstico**. La persistencia solo dice que el ranking del build informa del de fuera, no que haya ventaja: una madre puede conservar su ranking mientras toda la familia pierde.

Qué NO te dice

- **No te dice qué parámetros poner.** Dice si elegirlos sirve de algo.
- **No es un walk forward de verdad.** Hay **una** partición IS/OOS, no una ventana que rueda. Mide si la superficie de parámetros se mantiene entre dos trozos de historia, no si se mantiene a lo largo del tiempo. Para eso está `studies/optimisation/wfm`.
- **No te dice nada sobre el futuro.** El OOS de este estudio es historia que la estrategia no vio al optimizarse, pero que tú sí has visto ya muchas veces. La única ventana que no está contaminada es `oos2`, y `ledger.gate` sólo la abre a los pasos que la tienen reservada.
- **Un `rho` alto no es una estrategia buena.** Puede correlacionar perfectamente y perder dinero en las dos muestras. Mira la nube: si está entera en el cuadrante de abajo a la izquierda, correlaciona las pérdidas.
- **Los costes son provisionales.** Hasta que los sustituyas, todos estos beneficios son aproximados. El orden entre combinaciones aguanta mejor que las cifras.

Si algo falla

síntoma	qué pasa	qué haces
`los controles devuelven el MISMO resultado`	el retest no llegó a ejecutarse y las variantes traen las métricas del padre	mira el log del custodio: `tail ~/Desktop/SQX_w2/user/log/StrategyQuant/log_*.log`
la etapa `ran` no avanza nunca	SQX arrancó el proyecto y no testea nada	el arnés `Retester` de W2 necesita `retestSelected="false"`; y hay que lanzarlo con `action=start`, **nunca** con `action=startOnlyTask`, que miente
`AlgoData N GB de 90 GB`	el presupuesto de disco frenó la corrida	libera espacio o sube el presupuesto en `perf/config.yaml`
`Symbol '...' doesn't exist`	una tarea nombra un símbolo que el install no tiene	pasa aunque el cross-check esté apagado: todos los `<Chart>` de la tarea se resuelven igual
`no manual page` en `checks.py`	hay un comando nuevo sin documentar	es esta misma regla: copia `_PLANTILLA.md`
`the following arguments are required: --family`	desde el 2026-09-27 cada lectura se apunta en el Ledger, y el estudio necesita la familia	pásala: `--family <plantilla>`. El pipeline y el botón de la ventana aún no la pasan
`composición no admitida`	has pedido algo fuera de la regla (`build` fuera, un tramo a los dos lados)	usa una de las cuatro de la tabla
`PermissionError: ledger: el paso 17 no puede mirar oos2`	la política ya no reserva `oos2` para el WFC	es decisión tuya, en `assets/_policy.yaml`; o lee `--inside build --outside oos1`

25. CSCV — ¿te fías de tu forma de elegir parámetros?

Qué pregunta responde

El estudio de la página 19 te dice si **esta** superficie de parámetros aguantó fuera de muestra. Éste responde a la pregunta de debajo, que es más útil y más incómoda: **tu manera de elegir los parámetros, ¿escoge sistemáticamente los que van a decepcionar?**

La diferencia importa. Una correlación se mide sobre una sola partición: entrenas con 2008–2017, compruebas con 2018–2022, y sale un número. Pero esa frontera la elegiste tú, y si la hubieras puesto en otro sitio habría salido otra cosa. El CSCV parte la historia en dieciséis trozos y prueba **las 12.870 formas** de usar ocho para elegir y ocho para juzgar. En cada una elige los parámetros como los elegirías tú, y anota en qué puesto quedaron. La proporción de veces que quedaron por debajo de la mediana es el **PBO**, la probabilidad de sobreajuste del backtest.

Se hace como lo hacen Bailey, Borwein, López de Prado y Zhu («The Probability of Backtest Overfitting»; decisión tuya del 2026-10-01): una matriz con una fila por **día de mercado** y una columna por variante —todos los días que tiene la curva, también los de P&L cero, unos 5.000 en los lotes de USDJPY de 2007 a 2026—, cortada en 16 bloques; en cada partición se calcula el **Sharpe** de cada columna en cada mitad, se elige el mayor dentro, λ es el logit de su puesto relativo fuera y el PBO es la parte de particiones con $\lambda \leq 0$. Al lado van las otras dos salidas del paper: la **regresión de degradación** (el Sharpe fuera de lo elegido contra su Sharpe dentro, un punto por partición) y la **probabilidad de pérdida** («pierde»). El Sharpe se enseña anualizado ×√252, sólo para leerlo: multiplicar por una constante no cambia ningún puesto ni el PBO. Hasta el 2026-09-30 la matriz era de semanas (×√52), y hasta ese mismo día los bloques eran 12 (924 particiones).

En la ventana, el CSCV enseña las fichas **«Argmax»** y **«Plateau center»**; la regla `random_profitable` ya no sale como ficha (sigue en la salida de terminal de más abajo).

Lo que **no** es del paper va marcado como extensión en el resultado: las reglas `plateau_centre` y `random_profitable`, el desplegable Sharpe/Sortino, la «Pendiente» ajustada sobre todas las variantes, la columna «vs. la mediana» y el percentil sobre la partición cronológica real. El PBO y sus partes salen siempre como porcentaje con un decimal (44,7 %), también en las piezas del veredicto; las fracciones crudas siguen en `cscv.json`, que es lo que lee el pipeline.

Y como lo que se juzga es la *forma de elegir*, se corre tres veces cambiando solo eso. La conclusión deja de ser «esta estrategia decae» y pasa a ser algo accionable:

> Con esta estrategia, elegir el máximo dentro de muestra falla el 30 % de las veces. Elegir el centro de la meseta falla el 4 %. **Cambiar de regla vale más que cambiar de estrategia.**

Cuándo lo usas, y cuándo no

Lo usas después de fabricar y retestear las variantes de una estrategia madre. El pipeline lo corre solo; a mano tiene sentido cuando quieres reexaminar un lote ya recogido, o probar qué pasa con otra regla de selección.

No lo usas para saber si la estrategia va a ganar dinero el año que viene. El CSCV rompe la cronología a propósito: mezcla trozos de 2009 con trozos de 2021 y juzga el procedimiento, no el futuro. Lo que mira hacia delante es el holdout y el walk forward matrix de la página 14.

No lo usas con pocas variantes. Por debajo de unos cientos de puntos utilizables, el puesto de lo elegido entre sus compañeros es demasiado grueso para significar nada.

Y no lo uses para separar un 45 % de un 55 %. Está medido: sobre paneles de puro ruido, el PBO de un panel concreto tiene una desviación típica de **0,21**, y doce paneles idénticos en todo menos la semilla dieron desde 0,25 hasta 0,92. El PBO distingue bien un 4 % de un 30 %; no distingue un 45 % de un 55 %.

Antes de empezar

Tres cosas, y las tres son bloqueantes:

1. **Los costes del símbolo tienen que estar acordados.**

`bash python3 -m core.assets XAUUSD`

Si sale con código distinto de cero, para.

2. **El lote tiene que estar recogido**, es decir, la carpeta de trabajo tiene que llevar `metrics.parquet` y `equity.parquet`. El segundo lo escribe la etapa `equity`, que lee los `.sqx` reteseados del custodio. Si solo tienes `metrics.parquet`, te falta esa etapa.

3. **El custodio tiene que haber volcado su databank a disco.** Lo hace `sqx.variants.execute` al terminar, antes de apagar la instalación. Si lanzaste el retest a mano sin ese volcado, en disco habrá menos ficheros que variantes reteseadas y la matriz saldrá coja. Se ve en `equity.json`: `n` tiene que coincidir con `n_returned` de `ran.json`.

No toca SQX, no arranca nada y no gasta la GPU ni los 95 núcleos: es una lectura de ficheros y unos segundos de Python. Lo puedes correr con la GUI abierta.

Cómo se ejecuta

Dos comandos, en este orden. El primero solo hace falta una vez por lote:

```
python3 -m sqx.variants.equity --work ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39
python3 -m studies.optimisation.cscv.report --work ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39
--family <familia>
```

flag	obligatorio	qué hace
`--work`	sí	la carpeta del lote. Es de donde lee y donde escribe; no hay opción de salida
`--family`	sí	la familia de plantillas del lote: la tercera parte del estudio del Ledger (`<símbolo>_<timeframe>_<familia>`) donde se apunta la lectura. El símbolo y el timeframe los saca del lote
`--blocks`	no	en cuántos trozos partir la historia, por encima de los 16 del ledger (12.870 particiones). 12 da 924, 10 da 252

Cuánto tarda, medido el 2026-10-01 sobre `Strategy_10.1.79` de `Test_USDJPY_donchianUpperCrossUp_H1` (500 variantes, 4.997 días):

el CSCV entero, diario, 16 bloques (3 reglas × 2 puntuaciones × 12.870 particiones + agrupamiento + bootstrap)	**6,6 s**, unos 0,5 GB por proceso
lo mismo con semanas (`--set cscv.period=W`)	**2,0 s**
una regla con una puntuación, diario	**0,4 s**; calculado fila a fila, como antes, serían ~79 s

Cada mitad se puntúa con las sumas de sus ocho bloques, no juntando filas: por eso pasar de semanas a días cuesta segundos y no minutos (`knowhow/research/cscv-daily-returns.md`).

Los mandos están todos en `studies/optimisation/cscv/config.yaml` , bloque `cscv` (el número de bloques vive en `ledger/thresholds.yaml` , porque es un umbral tuyo): el periodo de agregación, con qué métrica se ordena (`score`), cuántos bloques, qué reglas comparar y cuántos remuestreos. `period` es `D` (días, el del paper); `W` y `ME` siguen funcionando. **`score` admite `sharpe` o `sortino`** , y no admite Ret/DD a propósito: el Ret/DD crece con la longitud de la ventana (el retorno crece con el tiempo y el drawdown solo con su raíz), así que no sirve para comparar ventanas de distinta longitud. El Deflated Sharpe sigue siendo un Sharpe aunque ordenes por Sortino, porque está definido contra un máximo de Sharpes.

La frontera dentro/fuera de muestra no está ahí: sale de los tramos que `equity.json` midió sobre las propias curvas, para que no haya dos ficheros diciendo dos fechas. Cuál de las dos se usa lo dice `split_mode` de `engines/variants/config.yaml` — `oos2_only` pone la frontera al empezar `oos2` , `oos1_oos2` al empezar `oos1` (ver `09-optimizacion.pdf › 19-wfc` , las composiciones).

⚠ **El PBO NO cambia con la composición, y está comprobado sobre el código**: `cscv.run` parte el historial en 16 bloques y lee sus 12.870 particiones (C(16,8)), y no mira nunca la frontera declarada. Lo que sí se mueve son los cuatro números cronológicos que van al lado en el mismo informe: el **coste** de cada regla de selección, el **Sharpe desinflado**, el **número de pruebas independientes** y el **drift**. Los cuatro se calculan sobre `inside` / `outside` , que son justo las dos mitades que la frontera define.

Qué lee de `oos2` , y dónde queda apuntado

El CSCV lee siempre los tres tramos: `build` , `oos1` y `oos2` , porque sus particiones cortan la historia entera. Decisión tuya del 2026-09-27: el CSCV tiene que ver `build+oos1+oos2` , así que `CSCV` está en la reserva de `oos2` (`reserved_for` en `assets/_policy.yaml`). **Antes de abrir el lote** pregunta a la puerta (`ledger.gate.allow`) por los tres tramos; si un día quitas el CSCV de la reserva, se para ahí sin leer nada.

Y **cada lectura se apunta en el Ledger**: tres renglones del paso 18, uno por tramo, con el PBO de la regla titular, el DSR y la cronología usada en la nota. Con eso el paso 18 cuenta como hecho para la puerta ciega del 20 sin necesidad de `ledger.backfill --blind` , y el backfill ya no lo reconstruye dos veces.

Probado el 2026-09-27 sobre el lote `Strategy_9.27.83` de `Test_USDJPY_donchianUpperCrossUp_M30` (entonces con semanas y 12 bloques):

```
$ python3 -m studies.optimisation.cscv.report --work <lote> --family
Test_USDJPY_donchianUpperCrossUp_M30
PROGRESS 5 USDJPY M30: build, oos1, oos2 permitidos por el ledger
PROGRESS 10 construyendo la matriz de rendimientos por periodo
PROGRESS 20 argmax, plateau_centre, random_profitable: 924 particiones sobre 150 variantes
PROGRESS 80 contando cuantas pruebas independientes hay de verdad
PROGRESS 90 dibujando
PROGRESS 100 PBO 14% con argmax, DSR 0.99
  argmax               PBO  13.5%  percentil OOS  32.0 [3, 38]  pierde 0%
  plateau_centre       PBO  14.1%  percentil OOS  11.3 [3, 37]  pierde 0%
  random_profitable    PBO  49.4%  percentil OOS  41.9 [2, 96]  pierde 1%

$ python3 -m ledger.report --study USDJPY_M30_Test_USDJPY_donchianUpperCrossUp_M30
...
2026-09-27T20:20:34+00:00    18             cscv/pbo   build   150    150   1.0
2026-09-27T20:20:34+00:00    18             cscv/pbo    oos1   150    150   1.0
2026-09-27T20:20:34+00:00    18             cscv/pbo    oos2   150    150   1.0
...
  17 hecho · 18 hecho · 19 pendiente
```

⚠ Por lo mismo, **no uses un lote real para comprobar un cambio en el CSCV**: cada corrida gasta `oos2` y deja tres renglones que el Ledger no borra nunca. Para eso está `tests/test_cscv.py`.

Qué produce

ruta	qué es
`<work>/equity.parquet`	el P&L de cada variante, día a día. Una columna por variante
`<work>/equity.json`	cuántas curvas, cuántos días, la frontera usada y cuántas curvas no cuadran
`<work>/cscv.json`	todos los números, planos, para que el veredicto los lea por nombre
`<work>/estudios/cscv.html`	**el informe. Es esto lo que se mira** (hasta el 25-09, `<work>/cscv.html`). Al lado, `cscv.md` y `cscv.json` del contrato de estudios

Los cuatro se sobrescriben al volver a correr.

Cómo se lee el resultado

Así quedó la salida real de `Strategy 17.9.39` (2026-09-23, con semanas y 12 bloques):

```
PROGRESS 100 PBO 30% con argmax, DSR 0.66
  argmax               PBO  30.4%  percentil OOS   0.2 [0, 49]  pierde 24%
  plateau_centre       PBO   3.6%  percentil OOS  63.3 [31, 82]  pierde 3%
  random_profitable    PBO  38.6%  percentil OOS  68.3 [6, 98]  pierde 29%

479 variantes que valen 21 pruebas independientes; el orden se conserva con pendiente +0.49
```

Léelo fila a fila:

- **PBO.** Por debajo del 50 % la regla sirve; por encima, elegir así es peor que no elegir. `argmax` (quedarse con el mejor) falla el 30 % de las veces. `plateau_centre` (quedarse con el mejor después de promediar con sus vecinos) falla el 4 %. **Ésa es la conclusión del estudio.**

- **Percentil OOS.** Sobre la partición cronológica de verdad, en qué puesto quedó lo que la regla habría elegido. 50 es lo que da elegir a ciegas. El máximo dentro de muestra quedó en el **percentil 0,2**: de 479 variantes, casi la peor. El corchete es el intervalo del 95 %, y cuando es ancho es que el número no soporta una conclusión fina.
- **Pierde.** En qué fracción de las particiones lo elegido acabó la mitad reservada en pérdidas — la probabilidad de pérdida del paper. El PBO habla de puestos; esto habla de dinero.
- **Pruebas independientes.** Las 479 variantes no son 479 intentos: comparten árbol de reglas y casi todas sus operaciones. Agrupadas por lo parecido de sus rendimientos valen **21**. Ese número es el que usa el Deflated Sharpe, y de él sale el `DSR 0.66`: la probabilidad de que el Sharpe del mejor sea real teniendo en cuenta cuántas cosas se probaron para encontrarlo.
- **Pendiente.** Cuánto del orden dentro de muestra se conserva fuera, ajustado sobre **todas** las variantes. 1 sería conservarlo entero, 0 que el número de dentro no decía nada. Aquí +0,49: hay información, pero la mitad justa.

En la página HTML, la figura de arriba es la que se lee sin saber estadística: tres bandas, una por regla, y cada barra cuenta particiones. **Lo rojo, a la izquierda de la raya, son las veces que la regla eligió mal.** La banda de `plateau_centre` está casi entera a la derecha; la de `argmax`, a caballo.

Un ejemplo completo

```
$ python3 -m sqx.variants.equity --work ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39
PROGRESS 5 leyendo 962 .sqx de RetestOut
PROGRESS 84 900 de 962 curvas leidas
PROGRESS 92 comprobando cada curva contra lo que SQX guardo
PROGRESS 100 962 curvas x 3925 dias, 5.9 MB en 1 s, 172 con posicion abierta al final
```

Las «172 con posición abierta al final» no son un error y no bloquean nada. SQX valora a mercado una posición que sigue abierta en la última barra, y el beneficio neto solo cuenta lo cerrado, así que las dos cifras discrepan justo en esas variantes. El estudio descarta el último día por eso.

Lo que **sí** bloquea es que `mismatch` no sea 0: eso significa que las curvas no cuadran con lo que SQX guardó en la frontera, y entonces se está leyendo el resultado equivocado del fichero.

```
$ python3 -m studies.optimisation.cscv.report --work ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-
39   # 2026-09-23, antes de --family
PROGRESS 30 argmax: 924 particiones sobre 479 variantes
...
  argmax               PBO  30.4%  percentil OOS   0.2 [0, 49]  pierde 24%
  plateau_centre       PBO   3.6%  percentil OOS  63.3 [31, 82]  pierde 3%
-> .../estudios/cscv.html
```

Y la lectura en una frase: **en esta estrategia, la meseta es la regla y el máximo es una trampa.**

Lo que cambió al pasar a días, en tres madres de `Test_USDJPY_donchianUpperCrossUp_H1` (16 bloques; PBO de argmax / plateau_centre / random_profitable; calculado en memoria el 2026-10-01 sin escribir nada — los `cscv.json` de esos lotes siguen siendo los semanales hasta que se vuelva a correr el paso 18):

madre	semanas (982 filas)	días (4.997 filas)
10.1.79	43,9 / 32,7 / 49,9 %	44,7 / 34,9 / 49,6 %
1.26.46	38,5 / 39,3 / 51,1 %	31,1 / 29,3 / 51,3 %
6.35.56	42,0 / 36,8 / 49,9 %	50,1 / 43,8 / 50,9 %

Todos los cambios caben en la desviación de 0,21 que tiene el PBO de puro ruido: la lectura no cambia, cambia que ahora es la del paper.

Qué NO te dice

- **No dice si la estrategia funcionará hacia delante.** Es lo primero y lo más importante. Baraja la historia a propósito.
- **No dice que la estrategia sea buena.** Un PBO bajo sobre una superficie donde todo pierde significa que eliges de forma fiable dentro de un conjunto malo. Por eso está la columna «pierde»: el PBO es un puesto, no un euro.
- **No dice que `plateau_centre` sea la regla correcta siempre.** Lo dice de esta madre. El estudio existe precisamente para medirlo en cada una en vez de suponerlo.
- **No distingue un PBO del 45 % de uno del 55 %.** Ver «cuándo no lo usas». Por el mismo motivo, tampoco esperes el mismo número al cambiar de bloques: con esta misma estrategia, pasar de 10 a 12 bloques movió el PBO de `argmax` del 41 % al 30 %, y las dos cifras dicen lo mismo.
- **El recuento de pruebas independientes es una regla, no una medición.** Está explicado en `studies/optimisation/wfc/POSSIBLE_IMPROVEMENTS.md`, sección 6, con la alternativa.

Si algo falla

- **`alguna curva diaria no cuadra con el beneficio que SQX guardo`** — el lector está cogiendo la curva equivocada del `.sqx`, o el databank del custodio ya no es el de este lote porque corrió otra madre por encima. Comprueba `ran.json: databank_dir`.
- **`KeyError: 'databank_dir'`** — el `ran.json` es de antes de que existiera esta etapa. Vuelve a correr `sqx.variants.execute`, o añade a mano la ruta del databank y el número de ficheros.
- **`the following arguments are required: --family`** — desde el 2026-09-27 cada lectura se apunta en el Ledger. Pásale la familia de plantillas del lote. El pipeline y el botón de la ventana todavía no la pasan.
- **`PermissionError: ledger: el paso 18 no puede mirar oos2`** — la reserva de `oos2` ya no incluye el CSCV. Es tu fichero: `assets/_policy.yaml`.
- **`FileNotFoundError: ... segments.parquet`** — el lote es anterior a las columnas por tramo (p. ej. `pipeline/XAUUSD/Strategy_17-9-39`); de él no se sabe ni el símbolo ni el timeframe. Vuelve a cosechar con `sqx.variants.collect`.
- **Salen muchas menos variantes de las que reteseaste** — el custodio no volcó a disco. Está en «Antes de empezar», punto 3.

39. La nube de parámetros — ¿el punto elegido es un pico de suerte?

La ventana dibuja además mapas de calor de la nube (`studies/optimisation/cloud/heatmaps.py`), con la escala del rojo al verde.

Qué pregunta responde

Cuando la fábrica de variantes (`09-optimizacion.pdf › 18-variantes`) escribe miles de versiones de una estrategia y el custodio las retestea, lo que queda en disco no es una lista de candidatas: es **la superficie** de lo que esa lógica rinde según cómo la ajustes. Este comando la lee y responde cuatro cosas sobre ella, todas sobre la estrategia original y ninguna sobre las copias:

1. **¿Dónde está la original entre sus vecinas?** Si le gana a casi todas y además casi ninguna se le acerca, está encaramada en un pico — y un pico no se puede operar, porque el mercado real no te deja aterrizar en el punto exacto. Si tiene compañía, es una meseta.
2. **¿Quién mueve el resultado?** Qué parámetros deciden y cuáles son decorativos.
3. **¿Hay superficie que leer, o es ruido?** Si dos ajustes casi idénticos rinden muy distinto, el número de cualquiera de ellos es el sorteo, no el ajuste.
4. **¿Esa forma se sostiene año a año?** Que la familia gane mirando los quince años de golpe no dice nada sobre si gana todos los años, ni sobre si el orden entre variantes se mantiene.

Y añade una comparación: **la meseta entera repartida contra el punto único**. Si la original le saca mucho a la mezcla de sus vecinas, esa distancia es sobreajuste medido.

Cuándo lo usas, y cuándo no

Lo usas después del paso 16.5, cuando ya existen `metrics.parquet` y `equity.parquet` de un lote de variantes. Es gratis: no toca SQX, no consume CPU del worker, tarda segundos.

No lo usas para elegir. Es la regla dura del módulo y no es negociable: fabricar dos mil clones **es** una búsqueda, más grande que la del builder. Quedarse con el clon que mejor puntúa produce una estrategia *más* sobreajustada, no más robusta. El comando no devuelve nunca una variante mejor, y si algún día quieres mover los parámetros al centro de la meseta, eso lo decides tú, se anota, y se revalida sobre datos que este estudio no ha mirado.

No sustituye al WFC ni al CSCV (`09-optimizacion.pdf › 19-wfc` , `09-optimizacion.pdf › 25-cscv`). Aquéllos preguntan si optimizar sirve de algo; éste pregunta qué forma tiene lo que optimizaste.

Antes de empezar

Tiene que existir el directorio del lote, con estos dos ficheros dentro:

- `metrics.parquet` — lo escribe `python3 -m sqx.variants.collect --work <dir>`
- `equity.parquet` — lo escribe `python3 -m sqx.variants.equity --work <dir>`

SQX puede estar abierto o cerrado: da igual, no se le habla.

Cómo se ejecuta

```
python3 -m studies.optimisation.cloud.report \
    --work ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39
```

flag	obligatorio	qué hace
`--work`	sí	el directorio del lote, el mismo que usan `collect` y `equity`
`--out`	no	escribe todos los números en un JSON, para el ledger o para comparar dos lotes
`--set`	no	cambia un ajuste sin editar el YAML: `--set stability.period=2QE`

Tarda **1,9 segundos** con 998 variantes y 3.925 días, y ocupa **410 MB** de RAM (medido 2026-09-24). No toca SQX.

Qué produce

Por pantalla, cuatro bloques. Con `--out`, además un JSON con las lecturas, los índices de sensibilidad, la tabla por periodo y las medianas. No sobrescribe nada del lote.

Cómo se lee el resultado

La cabecera dice sobre cuántas variantes se lee y qué se ha dejado fuera. Los **canarios** salen siempre (son controles colocados a propósito en los extremos), y también las variantes que operan menos de 30 veces, que no son puntos de una superficie sino accidentes.

La línea de **colapsados** es la que más información da por carácter:

```
! colapsados por los filtros, fuera del modelo: DICrossShift1
```

Significa que, una vez fuera las variantes que apenas operan, ese parámetro **sólo tiene un valor vivo**. No es un fallo: dice que ese parámetro no elige entre bueno y malo, elige entre operar y no operar. Ningún índice de sensibilidad te habría contado eso.

A1 — dónde está el punto elegido.

número	qué es	qué es bueno
`q (rango)`	fracción de la nube a la que la original le gana	por sí solo no dice nada
`pi (20%)`	fracción que se queda a menos de un 20 % de ella	cuanto más alto, más meseta
`M_shrunk`	la mediana del entorno	**éste** es el número a usar hacia adelante, no el de la original

`q` alto **con** `pi` bajo es el pico de ruido. `q` alto con `pi` alto es una meseta, que es lo que quieres. Y `M_shrunk` es lo que de verdad cabe esperar en vivo: la original fue seleccionada, sus vecinas no.

A2 y A3 — quién manda y si hay superficie. `S` es cuánto explica cada parámetro por su cuenta y `S_total` incluye sus interacciones. `S_total` casi cero = parámetro decorativo, se puede congelar. `S_total` muy por encima de `S` = ese parámetro sólo funciona en combinación con otro, que es la firma de una lógica ajustada a mano.

Debajo, tres medidas de forma. `r2` es cuánto de la superficie es una función suave de los parámetros; la **rugosidad residual** es lo que sobra; el **desacuerdo entre vecinos** mide lo mismo sin suponer ninguna fórmula, y sólo se habla de superficie rugosa cuando las dos coinciden. La `pendiente` en el origen dice si la estrategia está donde el modelo suave pondría su óptimo.

B2 — periodo a periodo. `f_y` es qué fracción de la nube ganó dinero ese año; `q_origen` es dónde quedó la original ese año; `rho_siguiente` es cuánto del orden entre variantes sobrevive al año siguiente, y `deriva` cuánto se desplaza la región buena.

`rho` cerca de cero **es el hallazgo importante**: significa que la superficie se rebaraja cada año, y entonces optimizar los parámetros mirando el histórico es optimizar ruido, por muy bien que se vea el conjunto.

⚠ `días_activos` son días con movimiento, **no operaciones**. Un recuento de operaciones por año exige exportar los trades, que son noventa minutos contra el segundo y medio que cuesta este fichero. Los periodos por debajo de `min_active_days` no se leen.

C1 — la meseta contra el punto único. Se eligen K miembros **repartidos** por la meseta (no los K mejores: eso volvería a ser elegir), cada uno con 1/K del riesgo, y se compara. Una diferencia de Sharpe pequeña a favor del punto único es buena señal.

Un ejemplo completo

Sobre el lote de `Strategy 17.9.39`, que son 2.000 variantes fabricadas y 962 retesteadas:

```
998 variantes sobre ReturnDDRatio (build), 1002 descartadas (canarios y pocas operaciones); origen
P00000
! colapsados por los filtros, fuera del modelo: DICrossShift1

-- A1 · dónde está el punto elegido
original 2.680
           n  q (rango)  pi (20%)  M_shrunk
near   253.0      0.889     0.269     1.530
cloud  998.0      0.957     0.099     0.075
-> middling: el punto elegido no destaca en su propia nube

-- A2 y A3 · quién mueve el resultado, y si hay superficie que leer
         parámetro     S  S_total
    DICrossPeriod1 0.870    0.888
    ExitAfterBars1 0.073    0.098
    MomentumShift1 0.006    0.020
CBlock_SqzMmnInt21 0.002    0.013
          IsShift1 0.000    0.011
           IsBars1 0.002    0.009
   MomentumPeriod1 0.002    0.004
suavidad r2 0.902 · rugosidad residual 0.098 · desacuerdo entre vecinos 0.151 IQR
curvatura en el origen: pendiente 6.02, autovalores [-5.7 -2.1 -0.5  1.2  2.   3.7 21.9]
-> smooth: la superficie es una función suave de los parámetros
-> off_centre: el óptimo del modelo suave no está donde se sitúa la estrategia

-- B2 · la superficie, periodo a periodo
              f_y  q_origen  días_activos  rho_siguiente  deriva
2008-12-31  0.514     0.797         138.0         -0.048   0.215
2009-12-31  0.000     0.983         138.0          0.216   0.092
2010-12-31  0.746     0.962         148.0          0.245   0.124
2011-12-31  0.719     0.975         147.0          0.225   0.144
2012-12-31  0.537     0.625         142.0          0.542   0.085
2013-12-31  0.421     0.901         130.0          0.098   0.343
2014-12-31  1.000     0.291         122.0         -0.363   0.354
2015-12-31  0.017     0.570         135.0          0.535   0.169
2016-12-31  0.738     0.918         134.0          0.120   0.094
2017-12-31  0.356     0.656         128.0         -0.340   0.347
2018-12-31  0.480     0.436         151.0          0.577   0.060
2019-12-31  0.725     0.690         145.0         -0.528   0.350
2020-12-31  0.897     0.771         155.0         -0.357   0.249
2021-12-31  0.849     0.382         153.0         -0.016   0.264
2022-12-31  0.847     0.532         152.0            NaN     NaN
mediana rho +0.109 · mediana deriva 0.192 · peor periodo f_y 0.000
-> carried: hay periodos que sostienen el resultado y otros en los que la familia entera pierde
-> reshuffled: la superficie se rebaraja cada periodo: optimizarla es optimizar ruido
-> drift_low: la región buena se queda donde estaba de un periodo al siguiente

-- C1 · la meseta entera contra el punto elegido
meseta 68 variantes, 6 con curva, a 1/6 del riesgo
```

```
                   net  sharpe   drawdown
punto único  35328.266   0.314 -11310.234
mezcla       31274.974   0.284  -9455.880
diferencia de Sharpe +0.030
```

Cómo se lee esto de verdad, que es lo que importa:

- La original **no está en un pico**: le gana al 89 % de su vecindad y un 27 % de ella se queda a menos de un 20 %. Pero su Ret/DD de 2.68 no es la expectativa: la mediana del entorno es **1.53**, y ése es el número con el que hay que contar.
- **Un solo parámetro manda**: `DICrossPeriod1` explica el 87 % de la varianza. Los otros seis, casi nada. Eso son cinco grados de libertad que la estrategia no está usando para nada.
- La superficie **es suave** (r^2 0.90), así que la forma se puede leer. Pero la pendiente en el origen no es cero: el óptimo del modelo está en otro sitio.
- Y aquí está el hallazgo: **rho mediano +0.11**. El orden entre variantes casi no sobrevive de un año al siguiente, y en cuatro años es negativo. En 2009 **ninguna** variante de las 998 ganó dinero; en 2014, todas. Esta familia no tiene una configuración buena y otra mala de forma estable: tiene años buenos y años malos que se llevan a todo el mundo por delante.
- La mezcla de la meseta rinde casi igual que el punto único (0.284 contra 0.314) **con menos drawdown**. Poco sobreajuste en ese eje.

Qué NO te dice

- **Nada sobre otros mercados.** Las superficies por mercado (A4) y si la región buena coincide entre mercados (B3) necesitan un retest de variantes que lleve los cross-checks de `assets/_markets.yaml`. Encargo `15-superficies-multimercado.md`.
- **Nada sobre las reglas**, sólo sobre los valores. Quitar una condición o invertir la señal exige editar la lógica del `.sqx`: encargo `12-tests-estructurales.md`.
- **Mira todo el histórico, `oos2` incluido** (dueño, 2026-09-28). Sólo si lo corre un agente autónomo (`ALGO_AUTONOMOUS=1`) se cortan las curvas donde empieza `oos2`, y la cabecera dice cuántos días se han dejado fuera.
- **Los índices de sensibilidad se calculan sobre el modelo suave**, no sobre los backtests. Heredan su r^2: con un r^2 bajo describen una forma que los datos sólo sostienen a medias. Por eso el r^2 se imprime al lado.
- **No es una puerta.** Ninguna de sus lecturas descarta ni aprueba nada.

Si algo falla

- `KeyError: 'equity.parquet'` o fichero no encontrado — falta la cosecha: corre `python3 -m sqx.variants.equity --work <dir>` antes.
- **Muchísimas variantes descartadas** (aquí, la mitad) es normal y está en la cabecera: un lote se fabrica más grande de lo que vuelve del custodio, y encima la mitad de las combinaciones apenas operan.
- Si aparecen **dos o tres parámetros colapsados** en vez de uno, no bajes `min_trades` para taparlo: lo que te está diciendo es que tu rejilla de variantes está llena de combinaciones que no operan, y eso se arregla en el diseño, no aquí.

52. Superficies por mercado — ¿la región buena de parámetros es la misma en otros mercados?

> ⚠ **Costes provisionales.** Todo lo que sale de este comando hoy (2026-09-26) está calculado con los costes de fábrica de SQX en los 9 pares y en USDJPY, **comisión cero**. Cada resultado lo lleva marcado (`costs_provisional`), en la primera pestaña, en el veredicto y en los avisos. Léelo como provisional hasta que fijes los costes de the5ers.

Qué pregunta responde

Que una estrategia gane en otro mercado es poca prueba: puede estar simplemente comprada en algo que se parece. Que **la misma zona de parámetros** sea la buena en dos mercados distintos es mucha más, porque eso no lo produce la suerte de un backtest.

El lote de variantes del paso 16.5 ya se retesteó en los 9 mercados de `assets/_markets.yaml`. Este comando hace, para cada tramo (`build` y `oos1`), **una superficie por mercado** —el beneficio neto de cada variante en ese mercado— y compara las superficies de dos en dos con dos números:

- **rho** (Spearman): ¿los dos mercados **ordenan** las variantes igual? 1 = mismo orden, 0 = nada que ver, −1 = al revés.
- **J** (Jaccard del decil superior): de las **mejores** variantes (el 10 % de arriba) de cada mercado, ¿qué parte son las mismas? Dos órdenes al azar ya comparten algo: J ≈ 0,053. Por eso la tabla trae siempre `J_azar` y `J_banda` al lado.

Y termina en una decisión para el paso 20: **la región viaja**, **viaja a medias** o **no viaja**.

Cuándo lo usas, y cuándo no

Lo usas en el paso **18.5**, al mismo nivel que el WFC (17) y el CSCV (18): consume el mismo lote del 16.5, y su resultado se lee junto a ellos en el paso 20, o cuando quieras. Es gratis: no toca SQX.

No lo usas para elegir una variante, ni para decidir que el edge funciona en otro mercado (eso es el crossmarket, `07-otros-mercados-y-timeframes.pdf › 39-crossmarket-lote`): aquí sólo se pregunta si **el orden** viaja.

Lee `oos2`, como el WFC y el WFM (dueño, 2026-09-26): los tres tramos por defecto, y se lee cuando quieras; en el paso 20, junto a 17, 18 y 19. Con `--set segments=[build,oos1]` lee sólo los dos primeros. Los números de ejemplo de esta página son de antes de ese cambio y cubren `build` y `oos1`.

Antes de empezar

Un lote de variantes retesteado **con los cross-checks de mercado**. El retest del 16.5 los lleva por defecto: `wfc.markets: true` en `assets/_build.yaml` hace que `sqx.projects.wfc` meta los 9 mercados en cada una de las tres tareas, y `sqx.variants.equity` / `collect` dejan en el lote:

- `segments.parquet` — métricas por variante, mercado y tramo (de aquí sale la superficie)
- `metrics.parquet` — las variantes que lee el WFC
- `equity.parquet` y `equity_markets.parquet` — las curvas diarias (sólo para verificar)

SQX puede estar abierto o cerrado: no se le habla.

Cómo se ejecuta

```
python3 -m studies.optimisation.marketSurfaces.report \
    --work ~/Desktop/AlgoData/profiling/variantes-2026-09-25/real/23-1-53 \
    --family TestUSDJPY_Workflow_v1 \
    --out ~/Desktop/AlgoData/profiling/variantes-2026-09-25/marketSurfaces/23-1-53
```

flag	obligatorio	qué hace
`--work`	sí	la carpeta del lote de variantes
`--family`	sí	la familia de plantillas del lote; con el activo y el timeframe forma el estudio del ledger
`--out`	no	dónde escribir; si no lo pones, `<work>/estudios/`, como el WFC
`--set clave=valor`	no	cambia un ajuste de `config.yaml` para esa ejecución (p. ej. `--set rho_floor=0.4`)

Cuánto tarda (medido el 2026-09-26 sobre los tres lotes de USDJPY, 5.000 variantes × 10 mercados × 2 tramos): **9,4 a 12,0 s** y **2,2 a 2,4 GB** de memoria como pico. El fichero de curvas por mercado (228 millones de filas) nunca se carga entero: se lee mercado a mercado y tramo a tramo.

Qué produce

En la carpeta de `--out`:

fichero	qué es
`marketSurfaces.html` / `.md` / `.json`	el informe: la lectura, las dos matrices, una superficie por mercado, la verificación
`pairs.csv`	cada par de mercados de cada tramo: n, n_eff, rho con su intervalo, rho_neutral, J, J_azar, J_banda
`checks.csv`	las comprobaciones de la verificación
`verdict.csv` + `manifest.json`	la llamada de la madre, y qué lote se juzgó

Y **una fila en el ledger por tramo** (paso 18.5): cuántos mercados y cuántas variantes se miraron. Mirar mercados también es buscar; sin esa cuenta, un resultado bonito no se puede interpretar.

Cómo se lee el resultado



Superficies por mercado — 23-1-53

USDJPY H1 contra 9 mercados de _markets.yaml, tramos build, oos1.

La lectura Todos los pares Una superficie por mercado Verificación

la región no viaja

Sólo 0 de 9 en build, 4 de 9 en oos1 ordenan las variantes como el mercado principal con el decil superior compartido: lo que es bueno aquí no lo es allí. Ganar en otro mercado, si gana, no es la misma región. Costes PROVISIONALES: los 9 pares y el principal se retestearon con los defaults de fábrica de SQX, comisión CERO. El coste por operación mueve más a las variantes que más operan, así que puede reordenar una superficie: lee esto como provisional hasta que el dueño fije los costes de the5ers.

- **EURUSD build** 0.1063 J 0.22 (azar 0.05)
- **GBPUSD build** -0.05908 J 0.06 (azar 0.05)
- **USDCHF build** -0.001977 J 0.07 (azar 0.05)
- **AUDUSD build** -0.2125 J 0.00 (azar 0.05)
- **USDCAD build** -0.2042 J 0.09 (azar 0.05)
- **EURJPY build** -0.04298 J 0.08 (azar 0.05)
- **GBPJPY build** 0.1358 J 0.05 (azar 0.05)
- **AUDJPY build** -0.2863 J 0.02 (azar 0.05)
- **CADJPY build** -0.1233 J 0.01 (azar 0.05)
- **EURUSD oos1** 0.3797 J 0.13 (azar 0.05)
- **GBPUSD oos1** -0.01372 J 0.10 (azar 0.05)
- **USDCHF oos1** 0.1751 J 0.04 (azar 0.05)
- **AUDUSD oos1** -0.3651 J 0.00 (azar 0.05)
- **USDCAD oos1** -0.4279 J 0.02 (azar 0.05)
- **EURJPY oos1** 0.272 J 0.03 (azar 0.05)
- **GBPJPY oos1** 0.5669 J 0.33 (azar 0.05)
- **AUDJPY oos1** 0.4268 J 0.13 (azar 0.05)
- **CADJPY oos1** 0.3699 J 0.26 (azar 0.05)

costs_provisional Costes PROVISIONALES: los 9 pares y el principal se retestearon con los defaults de fábrica de SQX, comisión CERO. El coste por operación mueve más a las variantes que más operan, así que puede reordenar una superficie: lee esto como provisional hasta que el dueño fije los costes de the5ers. Afecta a: USDJPY, EURUSD, GBPUSD, USDCHF, AUDUSD, USDCAD, EURJPY, GBPJPY, AUDJPY, CADJPY.

family_only Los 9 mercados son de la misma familia macro que el principal (_markets.yaml, sin structural): pasar aquí es la prueba fácil; fallar sí dice algo.

Arriba, la llamada y un punto por mercado y tramo. Debajo, la tabla que la justifica:

La lectura

¿La región buena de parámetros en el mercado principal es también la buena en los otros? Costes PROVISIONALES: los 9 pares y el principal se retestearon con los defaults de fábrica de SQX, comisión CERO. El coste por operación mueve más a las variantes que más operan, así que puede reordenar una superficie: lee esto como provisional hasta que el dueño fije los costes de the5ers.

Aquí se ve la combinación por defecto de Tramo (build). Las demás están en el resultado JSON y en la ventana.

`segment: build`

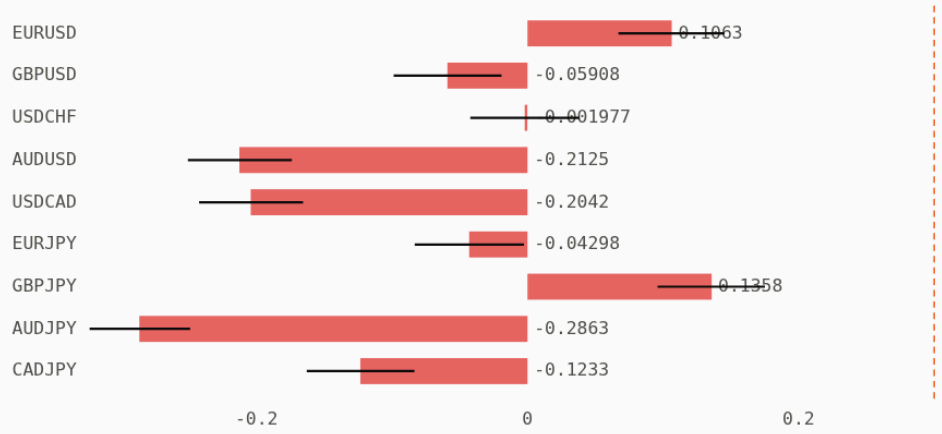
Principal contra cada mercado — build

MERCADO	N_EFF	RHO	RHO_LO	RHO_HI	RHO_SIN_EXP	J	J_AZAR	J_BANDA	SOLAPE	MADRE_PCT	ESTADO
EURUSD	2,465	0.1063	0.06714	0.1452	0.1026	0.2198	0.05274	0.07391	89/247	20.26	fail
GBPUSD	2,412	-0.05908	-0.09876	-0.01922	-0.07093	0.05908	0.05282	0.07317	27/242	15.94	fail
USDCHF	2,376	-0.001977	-0.04219	0.03824	0.04961	0.07449	0.05272	0.07449	33/238	36.95	fail
AUDUSD	2,385	-0.2125	-0.2505	-0.1739	-0.1241	0.002096	0.05275	0.07416	1/239	0.06001	fail
USDCAD	2,403	-0.2042	-0.2422	-0.1655	-0.218	0.08559	0.05279	0.0735	38/241	90.74	fail
EURJPY	2,359	-0.04298	-0.0832	-0.002629	-0.08264	0.07517	0.05266	0.07273	33/236	86.98	fail
GBPJPY	2,362	0.1358	0.09602	0.1752	0.1097	0.05333	0.05282	0.07483	24/237	49.91	fail
AUDJPY	2,354	-0.2863	-0.323	-0.2488	-0.173	0.02165	0.05277	0.07517	10/236	46.07	fail
CADJPY	2,365	-0.1233	-0.1628	-0.0834	-0.1088	0.01499	0.05275	0.07483	7/237	0.4801	fail

rho: Spearman del beneficio neto entre variantes, con su intervalo al 95 %. J: Jaccard de los deciles superiores; J_azar es lo que dan dos órdenes independientes y J_banda su percentil alto. madre_pct: dónde cae la madre en ese mercado (100 = la mejor). rho_sin_exp: el mismo rho quitando a cada mercado lo que explica su tiempo dentro — en estrategias sólo largas, parte del orden es exposición por deriva del mercado, no región de parámetros.

`segment: build`

rho con el principal — build



Todos los pares

Cada casilla compara dos mercados sobre las mismas variantes. La diagonal es 1 por construcción: es el control de que las variantes se emparejaron por su

columna	qué es
`n_eff`	variantes con un resultado distinto: dos combinaciones que dan el mismo backtest cuentan una vez
`rho`, `rho_lo`, `rho_hi`	el orden compartido y su intervalo al 95 %
`rho_sin_exp`	el mismo rho quitando a cada mercado lo que explica su **tiempo dentro** (ver abajo)
`J`, `J_azar`, `J_banda`	solape de los mejores, lo que daría el azar y su techo
`solape`	cuántas del decil superior comparten, de cuántas
`madre_pct`	dónde cae la estrategia original en ese mercado (100 = la mejor de todas)
`estado`	`pass`, `watch` o `fail`

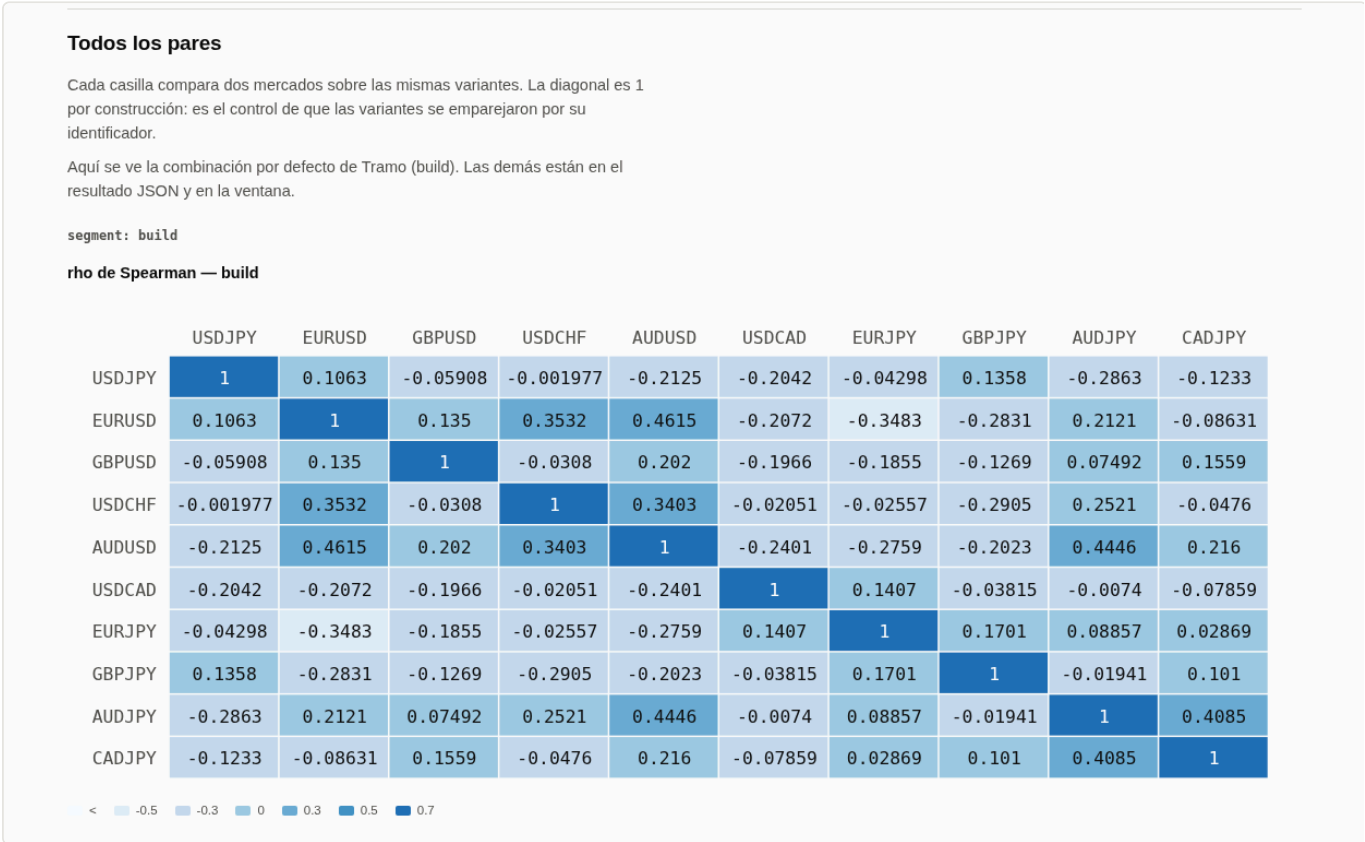
Un par **pasa** cuando el intervalo entero del rho está por encima de **0,30** (el mismo suelo del WFC) **y** el J supera la banda del azar. Se piden los dos porque fallan distinto: dos mercados pueden coincidir en qué variantes son **malas** (rho alto) y no compartir ninguna de las buenas (J al azar), y eso no es una región buena compartida. Un par **falla** cuando el intervalo entero está por debajo de 0,30.

La madre recibe:

- **la región viaja** — en cada tramo pasan al menos la mitad de los 9 mercados.
- **la región no viaja** — en ningún tramo llegan a la mitad.
- **viaja a medias** — en un tramo sí y en otro no.

El denominador son **siempre los 9 declarados**. Un mercado declarado que falte en el lote cuenta como no superado, nunca como ausente.

Las matrices enseñan todos los pares, no sólo contra USDJPY:



Todos los pares

Cada casilla compara dos mercados sobre las mismas variantes. La diagonal es 1 por construcción: es el control de que las variantes se emparejaron por su identificador.

Aquí se ve la combinación por defecto de Tramo (build). Las demás están en el resultado JSON y en la ventana.

`segment: build`

rho de Spearman — build

	USDJPY	EURUSD	GBPUSD	USDCHF	AUDUSD	USDCAD	EURJPY	GBPJPY	AUDJPY	CADJPY
USDJPY	1	0.1063	-0.05908	-0.001977	-0.2125	-0.2042	-0.04298	0.1358	-0.2863	-0.1233
EURUSD	0.1063	1	0.135	0.3532	0.4615	-0.2072	-0.3483	-0.2831	0.2121	-0.08631
GBPUSD	-0.05908	0.135	1	-0.0308	0.202	-0.1966	-0.1855	-0.1269	0.07492	0.1559
USDCHF	-0.001977	0.3532	-0.0308	1	0.3403	-0.02051	-0.02557	-0.2905	0.2521	-0.0476
AUDUSD	-0.2125	0.4615	0.202	0.3403	1	-0.2401	-0.2759	-0.2023	0.4446	0.216
USDCAD	-0.2042	-0.2072	-0.1966	-0.02051	-0.2401	1	0.1407	-0.03815	-0.0074	-0.07859
EURJPY	-0.04298	-0.3483	-0.1855	-0.02557	-0.2759	0.1407	1	0.1701	0.08857	0.02869
GBPJPY	0.1358	-0.2831	-0.1269	-0.2905	-0.2023	-0.03815	0.1701	1	-0.01941	0.101
AUDJPY	-0.2863	0.2121	0.07492	0.2521	0.4446	-0.0074	0.08857	-0.01941	1	0.4085
CADJPY	-0.1233	-0.08631	0.1559	-0.0476	0.216	-0.07859	0.02869	0.101	0.4085	1

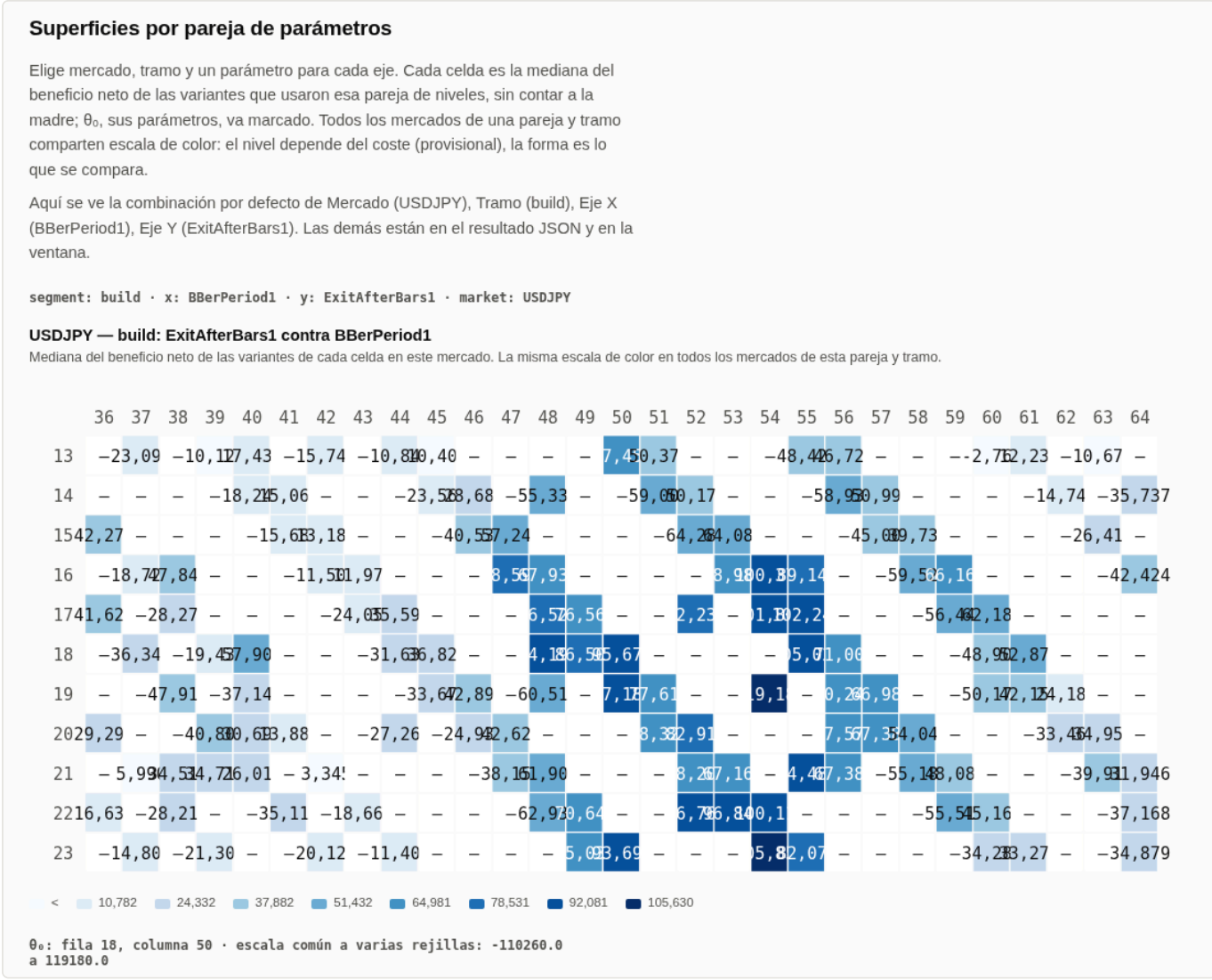
< -0.5 -0.3 0 0.3 0.5 0.7

⚠ `rho_sin_exp`**, léelo siempre al lado.** Las estrategias son sólo largas, así que parte de su beneficio es «cuánto tiempo está dentro» por «cuánto subió el mercado». Dos mercados que se movieron en sentidos contrarios ordenan las variantes **al revés** sólo por eso. Ejemplo real, madre 23-1-46, `oos1`: contra GBPJPY el rho es **+0,62**, y sin la exposición **−0,06**. Ese «pasa» era exposición, no región. La llamada usa el rho bruto (la métrica del WFC, como pediste); si prefieres que use el limpio, es una línea — dilo.

Las superficies de verdad y el mapa de consenso

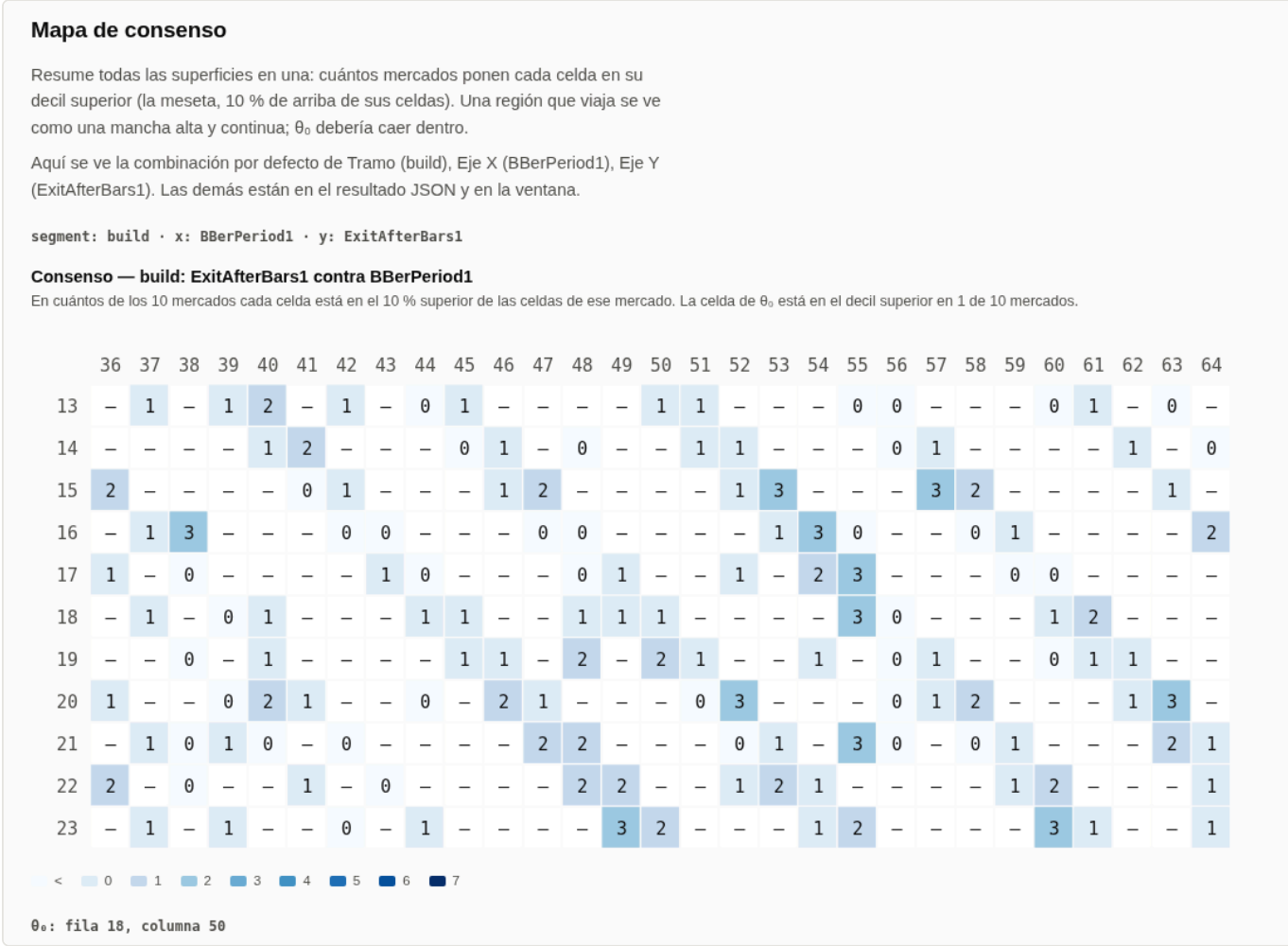
La pestaña «Una superficie por mercado» enseña **cómo se reparte** el beneficio de las variantes en cada mercado (un histograma), pero no **dónde** están las buenas. Dos pestañas más lo dibujan sobre los parámetros:

«Superficies por pareja de parámetros». Eliges mercado, tramo y, en dos desplegables, el parámetro del **eje X** y el del **eje Y** (todas las parejas están calculadas). Cada celda es la mediana del beneficio neto de las variantes que usaron esa pareja de valores, **sin contar a la madre**; θ_0 —los parámetros de la madre— va marcado debajo de la rejilla. **Todos los mercados de la misma pareja y tramo comparten escala de color**, para que puedas poner dos o tres uno al lado de otro y compararlos: el nivel depende del coste (provisional), así que lo que se compara es la forma, no el color absoluto. La escala va del rojo al verde. Un selector de **métrica** (Net Profit, Profit Factor, Sharpe, Ret/DD) cambia lo que pintan las superficies y los histogramas; las densidades de los mercados van superpuestas, cada una recortada según su propio mercado, y en las celdas estrechas las cifras salen cortas.



Lote `Test_USDJPY_donchianUpperCrossUp_M30` / `Strategy_9.27.83`, USDJPY, `build`, eje X `BBerPeriod1`, eje Y `ExitAfterBars1`. La escala común va de −110.260 a 119.180 $.

«Mapa de consenso». Resume todas las superficies en una: para cada celda, **en cuántos de los mercados está en el decil superior** de las celdas de ese mercado (la meseta, el 10 % de arriba, tu definición del 2026-09-27). Los niveles van de 0 a N, N = mercados leídos (10 con USDJPY).



- **Una región que viaja** se ve como una mancha alta y continua, y θ_0 cae dentro.
- **Números sueltos y bajos repartidos** —lo que sale aquí— es que cada mercado tiene su decil arriba en un sitio distinto.
- La nota de cada mapa dice dónde cae θ_0: en este ejemplo, **«La celda de θ_0 está en el decil superior en 1 de 10 mercados»**.

⚠ **Con un lote `minimal` este mapa no se puede leer.** El ejemplo es un lote de 150 variantes: la mayoría de las celdas tienen una variante o ninguna (los guiones), y un decil de celdas con una variante cada una es ruido. Hace falta el lote completo del diseño (5.000 variantes, 9 niveles por parámetro vivo), donde cada celda junta decenas.

Las dos pestañas van en `marketSurfaces.json` / `.html` junto a las otras cuatro. Con tres tramos y seis parámetros son 900 rejillas y 1,8 MB de JSON en un lote de 150 variantes; la página estática dibuja sólo la combinación por defecto y la ventana el resto.

Métricas para criterios — sin umbral todavía

Desde el 2026-10-04 la pestaña «La lectura» lleva, por tramo, una tabla más, y la tabla de Databanks dos columnas por tramo. **Ninguna tiene umbral ni cambia la lectura.**

- **Lift del Jaccard** (`lift_Jaccard_mediano_<tramo>`): L = J / J por azar, mercado a mercado, y la mediana entre mercados. El «J por azar» **no** es q / (2 − q): se mide sobre las propias superficies, dejando al otro mercado sin ventaja — el beneficio de cada día se multiplica por +1 o −1 al azar, el mismo signo para todas las variantes y, cada día, para todos los mercados, mil veces —, de modo que las variantes vecinas siguen pareciéndose y los mercados siguen correlacionados. Sale entre 0,04 y 0,09 según el mercado, no 0,053. Y es muy variable: en un solo mercado hace falta un lift de unos 3 para salirse de lo que da el azar, así que un mercado suelto con L = 2 no dice nada (tu AUS 200 a 0,21 entre doce); lo que se lee es la mediana.
- **La madre en el decil superior de los otros mercados** (`exceso_madre_en_top_<tramo>`): en qué porcentaje de los mercados secundarios el punto elegido (θ_0) está entre el 10 % mejor de las variantes de ese mercado, menos lo que saldría sin ventaja (en torno al 8 %, no un 10 % exacto).
- Un mercado con menos de 10 variantes utilizables en común con el principal no entra: su dato falta, no es un cero.
- El **mapa de consenso** ya marca θ_0 y dice en cuántos mercados su celda está en el decil superior. Es solo diagnóstico: no se usa para elegir parámetros nuevos.

En el Nikkei (diez madres, build y OOS1), ninguna de las dos se distingue de un mercado sin ventaja: la mediana del lift real queda en 0,4–0,6, por debajo incluso de la de «sin ventaja» (0,8), y el exceso de la madre supera el percentil 95 del azar en 1 de 10. Por eso **se quedan como diagnóstico** y no se ofrecen como criterio en la zona Criterios. Coincide con la lectura del propio estudio: la región no viaja.

Lo que no está hecho: la comprobación de unidades. Para saber si un parámetro es absoluto (puntos, pips, precio) o adimensional (periodos, múltiplos de ATR) hace falta un registro que hoy no existe — `sqx/variants/lengths.yaml` solo dice qué parámetros son longitudes en barras — y adivinarlo por el nombre sería suponer. Está preguntado al dueño.

Un ejemplo completo — las tres madres de USDJPY

```
$ python3 -m studies.optimisation.marketSurfaces.report --work .../real/23-1-53 --family
TestUSDJPY_Workflow_v1 --out ...
PROGRESS 5 USDJPY: build, oos1 permitidos por el ledger
PROGRESS 30 4999 variantes x 10 mercados
PROGRESS 100 la región no viaja — {'build': 0, 'oos1': 4}
[watch] Costes PROVISIONALES: ... Afecta a: USDJPY, EURUSD, GBPUSD, USDCHF, AUDUSD, USDCAD, EURJPY,
GBPJPY, AUDJPY, CADJPY.
segment market  n_eff    rho  rho_lo  rho_hi  rho_neutral     j    j0  j_hi  origin_pct state
  build EURUSD   2465  0.106   0.067   0.145        0.103 0.220 0.053 0.074      20.264  fail
  build AUDUSD   2385 -0.213  -0.251  -0.174       -0.124 0.002 0.053 0.074       0.060  fail
  ...
   oos1 EURUSD   2320  0.380   0.344   0.414        0.309 0.126 0.053 0.074      99.620  pass
   oos1 GBPJPY   2361  0.567   0.539   0.594        0.464 0.331 0.053 0.075      64.093  pass
   oos1 AUDJPY   2312  0.427   0.393   0.460        0.370 0.126 0.053 0.074      93.059  pass
   oos1 CADJPY   2301  0.370   0.334   0.405        0.300 0.259 0.053 0.074      32.306  pass
WALL 12.02 s  PEAK_RSS 2198856 KB
```

madre	build	oos1	llamada	lo que cambia sin la exposición
23-1-53	0 de 9	4 de 9 (EURUSD, GBPJPY, AUDJPY, CADJPY)	**no viaja**	lo mismo: los cuatro siguen por encima de 0,30 o rozándolo
23-1-46	0 de 9	2 de 9 (GBPJPY, CADJPY)	**no viaja**	los dos «pasa» se caen a ≈ 0: eran exposición
6-1-69	1 de 9 (USDCAD)	0 de 9	**no viaja**	lo mismo

La decisión: en las tres madres, la zona de parámetros buena en USDJPY **no** es la buena en los demás pares — y los 9 son de la misma familia macro, la prueba fácil. Si alguna de ellas gana en otro mercado en el crossmarket, no es «la misma región» y no cuenta como prueba de que el edge sea estructural. El coste de la lectura: ninguna estrategia se descarta aquí; es un dato más para el paso 20. Con costes provisionales.

Verificación (obligatoria, en la pestaña «Verificación» y en `checks.csv`):

madre	curva diaria sumada contra beneficio de SQX (27 celdas)	C3 del WFC contra la superficie	contra el mercado equivocado	diagonal
23-1-53	rho mínimo 0,9969	idéntico (0 $)	como mucho 0,44	20 de 20 = 1
23-1-46	rho mínimo 0,9931	idéntico	como mucho 0,58	20 de 20 = 1
6-1-69	rho mínimo 0,9988	idéntico	como mucho 0,49	20 de 20 = 1

USDJPY no está entre los 9 mercados, así que la comprobación del encargo («la columna del mercado principal de `equity_markets` = `equity.parquet`») se hace de la otra forma: cada curva diaria de cada mercado, sumada por tramo, contra el beneficio neto que SQX guardó para esa celda. **Los dólares no tienen por qué cuadrar** —en EURUSD `build` la curva queda 474 $ por debajo en la mediana— pero el orden sí, y eso es lo que se usa. Un emparejamiento equivocado daría lo que dice la columna «contra el mercado equivocado», no 0,99.

Qué NO te dice

- **Si el edge paga en otro mercado.** Sólo si el orden de las variantes viaja. Con costes de fábrica y comisión cero, además, el nivel del beneficio no vale nada; el orden algo más, pero un coste por operación distinto castiga más a las variantes que más operan, y puede reordenar.
- **Nada estructural.** Los 9 son de la familia de USDJPY (`_markets.yaml` no tiene `structural` para USDJPY). Pasar aquí es la prueba fácil; fallar sí dice algo.
- **Intervalos exactos.** Las variantes del diseño están agrupadas (vecindad, factorial, cobertura), así que el intervalo del rho y la banda del J son algo más estrechos de lo que deberían. Por eso la llamada se apoya en un tamaño (0,30), no en un p-valor.

Si algo falla

- **`PermissionError: ledger: el paso 18.5 no puede mirar oos2`** — el activo no reserva `oos2` para `MarketSurfaces` en `assets/_policy.yaml`. No se ha leído nada.
- **Aviso `market_absent`** — un mercado de `_markets.yaml` no está en el lote: el retest se hizo sin él (por ejemplo, porque le faltaban costes en `assets/symbols/`). Cuenta como no superado.
- **Aviso `verification`** — alguna curva no ordena como su beneficio, o la diagonal no es 1: la superficie puede no ser del mercado que dice. No leas el resultado; revisa la cosecha del lote.

34. Configurar la Walk Forward Matrix en SQX

> **Cambio del 2026-09-28.** La regla de «no quemar el oos2» ya no se aplica a humanos: puedes mirar el `oos2` y los pasos 17, 18 y 19 cuando quieras, desde la ventana o el terminal. Sólo un agente autónomo que decida solo (`ALGO_AUTONOMOUS=1`) sigue encontrando la puerta cerrada. El ledger sigue contando cada mirada.
>
> **Ojo con el nombre.** Esta página es la mitad de SQX del paso 19: **dejar la tarea `WFM` configurada** dentro de un custom project. Sacar lo que produce es `09-optimizacion.pdf › 09-wfm`, y leerlo —si predice o no— es `09-optimizacion.pdf › 14-walkforwardmatrix` (que forma parte del paso 20).

Qué pregunta responde

La Walk Forward Matrix reoptimiza la estrategia sobre una ventana que se desliza y corre cada elección sobre el tramo siguiente, **30 veces con particiones distintas**: 5 porcentajes de fuera de muestra (20 a 40 %) por 6 números de pasadas (6 a 16). La pregunta es si lo que optimiza bien predice lo que va bien después, y las 30 celdas existen para que la respuesta no dependa de haber elegido una partición con suerte.

Acaba en `oos2`

La ventana es `build..oos2`, o sea todo el histórico: el walk-forward necesita histórico delante para optimizar antes de correr nada. Puedes correrla y leerla cuando quieras, sin esperar a que el 17 y el 18 estén hechos.

Cuándo lo usas, y cuándo no

Úsalo cuando quedan pocos supervivientes, has hecho ya el 16.5/17 (variantes y WFC) y el 18 (CSCV), y no has mirado ninguno de sus resultados.

No lo uses para "ver qué sale" sobre una población grande. No es una cuestión de CPU: es que mirar gasta la muestra, y no hay otra.

La rejilla que escribe

ajuste	valor	de dónde sale
ventana	`build..oos2` — 2008.01.01 → 2026.08.30 en XAUUSD	`wfm.segment`
a qué precio se cobra	al tramo **`oos1`**, el intermedio — no al último de la ventana	`wfm.costs_segment`
`Param1` — % de cada pasada fuera de muestra	20 → 40 de 5 en 5 (5 columnas)	`wfm.oos_pct`
`Param2` — número de pasadas	6 → 16 de 2 en 2 (6 filas)	`wfm.runs`
celdas	**30**	el producto de los dos
`MaxTests`	5.000 por paso y celda	`wfm.max_tests`
`±distribution` y el paso	±35 %, un 4 % por paso → `maxSteps=18`	`wfm.distribution_pct`, `wfm.step_pct`
tipo de periodo	`percent` (=10) — **fijo**: los periodos van por nº de pasadas y % de OOS, nunca por días ni barras	`wfm.period_type`
tipo de optimización	`floating` (=15) — **fijo**: la ventana se desliza manteniendo su longitud	`wfm.optimization_type`
fidelidad del walk-forward	`exact_is_exact_oos` (=2) — "Exact IS, Exact OOS"	`wfm.wf_type`
qué se permuta	los parámetros **recomendados** por SQX, nada más	fijo
precisión	2 (1 minuto)	`wfm.precision`, igual que la doctrina

La ventana empieza en `build` a propósito: un walk-forward necesita histórico DELANTE del tramo reservado, porque la primera pasada tiene que optimizar sobre algo antes de poder correr sobre nada. Lo que se gasta es el final, `oos2`.

El cobro es la excepción del proyecto. Los demás spans se cobran al último tramo (el precio pesimista); éste se cobra a `oos1`, el intermedio, por decisión del dueño (2026-09-24): son 18,7 años de los que `oos2` es la cola de 3,7, y cobrar quince años al precio de cuatro no describe nada. Hoy `oos1` y `oos2` comparten `spread_oos` —10 puntos y slippage 5 en XAUUSD— así que el número sale idéntico; la línea decide el día que el activo declare un spread propio para `oos2`. Un intermedio de verdad, entre los 5 de `is` y los 10 de `oos`, habría que declararlo en `assets/symbols/<SIM>.yaml`: esa cifra la pone el dueño, no el catálogo.

La fidelidad es el atributo `type` del elemento `<WalkForward>`, que SQX llama "Walk-Forward type" y que aquí va en `exact_is_exact_oos` (decisión del dueño, 2026-09-24). Lo que cambia respecto al default de SQX no son los decimales: es **quién elige los parámetros de cada paso**. Con `simulated IS`, el ganador de cada paso sale de recortar a su ventana una única tanda de backtests sobre el rango entero —arrastrando el camino de la equity, que con sizing por ATR no es neutro, y los trades abiertos antes del borde—; con `exact`, cada paso hace su propia optimización sobre su ventana, que es lo que harías en vivo al reoptimizar. La pregunta de la WFM es si la optimización **predice** el tramo siguiente, y con IS simulado la "optimización" es un recorte.

No es territorio nuevo: la WFM que ya corrió el maestro llevaba `type="2"` con `MaxTests` 500 sobre cinco años, y terminó.

Y por eso `MaxTests` bajó de 15.000 a 5.000. Los dos mandos compran cosas distintas: `MaxTests` compra una elección mejor por paso y **nada que se pueda leer después** —SQX guarda sólo el ganador de cada paso, no la población que probó (medido, `knowhow/sqx-format/wfm-in-settings-xml.md`)—, mientras que `wf_type` cambia lo

que significa el número que sí se guarda. Habiendo que elegir dónde van los núcleos, van a la exactitud. Cuenta la factura antes de lanzar: son 5.000 backtests por paso, por los 6–16 pasos de cada celda, por 30 celdas, por estrategia, a precisión 2.

El criterio: cuándo aprueba una casilla y cuándo aprueba la estrategia

Esto es lo que la tarea del maestro NO tenía —corría con el elemento de condiciones vacío, y con cero condiciones SQX aprueba todas las casillas con un 100— y es lo que ahora escribe el comando. Decodificado del propio SQX el 2026-09-24 (`knowhow/conditions/wfm-acceptance.md` tiene la disección):

```
score de la casilla = round(condiciones cumplidas / condiciones activas * 100)
casilla aprobada    = score >= threshold_pct          (80 %)
estrategia aprobada = hay un rectángulo de 4x4 casillas, en alguna de sus 6 posiciones,
                      con 12 o más aprobadas          (grid_passing_size, min_squares)
```

Las diez condiciones, agrupadas por la pregunta que hacen:

pregunta	**condición**	**la cumplen**
¿gana dinero en lo que no vio?	`WF Net profit (OOS) > 0`	66 %
	`WF Profit factor (OOS) > 1,05`	45 %
¿se parece el fuera de muestra a lo que prometía la optimización?	`Stability Profit factor > 60 %`	80 %
	`Stability Net profit > 20 %`	30 %
	`Stability Max DD % < 150 %`	79 %
¿reoptimizar conserva la forma del original?	`Score Profit factor > 85 %`	83 %
¿está repartido el resultado entre pasadas?	`% de pasadas rentables > 50`	51 %
	`máximo beneficio en una pasada < 50 % del total`	11 %
	`mínimo de trades en una pasada > 20`	55 %
	`máximo DD % en una pasada <= 25`	100 %

La columna *"la cumplen"* es el porcentaje de las **150 casillas reales** del databank `WFM` del maestro (5 estrategias x 30 celdas) que pasan ese umbral: es la calibración, y está ahí para que se vea qué condición separa y cuál no. Con este juego y `threshold_pct: 80`, 2 de esas 5 estrategias encuentran el área 4x4 con 12 y 3 no.

⚠ **Los defaults de SQX habrían tirado todo.** Su `Stability Net profit > 60` lo cumple el 1 % de esas casillas —la mediana está en 8,6 %, porque el tramo de optimización es justo el que se eligió por ser el mejor— y su `% de pasadas rentables > 70`, el 9 %.

El criterio marca, no borra (comprobado el 2026-09-26). El comando deja `DeleteFailedStrategies` en `false`, así que la estrategia que no encuentra el área **se queda** en el databank, marcada por SQX: `Cross Check filter in 'Walk-Forward Matrix': Robustness score didn't pass.` El export lo guarda (`status.parquet`, y `failed_in_sqx` en el manifest) y el análisis lo muestra como aviso **"SQX la marcó como FAILED"** en la ficha de esa estrategia. El veredicto se toma en Python, como en el resto del proyecto. El mínimo de 12 casillas se queda (dueño, 2026-09-26).

Los umbrales se tocan en `assets/_build.yaml`, bloque `wfm.conditions`. Cada línea es `{read, metric, op, value}`, donde `read` elige de dónde sale el número: `oos` (las pasadas fuera de muestra concatenadas), `stability` (OOS contra optimización, en %), `score` (la WF contra el backtest original, en %) o `special` (las cuatro columnas que SQX calcula pasada a pasada).

Antes de empezar

- El proyecto tiene que ser **custom** y llevar la tarea titulada `WFM`, que viene del donante congelado. Si no la lleva, el comando se niega en vez de fabricarla: el bloque y sus condiciones vienen de una tarea que escribió SQX.
- El `.cfx` **no puede estar abierto** por una instancia (regla dura 4). El comando lo comprueba.
- Costes pactados: `python3 -m core.assets <SIMBOLO>` (regla dura 5).
- La ventana `oos2` del activo tiene que estar decidida en `assets/_policy.yaml`. En los activos con `from: null` no hay ventana y el comando revienta — que es lo correcto.

No toca SQX ni ningún worker: reescribe un fichero y tarda menos de un segundo.

Cómo se ejecuta

```
python3 -m core.assets XAUUSD                       # preflight, BLOQUEANTE
python3 -m sqx.projects.wfm XAUUSD \
    --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_estudio/project.cfx \
    --input "SPP OOS"
```

flag	obligatorio	qué hace
`<SIMBOLO>`	sí	el activo, para sus costes y su ventana `oos2`
`--cfx`	sí	el `project.cfx` del custom project
`--input`	sí	el databank que lee: los supervivientes que han llegado al paso 19
`--json`	no	lo mismo como JSON

Qué produce

Reescribe **en el sitio** el `project.cfx`: la tarea `WFM` queda activa, con el crosscheck `WalkForwardMatrix` encendido, la rejilla de 30 celdas escrita, la ventana `build..oos2` a los costes de `oos1` —también en `<Resources><Symbol>`, que es qué barras carga la tarea—, las condiciones del donante apagadas y las diez del catálogo escritas. Su salida va al databank `WFM`.

Cómo se lee el resultado

```
XAUUSD  XAUUSD  SPP OOS → WFM
  ✓ WFM  build..oos2 2008.01.01→2026.08.30  30 celdas (6 pasadas x 5 % OOS), tope 5000 tests
         por paso, 18 pasos de parametro, precision 2
      cobrada al tramo oos1: spread 10.0 — NO al ultimo tramo de la ventana
      fidelidad "Exact IS, Exact OOS (slow)" — 5000 tests por paso x 30 celdas, y cada paso
         es una optimizacion propia
      13 condicion(es) del donante apagadas y 10 propias escritas: una casilla aprueba con
         80 % de ellas cumplidas
      la estrategia pasa si encuentra 12 casillas aprobadas en un area de 4x4 — 6 posiciones
         posibles. Quien no lo encuentre queda marcada FAILED en SQX, NO se borra
         (DeleteFailedStrategies=false): el export lo anota y la ventana lo avisa

⚠ Y no se LEE hasta que 17, 18 y 19 esten los tres hechos.
```

- **`2008.01.01→2026.08.30`** — comprueba las fechas. Si el final no es el de `oos2`, la tarea no está mirando el tramo reservado; si el principio es 2023, el walk-forward se queda sin histórico para su primera optimización.
- **`cobrada al tramo oos1`** — si dice `oos2`, alguien cambió `costs_segment` y la ventana entera está corriendo al precio del tramo más caro.
- **`30 celdas`** — si sale otro número, alguien tocó los ejes en `_build.yaml`. No es un error; es un estudio distinto, y `studies/optimisation/wfm/` te dirá cuántas celdas leyó.
- **`13 del donante apagadas y 10 propias escritas`** — el 13 es lo normal en un clon del donante y un **0 es un aviso**: ese `.cfx` no viene del donante o alguien ya lo editó. Si el segundo número no es el de `wfm.conditions`, la matriz está juzgando con otra cosa.
- **`queda marcada FAILED en SQX, NO se borra`** — lo normal. Si alguna vez falta una estrategia en el databank de salida, es que `DeleteFailedStrategies` no estaba en `false`.
- **`⚠ esta tarea corre ademas: …`** — ese `.cfx` no pasó por la doctrina y correría otros crosschecks a la vez, cada uno con su factura. Vuelve a clonar con `sqx.projects.builder`.

Un ejemplo completo

```
python3 -m core.assets XAUUSD
python3 -m sqx.projects.wfm XAUUSD --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_estudio/project.cfx \
    --input "SPP OOS"

bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=start
name=XAUUSD_estudio','custodian')"
# ... termina ...
bin/sqx-worker.sh --role custodian stop

# el paso 20 empieza aquí, y sólo si el 17 y el 18 ya están hechos:
python3 -m sqx.export.export_wfm --project XAUUSD_estudio --databank WFM --role custodian
python3 -m studies.optimisation.wfm.report --project XAUUSD_estudio --databank WFM
```

Qué NO te dice

- **No dice si la estrategia gana dinero fuera de muestra.** Dice si su optimización **predice** su resultado posterior. Una estrategia puede ser rentable y completamente ciega a la vez.

- **No guarda la población de cada optimización**, sólo el ganador de cada paso. Lo más parecido a "cada juego de parámetros con su resultado IS y OOS" son los pasos del walk-forward, no el optimizador.
- **El último paso de cada celda no tiene resultado fuera de muestra**: se optimiza sobre la cola del histórico y ya no queda nada delante. El análisis los tira, y tirarlos es correcto.
- **No sustituye al WFC ni al CSCV.** Los tres se leen juntos, en el paso 20, y por eso el orden importa.

Si algo falla

lo que sale	qué significa
`el proyecto no lleva la tarea WFM`	el clon se hizo sin ella. Clona con `--tasks Retest` y el `--only` que la incluya
`el custodian tiene este proyecto abierto…`	párala antes; si no, SQX reescribe el `.cfx` al salir
un `KeyError` o fechas vacías	el activo tiene `oos2: {from: null}` en `_policy.yaml`. Sin ventana decidida no hay WFM
`un area de NxN no cabe en una matriz de …`	`grid_passing_size` es mayor que el eje más corto. SQX no avisaría: se quedaría con la mejor casilla suelta y descartaría todo
`min_squares N es imposible en un area de …`	pides más casillas de las que tiene el rectángulo
en el análisis, celdas con muy pocos pasos	`oos2` es corto (3,7 años en XAUUSD). 16 pasadas sobre esa ventana dejan tramos diminutos: míralo antes de leer una correlación por celda

9. Walk-Forward Matrix — sacar a Python todo lo que guarda el cross-check WFM

Qué pregunta responde

Cuando pasas una estrategia por el cross-check **Walk-Forward Matrix**, SQX prueba la misma estrategia con muchas combinaciones de "cuántos tramos" y "qué porcentaje de cada tramo es fuera de muestra", y te enseña una cuadrícula de colores. Este comando saca **toda** esa cuadrícula a CSV: cada celda, cada tramo dentro de cada celda, con qué parámetros se quedó el optimizador en ese tramo, cómo fue en la ventana de optimización y cómo fue después, y los trades de cada tramo.

Es lo que necesitas para responder en Python a "¿lo que va bien optimizando predice lo que va bien después?", que es la **correlación walk-forward**.

Cuándo lo usas, y cuándo no

Úsalo con un databank en el que hayas corrido el retest **Walk-Forward Matrix**. Las estrategias que estén en ese databank sin haber pasado el cross-check se saltan solas, sin error.

No sirve para sacar las miles de optimizaciones que SQX probó dentro de cada tramo. Eso no está guardado en ningún sitio: de cada tramo SQX se queda con el ganador y tira el resto. Lee la sección *Qué NO te dice*.

Antes de empezar

- Nada que preparar en la GUI. El comando **no toca el master**: copia las estrategias al worker y las exporta allí, así que lo puedes lanzar con SQX abierto.
- El worker tiene que estar parado. El comando lo arranca y lo para él solo.
- Ojo con el espacio: una estrategia con matriz de 30 celdas exporta unos **60.000 trades**. Sesenta estrategias son del orden de un giga en `~/Desktop/AlgoData`.

Cómo se ejecuta

```
python3 -m sqx.export.export_wfm --project XAUUSD --databank WFM
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	sí	el databank en el que escribió el retest WFM; entre comillas si tiene espacios

Las tablas se leen de los ficheros y tardan segundos. Los trades pasan por el worker: una arrancada de SQX y unos minutos, según cuántas estrategias haya.

```
$ python3 -m sqx.export.export_wfm --project XAUUSD --databank WFM
cells.csv              60 rows   308 columns
steps.csv             720 rows   314 columns
params.csv           6840 rows     7 columns
trades              65161 assigned, 0 unaccounted for
wrote /home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/WFM/2026-09-10/wfm
```

Qué produce

En `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/wfm/`:

archivo	qué es
`cells.parquet`	una fila por celda de la matriz (30 por estrategia): el `% OOS`, el número de tramos, y **152 estadísticos con prefijo `is_` y otros 152 con `oos_`**
`steps.parquet`	una fila por **tramo** de cada celda (360 por estrategia): las fechas de la ventana de optimización y de la ventana de after, y los mismos 152 + 152 estadísticos
`params.parquet`	una fila por tramo, una columna por parámetro, con el valor que eligió el optimizador en ese tramo
`check.parquet`	la comprobación del reparto de trades: trades asignados contra trades que SQX dice que hubo, tramo a tramo
`trades.parquet`	**todos** los trades de todas las estrategias en un fichero, con tres columnas añadidas: `result` (la celda), `period` (el tramo) y `sample` (`IS` o `OOS`). Se lee con `pandas.read_parquet` y se filtra por `strategy` y `result`
`status.parquet`	una fila por estrategia: `sqx_filter` (lo que SQX anotó: `Passed` o el motivo) y `sqx_failed`. Una estrategia que suspendió el criterio de la matriz **sigue aquí**, marcada; el manifest la lista en `failed_in_sqx`
`raw/`, `strategies/`	lo que escupió SQX antes de trocearlo, y las copias que se le pasaron. **Se borran solos** cuando la comprobación cuadra; si no cuadra, se quedan para mirarlos

La carpeta lleva fecha, así que **no sobrescribe** una exportación anterior.

Cómo se lee el resultado

Cada fila de `steps.parquet` es un experimento honesto: *"optimicé aquí, obtuve esto; luego lo solté ahí sin tocarlo, y obtuve esto otro"*.

index	is_NetProfit	oos_NetProfit	is_ProfitFactor	oos_ProfitFactor	is_SharpeRatio	oos_SharpeRatio	oos_NumberOfTrades
0	37445.44	4945.52	1.88	1.87	1.40	1.38	75.0
1	39706.14	5019.37	2.08	1.66	1.52	1.16	78.0
2	32116.86	2588.12	1.82	1.24	1.28	0.52	111.0
3	26233.45	2463.34	1.62	1.24	1.04	0.51	92.0
4	29347.05	-530.72	1.74	0.95	1.24	-0.11	89.0
5	34365.13	7325.51	1.87	1.64	1.25	1.19	114.0

Se lee así: en el tramo 0 la optimización dejó 37.445 $ con un profit factor de 1,88; en la ventana siguiente, sin tocar nada, hizo 4.945 $ con 1,87 en 75 operaciones. En el tramo 4 la optimización seguía dando 29.347 $ y 1,74 y el resultado fuera de muestra fue **−530 $** con 0,95: la optimización se veía igual de bien y el resultado ya no acompañó.

La correlación walk-forward es exactamente esto medido sobre todas las filas:

```
import pandas as pd
s = pd.read_parquet("steps.parquet")
s = s[~s.future]                      # el último tramo de cada celda no se llegó a correr
s.groupby("strategy").apply(lambda d: d.is_NetProfit.corr(d.oos_NetProfit))
```

Un valor cerca de 0 quiere decir que optimizar no informa de nada sobre lo que viene después. Cerca de 1, que sí. **En las dos estrategias medidas salió −0,04 y 0,06**, es decir, nada — con 330 tramos cada una, que es muestra suficiente para creérselo.

Antes de fiarte de los trades, mira `check.parquet`: la columna `assigned` tiene que ser igual a `stored` en todas las filas. En la comprobación que se hizo al construir esto, 39.873 trades repartidos en 30 celdas cuadraron **uno a uno**.

Un ejemplo completo

```
$ python3 -m sqx.export.export_wfm --project XAUUSD --databank WFM
cells.parquet         60 rows   308 columns
steps.parquet        720 rows   314 columns
params.parquet       720 rows    14 columns
trades             65161 assigned, 0 unaccounted for
wrote /home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/WFM/2026-09-10/wfm
```

Dos estrategias, 30 celdas cada una, 360 tramos cada una, y los 65.161 trades repartidos sin que sobre ni falte ninguno.

Qué NO te dice

- **No tiene las optimizaciones que se probaron.** Dentro de cada tramo SQX prueba hasta 10.000 combinaciones de parámetros; guarda la que ganó y tira las demás. No es que falte el comando: no existe el dato. Activar *"Don't store data for 3D charts"* **no cambia esto** — esa opción es del *Optimization profile* del cross-check SPP, y una estrategia pasada por WFM no tiene ninguno. Si lo que quieres son miles de juegos de parámetros con su rendimiento, lo que sí existe es `export_spp.py` (página 8), que ahora sí guarda las 4.000 permutaciones por estrategia — pero de una sola muestra, sin partir en IS y OOS.
- **El último tramo de cada celda no cuenta.** SQX lo optimiza pero no hay datos para correrlo: viene con `future = True` y sin ninguna columna `oos_`. Hay que quitarlo antes de correlacionar, o te comes 30 filas vacías por estrategia.
- **Las 30 celdas no son 30 experimentos independientes.** Todas se calculan sobre el mismo histórico y se solapan mucho entre sí. Sirven para ver si el resultado depende de cómo partas el periodo; no para multiplicar tu tamaño de muestra por 30.
- **34 de los 152 estadísticos siguen sin nombre**, y salen como `stat:f:63`. Valían cero en las 66 estrategias contra las que se calibró la tabla, así que no se pudieron identificar. Cuatro más llevan `?` porque su pareja vale exactamente lo mismo en todo el install y no se pueden separar.

Si algo falla

lo que ves	qué pasa
`cells.parquet 0 rows`	ninguna estrategia del databank pasó por WFM. Mira que sea el databank en el que escribió el retest, no el de origen
`trades ... N unaccounted for` con N > 0	el reparto por fechas no cuadra con lo que dice SQX. **No uses esos trades**; mira `check.parquet` para ver en qué tramo se descuadra
el worker no arranca	quedó uno colgado de antes: `bin/sqx-worker.sh stop` y vuelve a lanzarlo

14. Walk-Forward Matrix — ¿reoptimizar sirve de algo?

Qué pregunta responde

El cross-check **Walk-Forward Matrix** de SQX hace esto: coge un tramo de historia, optimiza los parámetros ahí, y luego corre esos parámetros en el tramo siguiente, que no ha visto. Repite avanzando. Y lo hace con 30 combinaciones distintas de "en cuántos tramos parto la historia" y "qué porcentaje de cada tramo dejo fuera de muestra".

Este comando lee todo eso y contesta **una sola pregunta, que es la que justifica reoptimizar**:

> Cuando el optimizador encuentra unos parámetros que van muy bien en el tramo de optimización, ¿van también mejor en el tramo siguiente?

Si la respuesta es que sí, reoptimizar tiene sentido. Si es que no, estás gastando cómputo para nada. Y si la respuesta es que **van peor**, reoptimizar te está eligiendo activamente lo que va a fallar.

De paso mide la **deriva del óptimo**: cuántos parámetros cambia el optimizador de un tramo al siguiente.

Cuándo lo usas, y cuándo no

Úsalo con un databank donde hayas corrido el cross-check **Walk-Forward Matrix**, y después de sacar los datos con `python3 -m sqx.export.export_wfm`.

No te dice si la estrategia funciona. Te dice si el *procedimiento de reoptimizarla* funciona. Son cosas distintas: una estrategia puede ganar dinero con parámetros fijos y salir `blind` aquí.

No sirve para sacar conclusiones generales con pocas estrategias. El export de ejemplo tiene dos. Dos no son una población.

Antes de empezar

- **No toca SQX.** Puedes lanzarlo con la GUI abierta.
- Tiene que existir `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/wfm/` con sus Parquet. Si no:

```
python3 -m sqx.export.export_wfm --project XAUUSD --databank WFM
```

Cómo se ejecuta

```
python3 -m studies.optimisation.wfm.report --project XAUUSD --databank WFM
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	no	carpeta del export, **con guiones bajos**. Por defecto `WFM`
`--day`	no	fecha del export. Por defecto, el más reciente
`--strategy`	no	una sola estrategia (`"Strategy 1.26.74"`): escribe solo `estrategias/<nombre>.json` y su página, idénticos a los que da el run de todo el export, **sin tocar** `wfm.json` , `verdict.csv` , `cell_correlations.csv` ni el manifiesto. Es lo que hace «Run solo esta estrategia» en la ventana

Tarda unos segundos.

Qué produce

En `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/wfm/` (hasta el 25-09, sueltos en la carpeta del día):

- **`wfm.html`** / **`wfm.md`** / **`wfm.json`** — el informe (antes `walkforwardmatrix.md`), y el mismo resultado en el contrato de estudios, que es lo que pinta la ventana.
- **`estrategias/<nombre>.html`** / **`.json`** — la matriz de una estrategia.
- **`verdict.csv`** — una fila por estrategia con su identidad y su veredicto, para `/curate` .
- **`cell_correlations.csv`** — la correlación de cada una de las 30 celdas, por si quieres mirarlas a mano.

Cómo se lee el resultado

```
Strategy 1.19.29       blind     rho=+0.076 [-0.034, +0.193]  celdas= 30  deriva=70%
Strategy 4.33.46       perverse  rho=-0.505 [-0.683, -0.339]  celdas= 30  deriva=78%
```

El veredicto es una de tres:

veredicto	qué significa	qué hacer
`predicts`	el intervalo está **por encima de cero**: lo que optimiza mejor va mejor después	reoptimizar sirve
`blind`	el intervalo **cruza cero**: la optimización no dice nada de lo que viene	reoptimizar no hace daño, pero es tirar cómputo
`perverse`	el intervalo está **por debajo de cero**: lo que optimiza mejor va **peor** después	reoptimizar te está eligiendo lo que va a fallar

El intervalo es lo importante, no el ρ. Está calculado remuestreando **celdas**, no tramos. La razón: las 30 celdas parten la *misma* historia de formas distintas, así que no son 30 observaciones independientes. Si se agruparan los 660 tramos en un solo número, el intervalo saldría varias veces más estrecho y sería mentira.

La deriva es qué porcentaje de los parámetros cambia el optimizador de un tramo al siguiente. Un 70-78 % significa que cada reoptimización elige una estrategia sustancialmente distinta.

Y las dos cosas se leen juntas: si la superficie no tiene señal fuera de muestra, no hay nada que sujete al optimizador en su sitio, y por eso salta tanto. No son dos hallazgos, es uno.

La tabla de otras métricas es el control:

strategy	metric	rho	low	high	share_negative
Strategy 4.33.46	ReturnDDRatio	-0.505	-0.683	-0.339	0.933
Strategy 4.33.46	NetProfit	-0.112	-0.191	-0.035	0.667
Strategy 4.33.46	SharpeRatio	-0.213	-0.306	-0.124	0.733
Strategy 4.33.46	ProfitFactor	-0.241	-0.331	-0.153	0.8

Negativa en las cuatro, y el 93 % de sus celdas negativas en la principal. Si solo saliera negativa en una, sería una propiedad de esa métrica y no de la estrategia. **Mira siempre esta tabla antes de creerte el veredicto.**

La matriz de SQX y sus diez objetivos, en la ventana

Desde el 2026-10-01 el export guarda, para cada celda, **las diez condiciones con las que SQX la juzgó** —su valor, el umbral y si la cumple— y la regla del área que llevaba la estrategia. SQX no guarda esos números (los recalcula al pintar), así que Python los rehace con sus mismas fórmulas; se comprobaron iguales a los de SQX en 14 columnas y 17 estrategias, y el veredicto final coincide con el que marcó SQX en 227 de 227 estrategias.

En la ficha de una estrategia, pestaña **«La matriz»**:

perverse -0.4945

0.9667 informa **0.6667** vigilar

Strategy 9.10.57 · calculado 2026-10-01T17:38:01 · config 56b496d4e2502909 · 6.58 s

Informe de lo que ves

La matriz | Los objetivos

Matriz pasa / no pasa

Cada celda: condiciones cumplidas sobre activas; aprueba con el 80 %. Pasa el ratón por una celda para ver cada condición con su valor.

Pasadas \ % OOS	20%	25%	30%	35%	40%
6	▣ 8/10	▣ 9/10	▣ 9/10	▣ 8/10	8/10
8	▣ 8/10	◆ 9/10	▣ 10/10	▣ 5/10	5/10
10	▣ 9/10	▣ 10/10	▣ 9/10	▣ 6/10	5/10
12	▣ 9/10	▣ 8/10	▣ 7/10	▣ 9/10	6/10
14	6/10	8/10	5/10	9/10	8/10
16	7/10	6/10	6/10	9/10	5/10

Aprueba: necesita 12 celdas aprobadas en un rectángulo 4×4. Mejor rectángulo 4×4 (▣): pasadas 6-12, % OOS 20-35, con 13 aprobadas. Combinación recomendada (◆, el centro de ese rectángulo): 8 pasadas con 25 % fuera de muestra. Coincide con lo que marcó SQX.

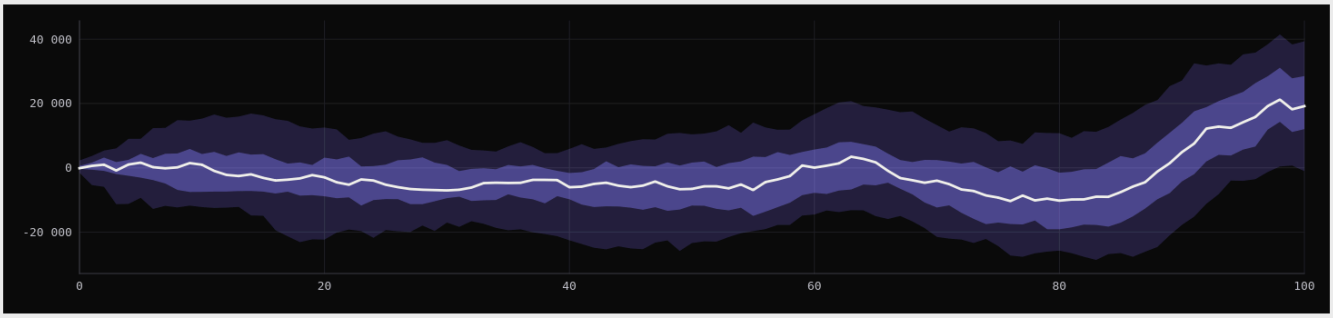
Qué condición manda

Cuántas celdas cumple cada condición: la que menos cumple es la que suspende la matriz.

Condición	Celdas que cumplen	Mediana de las celdas
WF beneficio neto > 0	28 de 30	19 170
WF PF > 1.05	20 de 30	1.08
Estabilidad del PF > 60	30 de 30	80.15
Estabilidad del beneficio neto (eficiencia WF) > 20	18 de 30	23.9
Estabilidad del DD máx. % < 150	24 de 30	128.7
Score del PF > 85	30 de 30	102
% de pasadas rentables > 50	14 de 30	50
mayor beneficio de una pasada, % del total < 50	2 de 30	106.1
mínimo de operaciones en una pasada > 20	30 de 30	46.5
peor DD % de una pasada ≤ 25	30 de 30	13.49

Equity de las 30 celdas fuera de muestra

Eje: % de operaciones fuera de muestra completadas, no fecha — las celdas no comparten calendario. La línea marcada es la mediana entre celdas; las bandas, p2.5-p97.5 y p25-p75 entre celdas. No hay un único equity que sea 'la' WFM.



El contenido de las celdas

Una fila por métrica: cómo se reparte entre las celdas. Sharpe clásico sobre todo el tramo fuera de muestra de la celda: P/L diario en días laborables (los días sin operaciones cuentan como 0), media / desviación × √252. Las celdas reparten la misma historia: la dispersión describe, no es un error estándar.

Métrica	Mediana	P25	P75	Peor	Mejor
beneficio neto (USD)	19 170	12 018	28 519	-1 739	40 220
PF	1.08	1.05	1.13	0.99	1.21
Sharpe anual	0.22	0.13	0.36	-0.02	0.52
DD máx. (USD)	23 570	17 717	26 994	51 448	12 367
operaciones	770	706.5	866	617	1 027
% de celdas en beneficio (30 celdas)	93.33	—	—	—	—

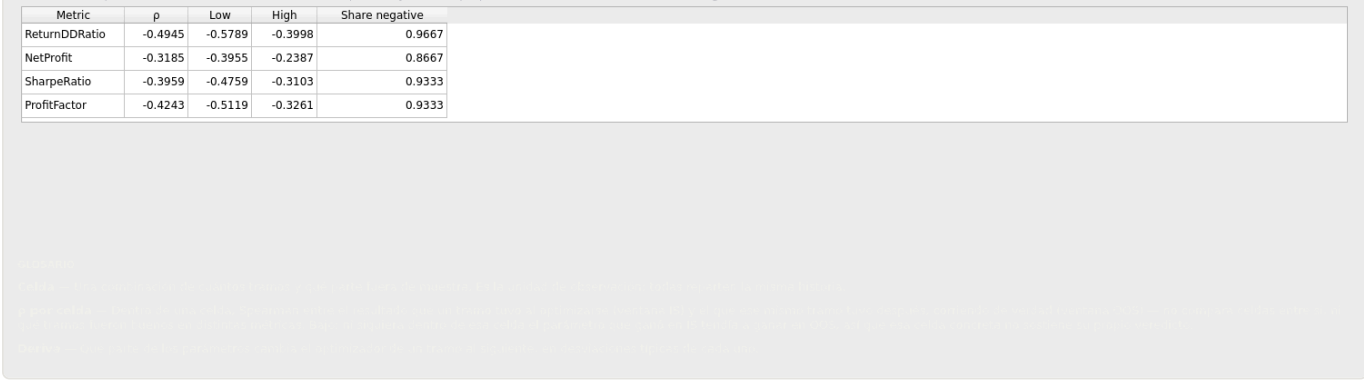
Lo peor celda a celda — Strategy 9.10.57

Filas: número de tramos. Columnas: parte fuera de muestra.

	20%	25%	30%	35%	40%
6	-0.7714	-0.08571	-0.5429	-0.7143	-0.7143
8	-0.4286	-0.6905	-0.4048	-0.7619	-0.7857
10	-0.3212	-0.6242	-0.8424	-0.5471	-0.6
12	-0.5455	-0.5175	-0.5734	-0.1608	-0.1469
14	-0.4374	-0.5545	-0.3278	-0.121	0.0154
16	-0.5941	-0.4559	-0.4176	-0.03529	-0.005882

La misma correlación en otras métricas

Un veredicto que sólo se sostiene en la métrica con la que se leyó es una propiedad de esa métrica, no de la estrategia.



Metric	ρ	Low	High	Share negative
ReturnDDRatio	-0.4945	-0.5789	-0.3998	0.9667
NetProfit	-0.3185	-0.3955	-0.2387	0.8667
SharpeRatio	-0.3959	-0.4759	-0.3103	0.9333
ProfitFactor	-0.4243	-0.5119	-0.3261	0.9333

- **La matriz pasa / no pasa.** Filas = número de pasadas, columnas = % fuera de muestra. Cada celda dice `cumplidas/activas` en verde (aprobada) o rojo. ▣ marca el mejor rectángulo y ◆ la combinación recomendada, su centro: es la que SQX te propone para reoptimizar en vivo. **Pasa el ratón por una celda** y ves las diez condiciones con su valor y ✓/✗. Desde el 2026-10-04 la matriz es compacta y la **matriz de ρ** va a su lado; el veredicto de SQX sale como «SQX: Failed» / «SQX: Pass» y un selector **«Celda»** elige la celda de las curvas de abajo.
- **La frase de debajo** dice si aprueba, dónde está el rectángulo y si coincide con lo que marcó SQX. Si alguna vez pone «NO coincide», avisa: la reconstrucción tiene un fallo.
- **«Qué condición manda»**: cuántas celdas cumple cada condición. La que menos cumple es la que te tumba la matriz (en el ejemplo, «mayor beneficio de una pasada < 50 %»: 2 de 30).
- **El conjunto de las celdas**: las curvas fuera de muestra de las 30 celdas juntas (mediana y bandas) y una sola tabla con la mediana, p25, p75, peor y mejor de beneficio, PF, Sharpe, DD y operaciones, más el % de celdas en beneficio.

Pestaña **«Los objetivos»**: un desplegable con **una matriz por objetivo**. Primero las diez condiciones (C1…C10), cada celda con su valor en verde o rojo frente a su umbral; luego las métricas walk-forward que SQX enseña sin umbral. La que más te interesa de esas:

objetivo	qué es
Estabilidad del beneficio neto (eficiencia WF)	beneficio por día fuera de muestra ÷ beneficio por día optimizando, ×100. Es la *walk-forward efficiency* de Pardo: por debajo de 50-60 el sistema rinde en vivo menos de la mitad de lo que promete
Estabilidad del PF / Sharpe / DD	lo mismo con esa métrica; 100 = igual fuera que dentro
Score del …	el walk-forward entero frente al backtest con parámetros fijos, ×100: por encima de 100, reoptimizar mejora a no tocar nada
% de pasadas rentables, mayor beneficio de una pasada, mínimo de operaciones, peor DD de una pasada	las «especiales» de SQX, pasada a pasada
Estabilidad de parámetros (SQX)	0-1, cuánto se parecen los parámetros elegidos de un tramo al siguiente

Pestaña **«Contra el activo y el azar»**: la pregunta de si merece la pena operarla. Juzga la **estrategia original** —sus parámetros fijos, no una celda de la matriz: escoger una de las 30 curvas después de verlas ya sería sobreajustar— y **sólo en el `oos2`**: el `oos1` ya se usó para cribar la población, y un tramo que sirvió para elegir no puede servir para juzgar.

perverse -0.4945

0.9667 informa **0.6667** vigilar

Strategy 9.10.57 · calculado 2026-10-01T19:10:24 · config 7d0c7ebf8e2c68d9 · 8.16 s

Informe de lo que ves

Contra el activo y el azar | La matriz | Los objetivos

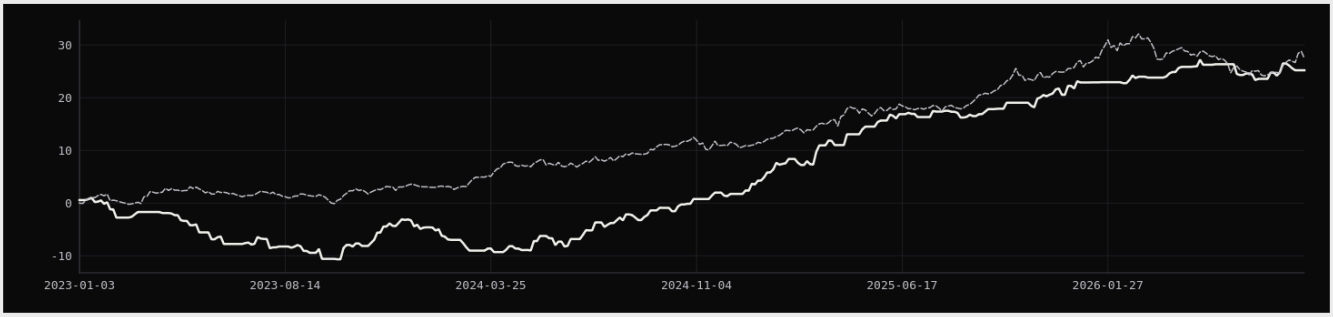
No se distingue de XAUUSD: el intervalo de la diferencia de Sharpe incluye el 0 — con estos días no se puede decir ni que lo bata ni que sea peor. Estrategia original, parámetros fijos, sólo el oos2 (2023-01-03 → 2026-08-28, 944 días).

Contra XAUUSD comprado

Sharpe anual sobre el P/L diario marcado al cierre de cada día. Intervalos al 95 % por bootstrap estacionario de días emparejados (bloques de un mes). «Sin la parte del activo»: lo que queda tras quitar lo que la estrategia gana sólo por estar expuesta a él; cerca de 0 = no aporta nada propio.

	Valor	Intervalo bajo	Intervalo alto
Sharpe de la estrategia	1.2	—	—
Sharpe de XAUUSD comprado	1.31	—	—
diferencia	-0.11	-1.45	1.1
beta frente a XAUUSD	0.093	—	—
Sharpe sin la parte de XAUUSD	0.82	-0.38	1.97

Estrategia y XAUUSD con el mismo riesgo



Año a año

Rentabilidad de cada año natural; XAUUSD escalado a la volatilidad de la estrategia.

Año	Estrategia %	XAUUSD %	Gana
2023	-3.3	3.6	no
2024	5.6	7.3	no
2025	20.6	14.9	sí
2026	2.3	1.8	sí

Bate al trader al azar en Sharpe: p = 0.0174. 10,000 traders con su misma huella — las mismas 214 operaciones, duración, tamaño, dirección y costes — que sólo cambian cuándo entran, sobre las barras reales del oos2.

Contra el trader al azar

p = parte de los traders al azar que igualan o superan a la estrategia; el menor observable es 1/(traders+1). En un activo que sube, el azar comprando también gana: por eso se mira además contra el activo. Reconciliación del motor con las operaciones reales: 0.99992; semilla 333911006782602338368170067988534443673.

Estadístico	La estrategia	Trader al azar medio	Al azar p95	% de traders al azar que supera	p
beneficio neto (USD)	29 226	11 320	54 183	75.1	0.2492
Sharpe por operación	0.188	0.035	0.154	98.3	0.0174
PF	1.625	1.175	1.773	90.9	0.09149
beneficio / DD	2.803	1.123	4.928	85.3	0.1469
DD máx. (USD, menos es mejor)	10 425	25 043	52 608	7.9	0.07929

Contra el activo comprado con el mismo riesgo. Las operaciones se marcan al cierre de cada día.

- **El recuadro**: verde si todo el intervalo de la diferencia de Sharpe está por encima de 0 (lo bate), rojo si todo está por debajo (es peor), amarillo si incluye el 0 (con esos días no se puede decir). El `oos2` es corto: lo normal es el amarillo.

- **Sharpe sin la parte del activo**: lo que gana la estrategia por sí misma, quitando lo que gana sólo por estar expuesta al activo. Cerca de 0 = no aporta nada que no dé comprar y esperar.
- **La curva y el año a año**: los dos, con el activo escalado a la volatilidad de la estrategia.

Contra el trader al azar. 10.000 traders con la misma huella que la original en el `oos2` —las mismas operaciones, duración, tamaño, dirección y costes— que sólo cambian **cuándo** entran, sobre las barras reales. Es el mismo motor del capítulo de los nulos (peldaño `timing`).

- **p** = qué parte de los traders al azar iguala o supera a la estrategia. El recuadro sale verde si p ≤ 0,05 en Sharpe. La tabla da los cinco estadísticos, cada uno con su p.
- En un activo que sube, el azar comprando también gana: por eso se miran las dos cosas. El activo dice si ganas más que estando expuesto; el azar, si tu momento de entrar vale algo.
- Si miras varias estrategias, alguna saldrá con p ≤ 0,05 por casualidad: con cinco, una de cada cuatro veces.
- Cada ejecución sortea traders nuevos (la semilla queda escrita bajo la tabla).

Necesita un export del 2026-10-01 en adelante (`main.parquet`); con uno anterior la pestaña no sale.

Un export anterior al 2026-10-01 sólo tiene 2 de las 10 condiciones; la ventana lo avisa en un recuadro amarillo. Vuelve a exportar con `python3 -m sqx.export.export_wfm` y salen las diez.

Un ejemplo completo

```
$ python3 -m studies.optimisation.wfm.report --project XAUUSD --databank WFM --day 2026-09-10
Strategy 1.19.29       blind     rho=+0.076 [-0.034, +0.193]  celdas= 30  deriva=70%
Strategy 4.33.46       perverse  rho=-0.505 [-0.683, -0.339]  celdas= 30  deriva=78%

-> /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/WFM/2026-09-21
```

Lectura: **ninguna de las dos estrategias mejora reoptimizándose.** `1.19.29` es ciega — la optimización no aporta información. `4.33.46` es peor que ciega: de sus 30 celdas, 28 dan correlación negativa, en las cuatro métricas. Con esos números, el walk-forward sobre esta estrategia no es un filtro de robustez: es una forma sistemática de elegir mal.

El informe empieza, antes de cualquier número, diciendo qué permite afirmar el export: cuántas celdas, que las ventanas de ejecución dentro de una celda **no se solapan**, que las de optimización se solapan un 79 %, y que todas las celdas reparten la misma historia. Eso está arriba a propósito: quien lea el ρ primero va a interpretar el intervalo como un error estándar, y no lo es.

Qué NO te dice

- **No dice si la estrategia gana dinero.** Dice si reoptimizarla ayuda.
- **No sabe lo que el optimizador descartó.** SQX guarda solo el ganador de cada tramo; las miles de combinaciones que probó dentro no se almacenan en ninguna parte. Así que no se puede saber si el segundo clasificado estaba pegado o lejísimos.
- **`perverse` tiene una explicación inocente que este módulo no puede descartar**: si el mercado alterna entre dos regímenes a un ritmo parecido al del tramo, optimizas en el régimen A y corres en el B, una y otra vez. Eso sería una propiedad del calendario, no de la estrategia. Para separarlo harían falta los regímenes clasificados o una estrategia de control conocida.
- **El intervalo es un suelo, no una medida.** Remuestrear celdas sigue tratando como independientes cosas que comparten historia. Lo que sostiene el veredicto no es el intervalo: es que el signo aguante en 28 de 30 celdas y en cuatro métricas.

- **`Ret/DD` crece con la longitud de la ventana**, y la matriz varía esa longitud a propósito en el eje `runs`. El informe avisa encima de esa tabla. Cualquier tendencia a lo largo de ese eje es en parte el propio eje.

Si algo falla

- **`IndexError` buscando el export** — no hay carpeta con `wfm/` dentro. Corre `export_wfm` antes, o revisa los guiones bajos del databank.
- **Un ρ que sale `nan`** — casi seguro que estás leyendo `Fitness`. SQX lo guarda a **cero en todos los tramos**; solo existe de verdad a nivel de celda. Usa otra métrica.
- **Una celda que no aparece en el informe** — tenía menos de 5 tramos utilizables tras quitar los que corren más allá de los datos. Es correcto: no se correlaciona.